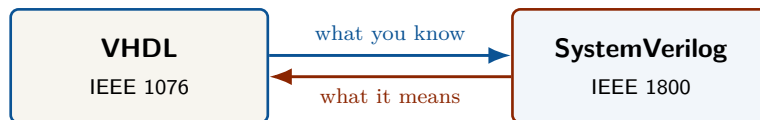


AN INTRODUCTION TO SYSTEMVERILOG

SystemVerilog for VHDL Designers

The Design Cookbook

From Entities to Modules · The Language, Side by Side



Luigi Giuffrida, Alessandra Dolmeta

Department of Electronics and Telecommunications
Politecnico di Torino

Draft — September 17, 2026

This booklet is one of four companions: Design (this one), Verification, Simulation, and a complete worked Example. It targets IEEE 1800-2017 (SystemVerilog) and IEEE 1076-2008 (VHDL) and covers only the design-side half of the language: everything that maps, construct for construct, onto what a VHDL designer already knows.

Preface

You have written VHDL for five years. You know what a signal is, why it is not a variable, what a delta cycle does to your simulation, and how to tell `numeric_std` exactly how many bits you meant. You have argued with a compiler that refused to add a `signed` to an `integer`, and you have usually turned out to be the one who was wrong.

Now somebody has told you the last course of your degree is in SystemVerilog.

The bad news first. SystemVerilog is not a tidier VHDL. It comes from a different tradition, it makes different assumptions about what a language is for, and several of the habits that made you a careful VHDL designer will make you a dangerous SystemVerilog one. The compiler that used to catch your mistakes will now accept them silently. A width mismatch that VHDL would refuse to analyse becomes a truncation that costs you an afternoon in the waveform viewer.

The good news is larger. Almost everything you know transfers. A process is an `always` block. A generic is a parameter. A package is a package. This booklet is the translation exercise, and you will move through it quickly.

What this booklet is

A cookbook, in the literal sense. It is organised so that you can look something up, find the VHDL you would have written next to the SystemVerilog you should write instead, and get on with your work. Almost every section is built around a side-by-side comparison:

VHDL	SystemVerilog
<pre>process (clk) is begin if rising_edge(clk) then q <= d; end if; end process;</pre>	<pre>always_ff @(posedge clk) begin q <= d; end</pre>

VHDL is always on the left, SystemVerilog always on the right. Where a construct has no counterpart in the other language, the booklet says so plainly rather than inventing one.

Three kinds of box interrupt the text. A *trap* marks something that specifically catches VHDL designers: not an obscure corner of the standard, but a mistake you are statistically likely to make in your first month. A *recipe* gives a pattern to copy. A plain note supplies background you can skip on a first reading.

What this booklet is not

It is not a language reference. IEEE 1800-2017 is 1315 pages long and this booklet is not a substitute for it. Where a construct has corners this text does not explore, it says so and points you at the standard.

It is not a synthesis manual. Everything presented as synthesisable is synthesisable with

mainstream tools, but the booklet does not discuss timing closure, physical design or vendor-specific attributes.

It is not advocacy. VHDL is an excellent language and, in several respects catalogued honestly at the end of Chapter 1, a better-designed one. The question this booklet answers is not which language is better but how to move between them without losing the discipline the first one taught you.

It is also not the whole book. This is the **Design Cookbook**, one of four companions covering SystemVerilog for VHDL Designers in full. Verification-only material (classes, randomisation, coverage, assertions, UVM) lives in the *Verification Cookbook*; Verilator, QuestaSim and Python testbenches live in the *Simulation Cookbook*; a complete worked design is in the *Example Cookbook*.

How this booklet is organised

Four chapters. Chapter 1 explains why the two languages feel so different, which is mostly a story about their simulation semantics and about what each one expects a compiler to do for you; it ends with an honest account of what VHDL does better. Read it first, even if you are tempted to skip to the syntax. Chapter 2 and Chapter 3 are the translation reference, split into the static half (files, ports, parameters, types, expressions) and the behavioural half (processes, statements, generate, instantiation, packages, time). Together they are the part of the booklet you will keep open beside your editor, and the complete mapping table at the end of Chapter 3 is the page to photocopy. Chapter 4 covers the design-side constructs SystemVerilog adds and VHDL has no answer to: interfaces, packed structures, parameterised types, bind.

Acknowledgement

The CORDIC rotator used as a running example throughout this booklet, and developed in full in the *Example Cookbook*, was chosen because it is small enough to read in one sitting and large enough to need a state machine, a parameterised datapath, a fixed-point format and a handshake protocol.

How to Read This Booklet

Typographic conventions

- Monospaced text is code: an identifier, a keyword, a file name or a command.
- Coloured monospaced text marks a language keyword under discussion.
- Green monospaced text is a file in the accompanying source tree.
- Sans-serif names are tools: QuestaSim, Verilator, cocotb, GTKWave.

The three kinds of box

△ Trap: What a trap looks like

A trap marks a mistake that VHDL experience makes you *more* likely to commit, not less. They are the part worth re-reading before your first design review.

Recipe: What a recipe looks like

A recipe is a pattern to copy. When the text says “write it like this”, this is the box it is in.

What a note looks like

A note supplies background: where a construct came from, what the standard actually says, why a tool behaves the way it does. Skip these on a first reading and come back when something surprises you.

A rule of thumb is one line you should be able to recite from memory.

If you have two weeks before the course starts

Read Chapter 1 carefully. Skim Chapter 2 once, without trying to memorise it, so that you know what is in it. Then move on to the *Verification Cookbook* for the testbench, and to the *Example Cookbook* to see it all used together.

If you are converting an existing VHDL design

Start at Chapter 2 and work through your source file construct by construct. Pay particular attention to the sections on widths and signedness, which is where mechanical translation goes

wrong. Then read Chapter 4 for the constructs that will let you delete half of what you just translated.

The companion booklets

This is the **Design Cookbook**. the *Verification Cookbook* covers everything that has no VHDL counterpart: classes, randomisation, coverage, assertions, UVM. the *Simulation Cookbook* covers Verilator, QuestaSim and Python/cocotb testbenches. the *Example Cookbook* works one CORDIC design through all of it.

Contents

Preface	v
How to Read This Booklet	vii
1 Two Languages, Two Philosophies	1
1.1 Two Origins, and Why They Still Matter	1
1.2 The Compiler Is Not Your Friend Any More	3
1.3 Design Units, Libraries and the Preprocessor	7
1.4 The Simulation Cycle: Where Your Intuition Breaks	9
1.5 Intent, Declared and Checked	14
1.6 Verification: the Bigger Break	16
1.7 The Contrasts in One Table	20
1.8 What Actually Changes in Your Habits	21
1.9 What VHDL has that SystemVerilog does not	22
1.10 Which language for which job	25
1.11 Checklist	26
2 The Rosetta Stone: Types and Expressions	29
2.1 Design units, files and the compilation model	29
2.2 Ports	33
2.3 Generics and parameters	36
2.4 Data types	38
2.5 Literals and aggregates	43
2.6 Operators, expressions and the width rules	45
2.7 Arrays, records and enumerations	49
2.8 Type conversion	54
2.9 Checklist	56
3 The Rosetta Stone: Behaviour	57
3.1 Concurrent statements	57
3.2 Processes and procedural blocks	60
3.3 Sequential statements	64
3.4 Generate	67

3.5	Instantiation	69
3.6	Subprograms	71
3.7	Packages and visibility	74
3.8	Attributes and system functions	76
3.9	Reporting, assertions and files	77
3.10	Time and delays	80
3.11	The complete mapping table	81
3.12	Checklist	84
4	Beyond VHDL: The Design-Side Additions	87
4.1	Interfaces	87
4.2	Clocking blocks	91
4.3	Declaring intent: <code>always_comb</code> , <code>always_ff</code> , <code>always_latch</code>	91
4.4	Packed structures and unions	93
4.5	Enumerations that carry a base type	95
4.6	Parameterised types and elaboration-time computation	96
4.7	<code>let</code> , <code>'define</code> , and the preprocessor	97
4.8	<code>generate</code> and <code>bind</code>	97
4.9	Checklist	99
A	Quick Reference	101
A.1	The ten things that will bite you first	101
A.2	Construct map	101
A.3	Format string cheat sheet	103
A.4	Severity and what it does	103
	Further Reading	105
	Index	107

Two Languages, Two Philosophies

This chapter answers one question: why does SystemVerilog feel wrong to someone who has written VHDL for five years? The answer is not syntax. The two languages were built for different purposes, by different communities, with different ideas about where correctness comes from. By the end of the chapter you will know where SystemVerilog puts the guard rails that VHDL puts in the type system, how the SystemVerilog event scheduler differs from the VHDL simulation cycle, why blocking and non-blocking assignments exist at all, and how the verification culture around SystemVerilog differs from the one around VHDL. The chapter ends with the other half of the argument: an honest account of what VHDL does better, and a set of criteria for choosing between the two.

You already know how to design digital hardware. Nothing in this book will teach you what a flip-flop is, what a handshake is, or why a long combinational path costs you frequency. What this book teaches is a second language for saying things you can already say, plus a set of things that the second language says which the first one cannot.

The temptation, in the first week, is to treat SystemVerilog as VHDL with different punctuation. Look up the keyword, substitute it, move on. That works for about two days. Then a design simulates correctly and synthesises into something else, or a testbench passes for the wrong reason, or a signal is one bit wide when you were certain you declared it as eight. At that point the missing knowledge is not a keyword. It is the model of the world that the language was built on. So this chapter is deliberately not a syntax table. Chapter 2 and Chapter 3 are the syntax tables. This chapter is about where the two languages come from, and which of your instincts have to be retrained before those tables are safe to use.

The order is: two origin stories, then the type system you are about to lose, then design units and the preprocessor, then the scheduler, which is the section that matters most, then declared intent, then the verification culture. The last two sections turn the argument round. Having spent the chapter on what SystemVerilog gives you, they close with what VHDL does better, which is more than a book with this title is normally willing to admit, and with criteria for deciding which language a given job wants. Read them: a designer who can only argue one way is not much use in a language review.

1.1 Two Origins, and Why They Still Matter

Languages carry their origins for decades. VHDL and Verilog were designed in the same half-decade, on the same continent, for the same industry, and they still ended up almost as different as two hardware description languages can be, because the problems their authors were paid to solve were not the same problem.

1.1.1 VHDL: a documentation language that learned to synthesise

VHDL began in 1981 as part of the United States Department of Defense Very High Speed Integrated Circuit (VHSIC) programme. The problem the DoD had was not simulation speed and it was not designer productivity. It was procurement. A military system built around a custom chip might have to be maintained, re-qualified, and possibly re-fabricated twenty or thirty years after the original vendor had lost interest or gone out of business. The DoD wanted a vendor-neutral, machine-readable, unambiguous description of a circuit that a different contractor could read, understand and re-simulate decades later.

That is a specification for a documentation language that happens to be executable. Everything about VHDL follows from it. The syntax was derived from Ada, which was itself the DoD's standard programming language and which had been designed by committee for long-lived, safety-critical, multi-author software. VHDL inherited Ada's verbosity, its strong typing, its explicit declarations, its named association, its packages and libraries, and its general belief that saying a thing twice in slightly different ways is a feature, because it lets the compiler check that you meant it.

The language was released publicly in 1985 and standardised as IEEE 1076-1987; the revisions you care about are IEEE 1076-1993, IEEE 1076-2008, which is what this book assumes, and IEEE 1076-2019, whose tool support is still thin. Note what is missing from that timeline: synthesis. VHDL was designed to be simulated, not to be turned into gates. Synthesis arrived from outside, in the early 1990s, as a commercially interesting subset that particular tools agreed to support. IEEE 1076.6 tried to standardise that subset and was eventually withdrawn. To this day the synthesisable part of VHDL is a convention enforced by tools, not a part of the language definition.

1.1.2 Verilog: a simulator that learned to be a language

Verilog has the opposite biography. Around 1984, Gateway Design Automation, a small company, sold a logic simulator called Verilog-XL. Verilog was its input format. Phil Moorby designed it, and he designed it for one purpose: to be fast to write, fast to parse, and fast to simulate on the hardware of 1984.

The language is therefore C-flavoured, terse, and deeply pragmatic. It has `begin/end` instead of Ada's fully spelled-out closers. It has a bit-vector type and an integer and very little else, because that is what an event simulator needs. It has four-valued logic with strengths, because a 1984 simulator had to model pull-ups, tri-states and buses at the transistor level. It has an explicit textual preprocessor, because C had one and it was cheap to implement. And most importantly, its semantics are the semantics of its simulator: the language was not specified first and implemented afterwards, it was implemented first and specified afterwards, from the behaviour of Verilog-XL.

Cadence acquired Gateway in 1989. Under commercial pressure from VHDL, which was an open standard, Cadence put Verilog into the public domain in 1990 and Open Verilog International standardised it; it became IEEE 1364-1995, then 1364-2001 (which added the things RTL designers actually needed, including ANSI-style port lists, `generate`, and signed arithmetic) and 1364-2005.

1.1.3 SystemVerilog: a verification language that swallowed its host

By the late 1990s Verilog was losing the verification argument. Testbenches were being written in dedicated verification languages – Synopsys Vera, Verisity's *e* – because Verilog had no classes,

no randomisation and no functional coverage. At the same time Co-Design Automation was developing Superlog, an extension of Verilog with C-like data types and higher-level constructs.

Accellera, an industry consortium, assembled SystemVerilog out of these donated pieces: Superlog for the design constructs and the type system, Vera for the object-oriented verification layer, and a separate assertion effort for SystemVerilog Assertions. SystemVerilog 3.0 (2002) was a design-side extension; SystemVerilog 3.1 (2003) added the verification half; Accellera SystemVerilog 3.1a became IEEE 1800-2005.

The decisive event is IEEE 1800-2009. Until then, SystemVerilog was formally defined as a set of extensions *on top of* IEEE 1364 Verilog, and you needed both documents. In 2009 the two standards were merged: IEEE 1800-2009 contains all of Verilog, and IEEE 1364 was withdrawn. Verilog is no longer a language with a standard of its own. It is a subset of SystemVerilog. The current revisions are IEEE 1800-2017, which this book targets, and IEEE 1800-2023.

What “the LRM” means

Engineers say *the LRM* for the Language Reference Manual: IEEE 1800 for SystemVerilog, IEEE 1076 for VHDL. IEEE 1800-2017 is about 1300 pages, roughly half of which describes things you will never synthesise. When this book says “the LRM requires”, you can check it: clause 4 is the scheduling semantics, clause 6 the data types, clause 9 the processes, clause 23 the modules and hierarchy.

1.1.4 The practical consequence

Here is the sentence to remember from this section. VHDL was designed as a language and later grew a synthesis subset. SystemVerilog was designed as a simulator input and later grew a type system.

Both statements have visible consequences in code you will write next week. Because VHDL was designed as a language first, its rules are uniform, checkable and occasionally infuriating: the analyser will reject a design that would have simulated perfectly well, because a rule says so. Because SystemVerilog grew outward from a simulator, its rules are permissive by default: almost anything you type has *some* defined meaning, so the tool accepts it and gives you that meaning, whether or not it is the one you had in mind. Neither approach is stupid. VHDL optimises for the case where the reader is not the author and the year is 2045; SystemVerilog optimises for the case where the author is at the terminal now and wants a waveform.

1.2 The Compiler Is Not Your Friend Any More

In VHDL you have an instinct, built over years, that goes like this: if it analyses and elaborates, the types are right. That instinct is correct in VHDL and it is worth a great deal. You cannot assign an `integer` to a `std_logic_vector`, you cannot add 1 to one without deciding in writing whether it is signed or unsigned, and you cannot connect a 12-bit port to a 16-bit signal. Each of those is a compile-time or elaboration-time failure, and each corresponds to a real design bug you therefore never shipped.

SystemVerilog does not work this way, and the single most dangerous thing a VHDL designer can do is carry the instinct across unchanged.

1.2.1 Widths do not have to match

In SystemVerilog, assignment between vectors of different widths is legal. The right-hand side is truncated from the most significant end if it is too wide, and extended if it is too narrow: with zeros if the expression is unsigned, by sign extension if it is signed. No warning is required by the LRM. Listing 1.1 shows the whole problem in six lines.

```

1 module width_demo;
2   logic [7:0] a;
3   logic [3:0] b;
4   logic [3:0] sum4;
5   logic [4:0] sum5;
6
7   initial begin
8     a = 8'hAB;
9     b = a;           // b = 4'hB : silently truncated
10    sum4 = 4'hF + 4'h1; // sum4 = 4'h0: the carry is lost
11    sum5 = 4'hF + 4'h1; // sum5 = 5'h10: operands widened to 5 bits
12    $display("b=%h sum4=%h sum5=%h", b, sum4, sum5);
13  end
14 endmodule

```

Listing 1.1. Width rules in action. Nothing here is an error.

Look carefully at the last two assignments, because they are the same expression producing two different results. SystemVerilog computes the width of an arithmetic expression from its *context*, and the assignment target is part of that context. In line 10 the target is four bits wide, so the addition is done in four bits and the carry out disappears. In line 11 the target is five bits wide, so both operands are first extended to five bits and the carry is kept. The expression did not change. The declaration of the variable you assigned it to changed the arithmetic.

VHDL has no equivalent behaviour because it has no equivalent hole. The corresponding code does not compile, and the fix is forced to be explicit.

VHDL	SystemVerilog
<pre> signal a : unsigned(7 downto 0); signal b : unsigned(3 downto 0); signal sum5 : unsigned(4 downto 0); -- b <= a; -- length mismatch: rejected b <= a(3 downto 0); sum5 <= resize(a(3 downto 0), 5) + to_unsigned(1, 5); </pre>	<pre> logic [7:0] a; logic [3:0] b; logic [4:0] sum5; // b = a; // legal, truncates assign b = a[3:0]; assign sum5 = 5'(a[3:0]) + 5'd1; </pre>

The VHDL side says `resize` because it must. The SystemVerilog side does not have to say anything, which is exactly why you should teach yourself to say something anyway: the cast `5'(...)` documents the intent and makes the widening visible to the next reader. Chapter 2 covers the width and signedness rules in full, including the difference between `»` and `»>`, which catches people at least as often.

△ Trap: Your ports do not have to match either

Port width mismatch at instantiation is also legal. Connecting a 16-bit signal to an 8-bit input port silently drops the top eight bits; connecting an 8-bit signal to a 16-bit port zero-extends it. In VHDL this is an elaboration error and your synthesis run stops. In SystemVerilog it is a Tuesday. This is the single most common cause of “it worked in the block-level testbench and broke in the system” among people new to the language.

Named port connections (`.data_i(my_data)`) protect you against wiring the wrong port; they do not protect you against wiring it at the wrong width.

1.2.2 Types that look familiar and are not

`logic` is the type you will use for essentially everything in RTL, and the first thing to say about it is that it is *not* `std_logic`. It is close: it is a four-valued type with the values 0, 1, `x` and `z`. But it has no strengths. `std_logic` has nine values, including the weak strengths L, H, W and the don't-care `-`. Those extra values matter in two places you have probably used them: pull-up modelling, and `-` in a `case` choice or a comparison, which is a trap in VHDL anyway because `-` does not mean don't-care to the `=` operator. SystemVerilog has no `-`. Its don't-care in a `case` statement is a different mechanism (`casez`, or better, the `unique` and `priority` qualifiers), covered in Chapter 3.

The second thing to say about `logic` is subtler and more important. `logic` is not a wire. In original Verilog there were two families, *nets* (`wire`, `tri`, `wand`) which model connections and can have several drivers resolved together, and *variables* (`reg`) which model storage inside a process and may have only one driver. `logic` is simply a better name for `reg`: it is a single-driver, four-valued variable that you may also drive with a single continuous assignment. If you genuinely need multiple drivers resolved – a tri-state bus – you need `wire`. The VHDL habit of writing `std_logic` everywhere and relying on the resolution function translates to `logic` everywhere and *not* relying on resolution, because `logic` will not resolve; a second driver is an error.

Then there are the two-state types, which have no VHDL counterpart at all and are a genuine trap.

△ Trap: `int` is not integer, and it hides your reset bugs

SystemVerilog has two-state types: `bit` (1 bit), `byte` (8), `shortint` (16), `int` (32), `longint` (64). Two-state means the variable can only hold 0 or 1. It cannot hold `x`. It initialises to 0, not to `x`.

That last sentence is the trap. A VHDL designer relies on a signal starting at 'U' or 'X' and propagating that unknown through the design until reset drives it: a red waveform after reset is how you find the register you forgot to reset. Declare that register as `int` or `bit` and it starts at zero, the simulation looks clean, and the silicon powers up arbitrarily. Use `logic` for anything that is hardware. Keep the two-state types for loop counters, testbench bookkeeping and class members, where the speed and the absence of `x` are what you want.

Note also that SystemVerilog's `int` is 32-bit and signed by definition, whereas VHDL's `integer` is a 32-bit signed type whose range is defined by the standard but whose subtype you are expected to constrain. There is no SystemVerilog equivalent of `natural` or of `range 0 to 15`, and therefore no runtime range check.

1.2.3 Names you never declared

Verilog inherited from its simulator days a rule that would be unthinkable in VHDL: some undeclared identifiers are not errors. If a name that has never been declared appears in an instance port connection, or on the left-hand side of a continuous assignment, the tool creates a one-bit net called that, and carries on.

```

1 module counter (input logic clk, output logic [3:0] cnt_o);
2   logic [3:0] cnt_d, cnt_q;
3
4   assign cnt_d = cnt_q + 4'd1;
5
6   sink u_sink (.d_i(cnt_qq), .q_o(cnt_o)); // typo: cnt_qq never declared
7
8   always_ff @(posedge clk) cnt_q <= cnt_d;
9 endmodule

```

Listing 1.2. A typo that costs you an afternoon

Listing 1.2 compiles. `cnt_qq` becomes a one-bit implicit net, driven by nothing, so it is `z`. The port it feeds is four bits wide, so the one-bit net is padded with zeros and the submodule sees `4'b000z`: the top bits are a clean 0 and only bit 0 carries the `z`. That is the reason the bug is hard to see. A wholly undriven signal shows up as `zzzz` in the waveform and in every print; `000z` looks almost like data, and any arithmetic on it quietly turns into `x`. In VHDL this is one line of error output naming the undeclared identifier.

The remedy is not optional. Put the directive ``default_nettype none` at the top of every file: it switches implicit net creation off, and the typo above becomes the compile error it should always have been.

Recipe: The two lines that go in every file

Begin every `.sv` file with ``default_nettype none` and end it with ``default_nettype wire`. The closing line matters because compiler directives are not scoped to a file: they persist through the whole compilation, so a file that switches the default off and does not switch it back will break the next file the tool reads, which may be somebody else's IP.

```

1 `default_nettype none
2
3 module fifo #(parameter int unsigned DEPTH = 8) (
4   input wire logic clk_i,
5   input wire logic rst_ni,
6   // ...
7 );
8   // ...
9 endmodule
10
11 `default_nettype wire

```

Once implicit nets are off, undeclared input ports also become errors, which is why the ports above are written `input wire logic`: with the default net type set to `none` you must say what kind of thing an input port is.

1.2.4 The linter is the new type checker

Put the previous three subsections together and a picture emerges. SystemVerilog will not stop you from writing a width mismatch, an inferred latch, an incomplete sensitivity list, an unconnected port, a multiply-driven signal or an undeclared name. The language does not consider these to be its problem.

The industry's answer is a lint tool, run automatically, treated as a gate. The cultural position that VHDL gives to the analyser, SystemVerilog gives to the linter. This is not a workaround adopted by sloppy teams. It is the mainstream professional practice, and every serious SystemVerilog project you will join has it wired into continuous integration.

Concretely, the free option is Verilator, which is a very good linter even if you never use it as a simulator:

```
verilator --lint-only -Wall --timing -f files.f --top-module cordic_rot
```

-Wall turns on the checks that are off by default, including the width-mismatch warnings (WIDTHTRUNC and WIDTHEXPAND in Verilator 5; older versions call both of them WIDTH), UNUSED, UNDRIVEN, CASEINCOMPLETE, LATCH and BLKSEQ. Commercial linters (Spyglass, AscentLint, Questa AutoCheck) do the same job with more rules and a rule-set configuration you inherit from your employer. the *Simulation Cookbook* sets Verilator up properly.

The other half of the answer is a written coding standard. The best public one is the lowRISC style guide, used by the OpenTitan project; it is freely available, it is short, and its rules are exactly the rules that stop the failures described in this section. It is worth reading in full before you write your first thousand lines. Google’s internal Verilog style guide and the ETH Zurich / PULP platform conventions are close relatives.

Replace the instinct “if it compiles, the types are right” with “if the linter is clean, the widths are right”. A SystemVerilog compile that produces no output has told you almost nothing.

1.3 Design Units, Libraries and the Preprocessor

VHDL has five kinds of design unit: entity, architecture, configuration, package and package body. Three of them are primary (entity, package, configuration) and two are secondary (architecture, package body), meaning they cannot be analysed until their primary exists. Each analysed unit is deposited into a *library*, which is a real directory of compiled objects on disk, and the default library is called *work*. Referring to something in another library requires you to say so twice, once to make the library visible and once to make the name visible:

VHDL

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

library work;
use work.cordic_pkg.all;

entity cordic_rot is
  generic (ITER : positive := 14);
  port (
    clk_i  : in  std_logic;
    rst_ni : in  std_logic;
    -- ...
  );
end entity cordic_rot;

architecture rtl of cordic_rot is
  signal x_q : signed(15 downto 0);
begin
  -- ...
end architecture rtl;
```

SystemVerilog

```
import cordic_pkg::*;

module cordic_rot #(
  parameter int unsigned ITER = 14
) (
  input logic clk_i,
  input logic rst_ni
  // ...
);
  data_t x_q;
  // ...
endmodule
```

The structural differences are worth spelling out.

One unit, not two. A module is entity and architecture in one. The interface and the body cannot drift apart, and there is no way to compile two architectures of the same entity and

pick between them at elaboration. The VHDL configuration has no everyday counterpart: SystemVerilog inherits a `config/library` mechanism from Verilog-2001, but it is rarely used, and parameterisation plus ``ifdef` do the same job. If you used configurations to swap a behavioural model for a gate netlist, in SystemVerilog you do it by choosing which files you compile.

No library clause. SystemVerilog module names live in a single global namespace. If two modules called `fifo` are compiled into the same design, that is a collision, and the usual fix is a project-wide name prefix, exactly as in C. What SystemVerilog has instead of libraries is the *compilation unit*. Everything declared outside any module lands in a scope called `$unit`. Whether `$unit` covers one file or all the files on the command line is, by the LRM, tool-dependent.

△ **Trap: Never declare anything at compilation-unit scope**

Because `$unit` may be one file for one tool and all files for another, a `typedef` or a `parameter` written outside a module compiles under Verilator and fails under VCS, or vice versa, or, worse still, compiles under both and means different things. Put every shared declaration in a `package` and import it. There is no VHDL habit that leads you into this, but there is a C habit that does, and SystemVerilog looks enough like C to tempt you.

Packages exist and behave nearly as you expect. A SystemVerilog package holds `typedefs`, `parameters`, `functions`, `tasks` and `class declarations`. Import it with `import cordic_pkg::*`; for everything or `import cordic_pkg::data_t`; for one name, and refer to individual names explicitly with `cordic_pkg::data_t`. The scope-resolution operator `::` is the counterpart of VHDL's selected name `work.cordic_pkg.data_t`. A package must be compiled before anything that imports it, exactly as in VHDL. There is no package body: the implementations go in the package itself.

And there is a preprocessor. This is the piece with no VHDL analogue whatsoever. Verilog inherited from C a purely textual substitution pass that runs before the language is parsed: ``define` creates a macro, ``ifdef`/`ifndef`/`else`/`endif` conditionally include text, and ``include` pastes one file into another.

```

1  `ifndef CORDIC_DEFS_SVH
2  `define CORDIC_DEFS_SVH
3
4  `define CORDIC_MAX_ITER 16
5  `define ASSERT(name, prop) name: assert property (prop)
6
7  `ifdef SYNTHESIS
8  `define SIM_ONLY(x)
9  `else
10 `define SIM_ONLY(x) x
11 `endif
12
13 `endif

```

Listing 1.3. Compiler directives: text substitution, not language

What this buys you is real. Conditional compilation lets one source tree serve simulation, synthesis, formal and emulation, with the assertion-heavy parts excluded from the flows that cannot take them, and it lets you define something once and use it in a hundred files without a package import. VHDL has no equivalent until the conditional analysis directives of VHDL-2019, whose tool support is thin enough that you should not plan around it.

What it costs you is also real. Macros have no types, no scope and no namespace; one defined anywhere is visible everywhere after that point, and redefining it is legal. They are invisible to the elaborated design, so no waveform or hierarchy browser will show you one. The universal

conventions are to prefix every macro name with the project name, to guard every included file as in Listing 1.3, and to prefer a package `parameter` to a ``define` whenever a parameter will do. Use the preprocessor for what must happen before parsing, and for nothing else.

``include` is not `use`

``include` is textual. It does not create a scope, does not check for duplicate definitions, and can be placed inside a module body, inside a port list, or anywhere else the resulting text would be legal. VHDL's `use` is a name-visibility statement about a unit that has already been analysed. The two look similar in a file header and have nothing in common. If a colleague tells you to “just include the package”, they mean import it.

1.4 The Simulation Cycle: Where Your Intuition Breaks

This is the most important section in the chapter. Everything above is a matter of discipline; this is a matter of semantics. If you carry your VHDL scheduling model into SystemVerilog unchanged, you will write designs that simulate differently on two simulators and blame the tools.

1.4.1 What VHDL does

You know this already, but state it precisely, because the precision is the point of comparison.

A VHDL simulation cycle has two phases in a fixed order. First the *update* phase: every signal with a transaction scheduled for the current time T_c takes its new value, and if the new value differs from the old one, an event is posted on that signal. Second the *evaluate* phase: every process sensitive to a signal that had an event resumes, and runs until it suspends again. Assignments a process makes to signals do not take effect during the evaluate phase. They schedule transactions, and the earliest a transaction can be scheduled for is T_c plus one *delta* – an infinitesimal step that does not advance simulated time.

Then the simulator computes the next time T_n . If $T_n = T_c$, because somebody scheduled a delta-delayed transaction, the whole cycle repeats at the same simulated time. That is a delta cycle. Time only advances when no transaction remains scheduled for the current time.

Two consequences follow, and both are why VHDL feels safe.

First, *a signal read inside a process always returns the value it had at the start of the current cycle*, no matter what the process assigned to it three lines earlier. That is a rule of the language, not a convention. A variable behaves the other way round: it updates immediately, because it is not visible outside the process and needs no scheduling discipline.

Second, and this is the important one, *a clocked VHDL process is race-free by construction*. Consider two processes, both sensitive to `clk`, one assigning `q1 <= d` and the other `q2 <= q1`. Both resume in the evaluate phase of the same cycle. Both read the pre-update values. Both schedule for the next delta. The order in which the simulator happens to run them cannot change the result, so a shift register is a shift register regardless. You have never had to think about this, and that is the achievement.

Postponed processes

VHDL also has **postponed process**, which runs once at the end of the last delta cycle of a time step, when no further delta activity remains. It exists for monitoring and reporting code that must see the settled value rather than an intermediate delta. Keep it in mind: SystemVerilog has a region with exactly this purpose, and it is even called Postponed.

1.4.2 What SystemVerilog does

SystemVerilog does not have a two-phase cycle. It has a *time slot* containing nine regions, executed in order, with two loops that can send execution back to an earlier region. The regions are Preponed, Active, Inactive, NBA, Observed, Reactive, Re-Inactive, Re-NBA and Postponed. Figure 1.1 puts the two models side by side.

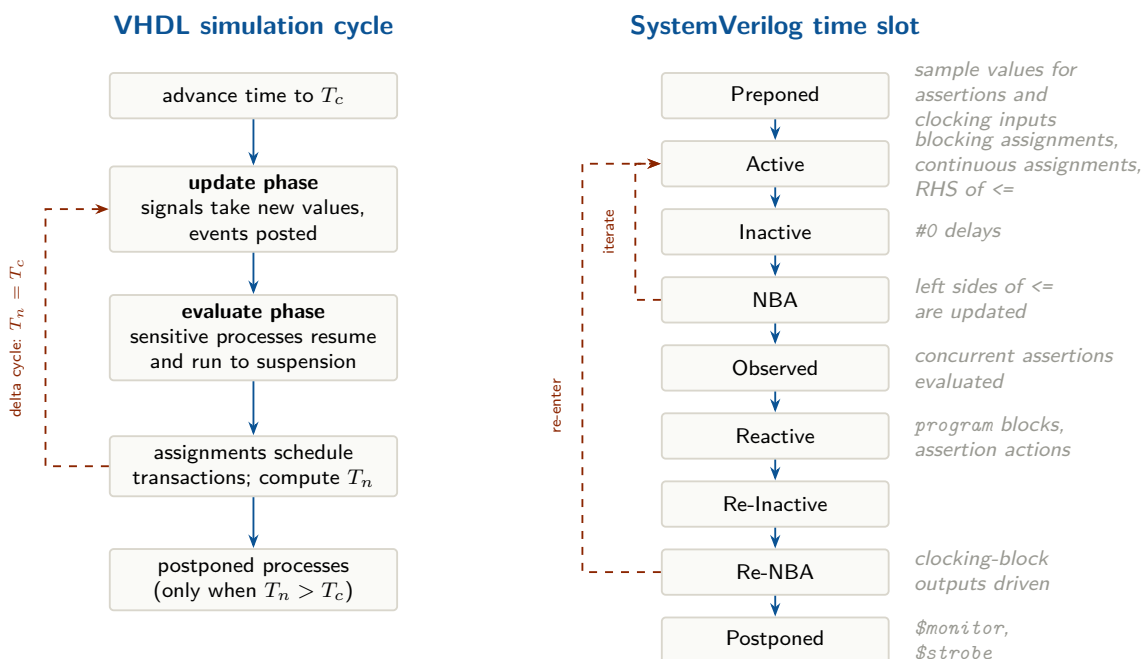


Figure 1.1. The VHDL simulation cycle and the SystemVerilog time slot. VHDL separates update from evaluate once, and repeats the pair in delta cycles. SystemVerilog splits the same job across nine regions and iterates between them until no events remain.

Region by region, in the terms you already have:

Preponed

Nothing changes value here. Values are *sampled* for concurrent assertions and for clocking-block inputs. This is what **#1step** means: sample as the value was immediately before this time slot.

Active

Where ordinary procedural code runs. Blocking assignments take effect, continuous assignments are evaluated, $\$display$ prints, and the *right-hand sides* of non-blocking assignments are evaluated and queued. Processes here execute in an order the LRM deliberately leaves unspecified.

Inactive

Processes that executed an explicit $\#0$ delay.

NBA

The non-blocking assignment queue is applied: the left-hand sides now take the values that were computed in Active. This is VHDL's update phase.

Observed

Concurrent assertion properties are evaluated, using the values sampled in Preponed.

Reactive

Testbench code in a `program` block, and the action blocks of assertions.

Re-Inactive, Re-NBA

The same as Inactive and NBA, for processes in the reactive set. Clocking-block outputs are driven in Re-NBA.

Postponed

`$monitor` and `$strobe`. Read-only: nothing may change a value here.

The loops matter as much as the regions. After NBA, if anything that just happened created new active events, execution returns to Active and the Active/Inactive/NBA group repeats until all three are empty. This iteration is the exact counterpart of a chain of VHDL delta cycles. There is a second loop from the reactive group back to Active, so a testbench that drives a signal can wake design code inside the same time slot.

The regions the LRM lists that this book ignores

IEEE 1800-2017 Figure 4-1 actually shows more than nine regions: Pre-NBA, Post-NBA, Post-Observed, Pre-Re-NBA and Post-Re-NBA exist as well. They are reserved for PLI and VPI callbacks, so that a C model attached to the simulator can run at a defined point relative to the language's own events. No SystemVerilog you write ever executes in them, and they are omitted from Figure 1.1 for clarity.

1.4.3 Blocking, non-blocking, and why Verilog needed them

Now the pieces join up. SystemVerilog has two procedural assignment operators:

- `=`, the *blocking* assignment. It evaluates its right-hand side and updates its target immediately, in the Active region, before the next statement in the process runs. It behaves like a VHDL [variable](#).
- `<=`, the *non-blocking* assignment. It evaluates its right-hand side in the Active region, queues the result, and updates its target later, in the NBA region. It behaves like a VHDL [signal](#) with a delta delay.

That is the whole mapping, and it is exact enough to rely on:

VHDL	SystemVerilog
variable <code>v</code> : ...; <code>v := expr</code> ;	logic <code>v</code> ; <code>v = expr</code> ;
signal <code>s</code> : ...; <code>s <= expr</code> ;	logic <code>s</code> ; <code>s <= expr</code> ;

Here is the historical point that makes the whole design comprehensible. VHDL never needed to invent the non-blocking assignment, because the signal *already* had a delta-delayed update built into its definition: there is only one kind of assignment to a signal, and it is always deferred. Verilog started from the opposite place, with an immediate procedural assignment and `always` blocks scheduled in an unspecified order, so writing a shift register was genuinely ambiguous.

The non-blocking assignment was added to Verilog-XL, and standardised in IEEE 1364-1995, precisely to recover the discipline VHDL had from birth: evaluate everything, then update everything.

Look at the same register in both languages, and read the right-hand pane as “the <= is doing what your VHDL signal did automatically”:

VHDL	SystemVerilog
<pre>regs : process (clk_i, rst_ni) is begin if rst_ni = '0' then q <= (others => '0'); elsif rising_edge(clk_i) then q <= d; end if; end process regs;</pre>	<pre>always_ff @(posedge clk_i or negedge rst_ni) begin if (!rst_ni) q <= '0; else q <= d; end</pre>

Eight lines against five, and the five say the same thing. The '0 is SystemVerilog's unsized fill literal, the counterpart of (others => '0'): it fills the target with zeros whatever its width. Note that the SystemVerilog sensitivity list is `negedge rst_ni`, not the plain `rst_ni` of the VHDL process. SystemVerilog requires every term of an `always_ff` event control to be edge-qualified, and it needs the polarity to know which edge starts the asynchronous reset. VHDL works out the same thing from the `if` statement.

1.4.4 The classic race

Now the failure that this machinery exists to prevent. Listing 1.4 is a two-stage shift register written with blocking assignments.

```
1 module shift_bad (input logic clk, input logic d, output logic q2);
2   logic q1;
3   always_ff @(posedge clk) q1 = d;    // blocking: wrong
4   always_ff @(posedge clk) q2 = q1;  // blocking: wrong
5 endmodule
```

Listing 1.4. A race. This design has two legal outcomes.

Both processes wake in the Active region of the same time slot. The LRM does not say which runs first. If the first runs first, `q1` takes the value of `d` immediately, then the second reads the *new* `q1` and `q2` follows `d` in a single cycle: one flop, not two. If the second runs first, `q2` takes the old `q1` and you get the two-stage shift register you intended. Both are legal. Two simulators may disagree; one simulator may disagree with itself after you add an unrelated module and change its internal scheduling order. Synthesis, meanwhile, will happily produce two flip-flops, so the gate-level netlist need not match the RTL simulation.

△ Trap: The failure mode you have never had to think about

In VHDL you could write the two processes in either order, in either file, and get the same answer every time, because signal semantics forbid the race. In SystemVerilog the ordering of `always` blocks within the Active region is explicitly unspecified, so the race is real and the language will not warn you. Change one character in Listing 1.4:

```
1 always_ff @(posedge clk) q1 <= d;
2 always_ff @(posedge clk) q2 <= q1;
```

Now both right-hand sides are evaluated in Active, using the old values, and both targets

are updated in NBA. The order of the two blocks is irrelevant. This is why the rule exists, and it is why Verilator's BLKSEQ warning and every commercial lint rule set flag a blocking assignment inside a clocked block as an error, not a style issue.

The rule that follows is the first thing anyone will tell you about SystemVerilog, and now you know why it is true rather than merely being told it.

Non-blocking (<=) for everything a clock edge writes. Blocking (=) for everything combinational. Never mix the two in one `always` block, and never write the same variable from two blocks.

The reasoning for the combinational half is the mirror image. Inside an `always_comb`, a blocking assignment behaves like a VHDL variable in a combinational process: intermediate results are visible to later statements in the same execution, which is exactly what you want when you are describing a chain of logic. Using `<=` there would push the update into the NBA region, so a later statement in the same block would read the stale value, and the block would then have to be re-executed, producing extra delta activity, slower simulation and, in some cases, a mismatch with synthesis.

1.4.5 Testbenches race too

The design-side race has a clean rule. The testbench-side race does not, and it needs language support.

Consider a testbench driving a synchronous input:

```
1 initial begin
2   @(posedge clk);
3   data = 8'h5A;    // Active region, same time slot as the DUT's clock edge
4 end
```

The DUT samples `data` on the same edge, and the testbench writes it in the Active region of that same time slot, so whether the DUT sees the old value or the new one depends on which process the simulator happens to run first. The “use non-blocking” rule does not fully save you here, because the testbench also wants to *read* DUT outputs at the clock edge, and reading has the same ambiguity. Verilog testbench writers solved this by hand for twenty years, inserting small delays after the clock edge. SystemVerilog solved it in the language, with the *clocking block*: a declaration that names a clock and states, once, when its signals are sampled and when they are driven.

```
1 module tb;
2   logic      clk = 1'b0;
3   logic      vld, ready;
4   logic [7:0] data;
5
6   always #5ns clk = ~clk;
7
8   clocking cb @(posedge clk);
9     default input #1step output #1ns;
10    output vld, data;
11    input  ready;
12  endclocking
13
14  initial begin
```

```

15   @(cb);
16   cb.vld  <= 1'b1;           // driven in Re-NBA, 1 ns after the edge
17   cb.data <= 8'h5A;
18   @(cb);
19   if (cb.ready) $display("accepted"); // sampled 1step before the edge
20   end
21 endmodule

```

Listing 1.5. A clocking block removes the testbench race by construction

input `#1step` means every clocking-block input is sampled in the Preponed region, before any design code in that time slot has run, so it can never see a value the DUT produced at this very edge. output `#1ns` means every drive happens 1 ns after the edge, safely inside the cycle. The race is gone by construction, and it stays gone when somebody adds another `always` block to the design six months later. the *Verification Cookbook* builds testbenches on this foundation.

Whatever happened to program blocks?

SystemVerilog 3.1 introduced the `program` block, borrowed from Vera: a container whose code runs in the Reactive region, which is after all design code in the same time slot, so testbench statements could never race with the design. It solved a real problem, but it also brought restrictions (no `always` blocks inside, particular termination semantics) and it was largely superseded by clocking blocks, which fix the same races without constraining the testbench structure. UVM does not use `program` blocks. You will meet them in older code and in textbooks; you do not need to write them.

△ Trap: after is not

VHDL's `s <= v after 5 ns` schedules a transaction, with inertial delay: a pulse shorter than 5 ns is swallowed. SystemVerilog's `#` is not a scheduling annotation on an assignment, it is a statement delay, and its two placements mean different things. `a = #5 b` evaluates `b` now, suspends the process for 5 ns and then assigns; the process is blocked for those 5 ns. `a <= #5 b` evaluates `b` now and schedules the update for 5 ns later without blocking. Neither is inertial. And in synthesisable RTL you should write no `#` delays at all – the closest SystemVerilog gets to VHDL's inertial behaviour is on gate primitives and continuous assignments, which is not where you will be working.

1.5 Intent, Declared and Checked

SystemVerilog gave away the type safety that VHDL has. It got something back that VHDL does not have, and the trade is more even than it first looks. In VHDL a process is a process: whether it describes a flip-flop, combinational logic or a latch is something the reader, the simulator and the synthesiser each work out from the shape of the code inside it. There is no place to write down which of the three you meant, so no tool can tell you that you wrote something else. Every VHDL designer has produced a latch by forgetting an `else` in a combinational process, and the analyser said nothing, because nothing was wrong with the *types*.

SystemVerilog-2005 added three specialised process keywords whose only purpose is to state intent so that a tool can check it.

- `always_ff` declares sequential logic. Its event control must contain only edge-qualified terms; the variables it writes may not be written anywhere else; and tools check that what

it describes is in fact a flip-flop.

- `always_comb` declares combinational logic. Its sensitivity list is inferred from everything the block reads, *including inside functions it calls*; the variables it writes may not be written anywhere else; and a tool will warn if the code as written infers a latch. It also executes once at the end of time zero, whether or not any input changed, so the outputs are consistent from the start of the simulation.
- `always_latch` declares a latch on purpose, so that a real latch does not look like a mistake.

VHDL

```
-- intent is not stated; a missing
-- else silently infers a latch
comb : process (all) is
begin
    if sel = '1' then
        y <= a;
    else
        y <= b;
    end if;
end process comb;
```

SystemVerilog

```
// intent is stated and checked
always_comb begin
    if (sel) y = a;
    else    y = b;
end
```

The VHDL-2008 `process (all)` form removes the incomplete sensitivity list problem, which is the other half of the same class of bug, so that part is a draw. What VHDL still has no way to express is the sentence “this block is combinational, please complain if it is not”.

Argue it this way. VHDL applies static checking to *data*: what a value is, how wide it is, what may be assigned to what. SystemVerilog applies static checking to *structure*: what kind of hardware a block of code describes, and whether anything else drives the same variable. Neither language checks what the other checks. A VHDL designer arriving in SystemVerilog should use `always_ff` and `always_comb` everywhere with the same reflex that made them declare a signal as `unsigned(11 downto 0)` rather than `std_logic_vector`: it is free, and it converts a class of silent bugs into a message.

Recipe: Naming, and why the industry cares about it

SystemVerilog has no reserved suffix conventions, so a project’s naming rules do a job that the language does not. Two notes before the list. These rules apply to design code you intend to keep; the small throwaway examples in this book — a counter, a FIFO, a shift register — drop the suffixes so that the line under discussion fits in a narrow column, and the CORDIC sources of the *Example Cookbook* follow the rules in full. And the rules themselves matter less than picking one set and holding to it. The set below comes from the lowRISC guide and is close to universal in modern SystemVerilog:

- `_i`, `_o`, `_io` on module ports for input, output and bidirectional. The direction is then visible at every use, not only at the declaration.
- `_n` for active-low, always as the last element before the direction: a reset input is `rst_ni`.
- `_d` and `_q` for the input and the output of a register. `cnt_d` is what will be stored, `cnt_q` is what is stored. An `always_ff` then contains nothing but `cnt_q <= cnt_d`; and all the logic lives in an `always_comb` that computes `cnt_d`.
- Lower case with underscores for signals and instances, upper case for parameters and macros, `_t` for a typedef, `_e` for an enum, `_pkg` for a package.

The `_d/_q` convention deserves a specific defence: it makes the register boundary visible

in the source. In VHDL you can usually see it because the clocked process is a separate lump of text; in a language where `always_ff @(posedge clk_i) x <= y;` is one line, you need the names to carry the information.

Recipe: One clock, one reset, one `always_ff`

Give every `always_ff` exactly one clock and at most one asynchronous reset. Do not put logic for two clock domains in one block, do not gate a clock inside the event control, and do not use a signal as both a clock and data. Clock-domain crossings get their own dedicated modules with their own lint and CDC checks. This is the same discipline you already apply in VHDL; SystemVerilog simply makes it easier to violate, because the terse syntax invites you to put more into one block.

1.5.1 Brevity is a feature

The register comparison above was eight lines against five. That ratio holds across most RTL, and it is not only about typing speed. Verbosity has a real cost: a design that fits on one screen can be checked by eye, and a design that takes three screens cannot. When the same finite-state machine is 120 lines instead of 300, more of it is visible at once and structural mistakes stand out.

The savings come from several places at once: no separate entity, no `architecture` wrapper, no declarative region separate from the body, no explicit type conversions, `begin/end` omitted for a single statement, ANSI port lists with types inline, and the `'0` fill literal. Nothing on that list is free, since every item is an explicitness you have given up, but the aggregate effect on readability is positive and it is one of the honest reasons the industry moved.

1.6 Verification: the Bigger Break

Everything so far has been about writing hardware. The larger difference between the two ecosystems is about proving that the hardware is right, and here the gap is not a matter of syntax at all. It is a different theory of what a testbench *is*.

1.6.1 The VHDL testbench you already write

In VHDL, a testbench is another design unit. It is an entity with an empty port list and an architecture that instantiates the device under test and drives it from one or more processes. The stimulus is procedural, directed and written by hand: assert reset, wait, apply a value, wait for a clock edge, check the output with `assert ... report ... severity ...`. Reference data comes from a file read with `TEXTIO` or from a function written in the testbench.

```

1 stim : process is
2 begin
3     rst_ni <= '0';
4     wait for 40 ns;
5     rst_ni <= '1';
6     wait until rising_edge(clk_i);
7
8     req_x_i    <= to_signed(16#2000#, 16);
9     req_y_i    <= (others => '0');
```

```

10 req_valid_i <= '1';
11 wait until rising_edge(clk_i) and req_ready_o = '1';
12 req_valid_i <= '0';
13
14 wait until rising_edge(clk_i) and rsp_valid_o = '1';
15 assert abs(to_integer(rsp_x_o) - 11585) < 4
16     report "cos result out of tolerance" severity error;
17
18 std.env.stop;
19 end process stim;

```

Listing 1.6. The shape of a directed VHDL testbench

There is nothing wrong with this. It is readable, deterministic and easy to debug, and for a small block with a handful of interesting cases it is the right answer. Its limitation is arithmetical rather than aesthetic: you verify exactly the cases you thought of, and the number of cases you can hand-write does not grow with the size of the design, while the number of cases the design can get wrong does. The language gives you nothing beyond this: VHDL has no classes, no inheritance, no constrained randomisation and no functional coverage constructs, only `ieee.math_real.uniform`, which is a random number generator rather than a stimulus generator.

1.6.2 Verification as a software problem

The SystemVerilog verification half of the language starts from a different premise: verifying a large design is a software engineering problem, so give the verification engineer software engineering tools. Four ideas follow, and each one changes what a testbench looks like. The constructs behind them are the *Verification Cookbook*; what follows here is only the argument for why they exist.

Transactions instead of pin wiggles. You stop writing values onto individual signals and start describing a *transaction*: a read of four bytes from address 0x40, a rotation of a vector by an angle. A separate component, the driver, knows how to turn a transaction into pin activity for a particular protocol. The test then says what should happen, not how the wires move, and the same test survives a change of bus protocol.

Constrained random instead of directed vectors. Instead of enumerating inputs, you describe the *legal input space* and let the tool draw from it. This is the deepest change of habit in the whole book.

Functional coverage as the exit criterion. If the stimulus is random, “all my tests passed” means nothing on its own, because you no longer know what was tried. So you write down, separately and explicitly, the situations that must occur, and the simulator counts them. A run is worth something when the coverage model says the interesting cases happened.

Object-oriented reuse. Drivers, monitors, scoreboards and sequences are classes, and a new project extends them rather than copying them. This is what UVM is: not an extension of the language but a class library written in SystemVerilog, which the *Verification Cookbook* takes apart.

Listing 1.7 is a minimal version of the first three ideas.

```

1 class rot_req;
2     rand bit signed [15:0] x, y;
3     rand bit signed [15:0] theta;
4     rand bit                small_angle;
5
6     // the legal input space, written once

```

```

7  constraint c_range {
8      x inside {[-16384:16383]};
9      y inside {[-16384:16383]};
10     small_angle -> theta inside {[-1024:1023]};
11 }
12 endclass
13
14 covergroup cg_req with function sample(bit signed [15:0] theta);
15     cp_theta : coverpoint theta {
16         bins neg_large = {[-32768:-8193]};
17         bins neg_small = {[-8192:-1]};
18         bins zero      = {0};
19         bins pos_small = {[1:8192]};
20         bins pos_large = {[8193:32767]};
21     }
22 endgroup

```

Listing 1.7. Constraints describe the legal space; coverage measures what happened

The constraint block is declarative. It does not say what to do; it says what is true of every legal input. Calling `req.randomize()` asks a constraint solver for a value satisfying all of it, and calling it ten thousand times with different seeds explores parts of the space you would never have enumerated by hand. The `covergroup` then records what actually came out, so you can tell the difference between “ten thousand runs” and “ten thousand runs that all did the same thing”.

1.6.3 The metric-driven loop

The two flows differ in shape, not only in tooling. Figure 1.2 draws them.

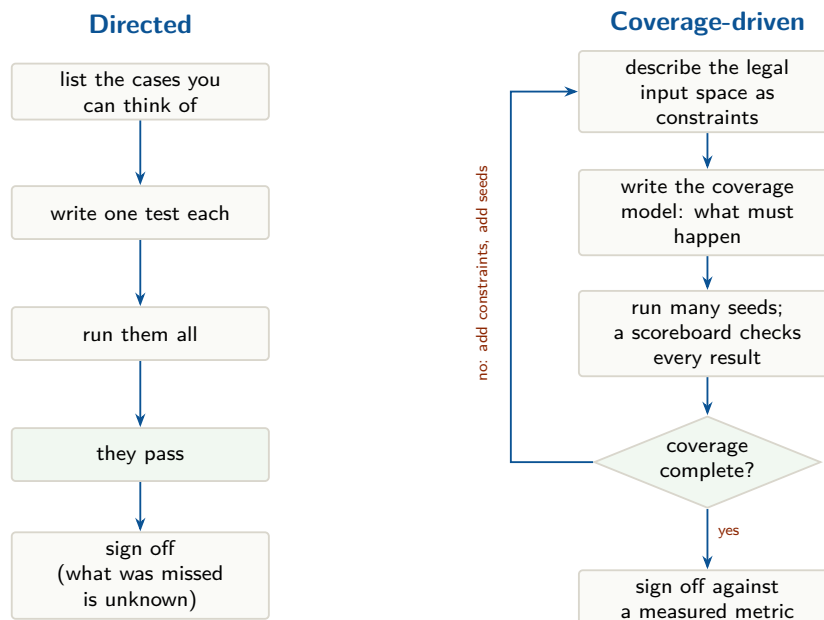


Figure 1.2. Directed verification stops when the tests you wrote pass. Coverage-driven verification stops when a measurement says the input space has been exercised. Only the second one has a closed loop.

The mental shift is from “I will write a test for every case I can think of” to “I will describe the legal input space and let the tool find the cases”. The first is bounded by your imagination. The second is bounded by your coverage model, which is a written artefact that a colleague can review and that a project manager can track. When a bug escapes a coverage-driven flow, the

post-mortem question is precise and answerable: which coverage point would have caught it, and why was it not in the model?

△ Trap: Random stimulus without a scoreboard proves nothing

A directed test carries its expected result in the same file. A random test cannot. If you randomise the inputs, something else has to know what the right answer is: a reference model, a golden function, a protocol checker, an assertion. Newcomers write the constraints first because they are the fun part, run a million cycles, see no errors and conclude that the design works. The design was never checked. Build the checker before the generator, always.

1.6.4 Be fair to VHDL

None of the above is impossible in VHDL, and it would be dishonest to suggest otherwise. OSVVM and UVVM are mature, maintained, free VHDL verification libraries. OSVVM's `RandomPkg` gives randomisation with weighted distributions and `CoveragePkg` gives functional coverage with bins and goals; both libraries add scoreboards and transaction-level interfaces, and both are used in production, particularly in European aerospace and industrial companies.

```

1  library osvvm;
2  use osvvm.RandomPkg.all;
3  use osvvm.CoveragePkg.all;
4  -- ...
5  stim : process is
6    variable rv : RandomPType;
7    variable cov : CoveragePType;
8  begin
9    cov.AddBins(GenBin(-32768, 32767, 5));  -- five bins over the range
10   while not cov.IsCovered loop
11     theta := rv.RandInt(-32768, 32767);
12     drive_transaction(theta);             -- a procedure, not pin wiggles
13     cov.ICover(theta);
14   end loop;
15   cov.WriteBin;
16   std.env.stop;
17 end process stim;
```

Listing 1.8. OSVVM: constrained random and coverage in VHDL (sketch)

So the honest statement of the difference is this. It is not that VHDL cannot do coverage-driven verification. It is that in SystemVerilog these constructs are in the language, in the LRM, in every commercial simulator, in the university curriculum and in the job advertisements, whereas in VHDL they are a library you must choose, install and persuade your team to adopt. The difference is one of ecosystem and industrial default rather than of possibility. That distinction matters when you are deciding what to learn, because the ecosystem is what you will be hired into.

Assertions on both sides

VHDL's `assert ... report ... severity` is an immediate, procedural check: it tests a condition at the moment the statement executes. SystemVerilog has the same thing (`assert (cond) else $error(...);`) and adds *concurrent* assertions, which describe behaviour over time using a temporal sequence language: `assert property (@(posedge clk_i) req_valid_i && !req_ready_o | => $stable(req_x_i));`. Those are evaluated in

the Observed region on values sampled in Preponed, which is exactly why the scheduler section came first. The sequence language is a chapter of its own: the *Verification Cookbook*. VHDL can express the same properties with PSL, which VHDL-2008 integrated into the language, but PSL support varies between tools and the construct is much less commonly taught.

1.7 The Contrasts in One Table

Table 1.1 collects the argument. Read it as a summary, not as a reference: every row is a topic that a later chapter treats properly.

Table 1.1. The philosophical contrasts, summarised.

	VHDL	SystemVerilog
Typing	Strong and static. Nearly everything is checked at analysis or elaboration.	Weak by default. Widths and signedness are adjusted silently.
Conversions	Never implicit. <code>to_integer</code> , <code>unsigned()</code> , <code>resize</code> are compulsory.	Almost always implicit. Casts exist and are a matter of discipline.
Where errors are caught	The analyser.	The linter, the coding standard and simulation.
Design units	Entity, architecture, configuration, package, package body.	One <code>module</code> . Packages exist; configurations do not, in practice.
Namespaces	Libraries with <code>work</code> as default; <code>library/use</code> .	One global namespace plus packages and <code>::</code> ; the compilation unit is best left empty.
Preprocessor	None before VHDL-2019, whose support is thin.	Full textual preprocessor: <code>`define</code> , <code>`ifdef</code> , <code>`include</code> .
Concurrency model	Signals with delta-delayed update; two-phase cycle.	Nine event regions per time slot, with iteration.
Deferred update	Built into the <code>signal</code> ; there is only one kind of signal assignment.	Chosen per statement: <code><=</code> defers, <code>=</code> does not.
Race handling	Race-free between processes by construction.	Race-free only if you follow the assignment rules and use clocking blocks.
Declared intent	None. A process is a process.	<code>always_ff</code> , <code>always_comb</code> , <code>always_latch</code> , checked by the tool.
Genericity	Generics, generate, unconstrained arrays, VHDL-2008 generic packages.	Parameters, <code>generate</code> , parameterised interfaces and classes.
Verification constructs	In the language: none beyond <code>assert/report</code> . In libraries: OSVVM, UVVM.	In the language: classes, <code>rand</code> , constraints, <code>covergroup</code> , SVA, mailboxes.
Standard	IEEE 1076; current 2008, with 2019 poorly supported.	IEEE 1800; current 2017/2023. Absorbed IEEE 1364 in 2009.
Free tools	Mature free simulation of the whole language. Synthesis via Yosys plugins.	Verilator and Yosys: fast, but a language subset (the <i>Simulation Cookbook</i>).
Industrial default	European aerospace, defence, FPGA houses, academia.	ASIC design and verification worldwide; UVM is the de facto methodology.

One row deserves an extra sentence. “Free tools” is not a footnote for this book: Verilator is the reason you can do serious SystemVerilog work on a laptop, and the *Simulation Cookbook* is built

on it. Be aware of its boundaries, though. Verilator compiles a language subset into C++; it is not a general-purpose event simulator, it is two-state, and it implements neither covergroups nor the constraint solver, so UVM is out of reach. the *Simulation Cookbook* states this precisely and lists what works where. The row is picked up again in Section 1.9, where the free-tool position is one of the places VHDL is ahead.

1.8 What Actually Changes in Your Habits

Abstractions are easier to remember when they come with a list of behaviours. Here is what to change on day one.

1.8.1 Stop doing

- **Stop trusting a clean compile.** No messages means the file parsed. It does not mean the widths agree, the ports are connected, the block is combinational or the name existed. A file is not finished until the linter is clean too.
- **Stop assuming a name is declared.** Until ``default_nettype none` is in the file, an undeclared name is a one-bit wire, not an error.
- **Stop expecting a range check.** There is no natural, no range 0 to 9 and no subtype constraint, so nothing complains when a value leaves the range you had in mind. If it must be checked, assert it.
- **Stop reaching for two-state types because they look tidy.** `int` and `bit` in RTL hide exactly the reset bugs that 'U' propagation used to reveal.

1.8.2 Start doing

- **Start with the directive pair.** Put ``default_nettype none` at the top of every file, and ``default_nettype wire` at the bottom of it.
- **Start using `always_ff` and `always_comb` exclusively.** Plain `always` in new RTL is a smell, and `always @(*)` is a worse one.
- **Start naming with suffixes:** `_i`, `_o`, `_ni`, `_d`, `_q`, `_t`, `_e`. It costs nothing and it makes a code review possible.
- **Start running the linter from the first hour.** Put `verilator -lint-only -Wall` in your `Makefile` before the second module, not after the first bug; the *Simulation Cookbook* shows the full setup.
- **Start reading the LRM, clause 4.** Ten pages on scheduling semantics will save you a week over a project.
- **Start writing the checker before the stimulus.** the *Verification Cookbook* builds testbenches in that order for a reason.

1.8.3 Be paranoid about, for the first month

- **Every width:** every assignment, port connection, concatenation and constant. Read the lint report before you open the waveform.
- **Every `=` inside a clocked block, and every `<=` inside a combinational one.** Both are legal and both are wrong.

- **Every signed operation.** If any operand of an expression is unsigned, the whole expression is evaluated unsigned, so one unsigned constant changes the result. Chapter 2 treats this properly.
- **Every port left unconnected.** `.*` and omitted ports are legal and quiet.
- **Every testbench signal driven at a clock edge.** If it is not inside a clocking block, assume it races until you have proved otherwise.
- **Every case without a default.** There is no `when others` obligation in SystemVerilog: write the `default`, or use `unique case` and let the tool check it.

The list is not a list of hard things. It is a list of things the language does not check for you, and almost every one is caught by a lint rule that somebody else has already written, in a configuration file you will inherit on your first day of work. The point of this chapter is to explain why those rules exist, so that when one fires you know what it is protecting you from rather than looking for the pragma that switches it off.

Which brings the argument to its other half. Everything up to here has been about what changes, and mostly about what you gain, when you move to SystemVerilog. The two sections that close the chapter are about what you leave behind, and about how to decide, for a particular project, whether leaving it behind is the right trade. The honest answer to the first question is: more than a book with this title normally admits.

1.9 What VHDL has that SystemVerilog does not

Almost everything above is a case for SystemVerilog, or at least an explanation of it. This section is the case against, and it is not a short list. Read it before you argue that one language is simply better than the other, and read it again before you propose a language change to a team that has a working VHDL flow.

1.9.1 A type system that catches errors at compile time

This is the big one, and Section 1.2 has already made the core of the argument: widths and signedness that VHDL checks at analysis time, SystemVerilog adjusts silently, and the compensation is a lint tool rather than a compiler. Two consequences of that were not stated there.

The first is about your own types. VHDL lets you define a scalar type that is genuinely new, an address type that is not interchangeable with a data type of the same width, and the compiler enforces the distinction. SystemVerilog has `typedef`, but a `typedef` is an alias, not a new type: assigning an `addr_t` to a `data_t` of the same width is legal and silent. All the discipline a VHDL designer gets from naming a type has to come from review instead.

The second is about arithmetic in the large. VHDL's `signed` and `unsigned` are distinct types and mixing them requires a conversion you have to write down. In SystemVerilog, signedness is a property of an expression, computed by rules that involve every operand, so one unsigned intermediate three terms away turns an arithmetic right shift into a logical one. Chapter 2 spends pages on those rules for a reason, and the reason is that they are the price of this section's first paragraph.

The single biggest practical loss in moving to SystemVerilog is compile-time width checking. Compensate with lint. Run a lint tool on every commit and treat width warnings as errors,

because your compiler will not.

1.9.2 Physical types with units

VHDL has physical types: quantities with units and a compiler that checks them. `time` is the built-in one, and you can declare your own:

```

1  type capacitance is range 0 to 1e9
2    units
3      ff; pf = 1000 ff; nf = 1000 pf;
4    end units;
5
6  constant t_setup : time := 250 ps;
7  wait for t_setup;
```

250 ps is a value of type `time`, not an integer that you have agreed to interpret as picoseconds. Adding it to a voltage is a compile error. SystemVerilog has `time` as a 64-bit integer scaled by `timeunit` and `timeprecision` directives that are per-file and easy to get wrong; the number 250 in `#250` means whatever the current time unit says it means.

1.9.3 User-defined resolution functions

`std_logic` is not built into VHDL. It is an ordinary enumerated type in an ordinary package, and its multi-driver behaviour comes from an ordinary function that you could have written. If you need a bus where multiple drivers resolve by summing, or by taking the maximum, or by flagging a conflict as an error, you write the resolution function and declare the signal to use it.

SystemVerilog has exactly one resolution rule, wired into the language for `wire` and friends, and no way to change it. In practice this matters less than it sounds, because almost nobody writes custom resolution functions. It is still a real expressive difference, and it explains why `std_logic` feels so principled compared with SystemVerilog's four-state `logic`.

1.9.4 Operator overloading

In VHDL you can define `+` for your own type. This is why `numeric_std` exists at all, why `fixed_pkg` can write `a * b` for fixed-point values, and why a package of complex numbers is twenty lines long. SystemVerilog has no operator overloading of any kind. A complex multiply is a function call, and it looks like one.

1.9.5 Configurations

A VHDL configuration is a separate design unit that binds entities to architectures and component instances to entities, without touching the design. It is how you swap a behavioural model for a gate-level netlist for one instance in one simulation.

SystemVerilog has `config` blocks in the LRM, and almost no support for them in tools. In practice you do the same job with file lists, ``ifdef`, or a wrapper module, and you lose the compile-time record of what was bound to what.

Note the asymmetry, because it is the fairest way to state this one. SystemVerilog has a construct VHDL lacks that points the other way: `bind` attaches an extra module instance to an existing one from outside, without editing the design, which is how assertions and coverage are

kept out of synthesisable files (Chapter 4). So VHDL can swap an implementation cleanly and SystemVerilog can attach an observer cleanly, and neither can do the other's trick.

1.9.6 Fixed- and floating-point packages

VHDL-2008 ships `ieee.fixed_pkg` and `ieee.float_pkg` in the standard library. These are not toys. `sfixed` and `ufixed` are arrays indexed by a signed range where index 0 is the units bit, so the type carries the binary point:

```
1 signal a : sfixed(2 downto -13); -- Q3.13
2 signal b : sfixed(1 downto -14); -- Q2.14
3 signal p : sfixed(4 downto -27); -- exactly right for a*b
4 begin
5   p <= a * b; -- width and binary point tracked
6   q <= resize(p, q'high, q'low); -- explicit, checked rounding
```

The width of a product, the alignment of an addition, and the rounding of a resize are all computed by the package from the types. The compiler catches the mistake where you add a Q1.14 to a Q2.13 without shifting.

SystemVerilog has nothing comparable. There is no standard fixed-point library, no binary-point tracking, and no compiler help. You write logic `signed [15:0]`, you write `// Q1.14` in a comment, and you keep the alignment right in your head:

```
1 // a is Q1.14, b is Q1.14 -> product is Q2.28 in 32 bits
2 logic signed [31:0] prod;
3 assign prod = a * b;
4 // take Q1.14 back out: >>> 14, then truncate. Rounding is your problem.
5 assign result = prod[29:14];
```

This matters directly for the *Example Cookbook*. The CORDIC rotator is fixed-point arithmetic from end to end, and every shift, every truncation and every saturation is checked by nothing but review. A VHDL implementation of the same algorithm would have caught several classes of error at compile time. Be disciplined: put the format in the type name (`data_t` for Q1.14 in `cordic_pkg`), never mix formats without a comment saying why, and write assertions on the numeric range.

1.9.7 Deferred constants and separate package bodies

A VHDL package declares what is visible; the package body holds the implementation, and can be recompiled without recompiling everything that uses the package. A deferred constant lets the value itself live in the body, so a change to a lookup table does not force a recompilation of every client. SystemVerilog packages have no body: everything is in one file, and every change invalidates every dependent compilation unit. On a large design this is a real difference in build time.

1.9.8 Protected types, generic packages, and 2019

VHDL-2002 protected types give encapsulated shared state with mutual exclusion, which is most of what a SystemVerilog class does (the *Verification Cookbook*) minus inheritance. VHDL-2008 generic packages and generic types give a genuine template mechanism, closer in spirit to a C++ template than SystemVerilog parameters are. Both are well specified. Support is the problem: generic packages work in the major simulators, thinly in synthesis; VHDL-2019 interfaces, mode

views and conditional analysis are excellent ideas with, at present, one or two implementations between them.

1.9.9 What you can do with a zero tool budget

This one is decided by which half of the language a free tool has to cover.

For VHDL, free simulation is mature and covers essentially the whole language, VHDL-93, -2002 and -2008 alike, at a speed that is perfectly usable for a block-level regression. A student with no licences can analyse, elaborate, simulate and dump waveforms for any VHDL design in this book, and the language they are learning is the whole language.

For SystemVerilog the free tools implement a subset, and the subset is not an arbitrary one. Verilator is excellent, the *Simulation Cookbook* is devoted to it, and it is a compiler rather than an event-driven kernel. It does not read VHDL at all, so a mixed-language hierarchy needs a commercial simulator. It is two-state, so the reset bug whose symptom is an `x` propagating through the design does not appear. It implements neither covergroups nor the constraint solver, and it has no route to UVM. the *Simulation Cookbook* states those four boundaries precisely and gives the feature-by-feature position for the version used in this book. Icarus Verilog covers Verilog-2005 well and SystemVerilog patchily, and it is not a tool to build a flow on.

Put the two together and the asymmetry is uncomfortable. If your budget is zero and your language is VHDL, you can do everything. If your language is SystemVerilog, you can do the design half completely and quickly, and what falls outside the free tools is exactly the constrained-random and coverage machinery of Section 1.6, which is the part this chapter has just spent ten pages calling the reason to move.

1.9.10 Verification frameworks that need no licence either

Section 1.6 already said that OSVVM and UVVM give VHDL most of the same verification capability. Two points belong here rather than there, because they are about the comparison rather than about the technique.

The first is that OSVVM's randomisation is coverage-driven: it biases the next draw towards bins that are still empty, instead of drawing uniformly and hoping. For many problems that closes the loop faster than a raw constraint solver does, and it is a genuinely better idea, not a poor imitation of one.

The second is the learning cost. A working OSVVM testbench is a day's work. A working UVM testbench is not, as the *Verification Cookbook* will make plain. Against that, what SystemVerilog has and they do not is the constraint solver as a language feature, assertions as a language feature, and the industry's momentum: the verification IP you buy comes as UVM, and the job advertisements ask for UVM.

1.10 Which language for which job

The previous section listed what VHDL does better; the rest of the chapter listed what SystemVerilog does better. Neither list settles anything by itself, because the question is never "which language is better" but "which language for this project". So: no advocacy, criteria.

Choose VHDL for RTL when the design is arithmetic-heavy and would benefit from `fixed_pkg`; when the team is already fluent and the cost of retraining is not repaid; when the flow is safety-critical and the stronger type system is worth real money in review time; when

the tool budget is zero, since free VHDL simulation covers the whole language and the free SystemVerilog tools do not; or when the customer requires it, which in aerospace and rail is common.

Choose SystemVerilog for RTL when the design is control-heavy and structural, with many similar blocks and wide buses that interfaces and packed structs make tractable, both of which Chapter 4 covers; when you need parameterised types across a hierarchy; when the same source must be read by a Verilog-only tool, which is most open-source RTL and most vendor IP; or when the verification is going to be SystemVerilog anyway and you want one language in the repository.

Choose SystemVerilog for verification in almost all cases where the environment must scale: constrained random, functional coverage, assertions, and UVM are a coherent package, and the industry has standardised on it. That package is what the *Verification Cookbook* teaches. The exception is a VHDL-only shop with a working OSVVM or UVVM environment, and that exception is legitimate.

Mix them. This is the common industrial answer and it works. Practically every commercial simulator supports a mixed-language hierarchy: VHDL RTL instantiated inside a SystemVerilog testbench, or the reverse. In QuestaSim you compile each language with its own compiler, `vcom-2008` and `vlog -sv`, into the same library, and elaborate the two together. The seams are the port types (map `std_logic_vector` to packed vectors and keep the boundary plain: no records, no interfaces, no enumerations across the boundary) and the inability to bind SystemVerilog assertions into a VHDL instance. Plan the boundary deliberately and the arrangement is comfortable. Note that free tooling is not an option here at all, because Verilator does not read VHDL.

What actually decides it

In practice the choice is made by the existing code base, the customer, and the tool licences, in that order. Language merit comes fourth. Knowing both languages well is worth more than a correct opinion about which is better.

1.11 Checklist

- VHDL was designed as a language and grew a synthesis subset. SystemVerilog was designed as a simulator input and grew a type system. Every difference in this chapter follows from that.
- Correctness in VHDL comes from the analyser; in SystemVerilog it comes from the linter, the coding standard and simulation. Wire `verilator -lint-only -Wall` into your build on day one.
- Widths never have to match in SystemVerilog. Assignments, expressions and port connections truncate or extend silently. Check every one.
- `logic` is `std_logic` without strengths and with a single driver. Use it for all RTL. Do not use `int` or `bit` for hardware: two-state types start at zero and hide missing resets.
- Put ``default_nettype none` at the top of every file and ``default_nettype wire` at the bottom, or an undeclared name will silently become a one-bit wire.
- When you see `library ieee; use ieee.numeric_std.all;`, write `import my_pkg::*;`. Put every shared `typedef` and `parameter` in a package; never declare anything at compilation-unit scope.
- ``define` and ``include` are textual and unscoped. Prefix macro names, guard included files,

and prefer a package parameter whenever one will do.

- VHDL signal assignment $\equiv$ SystemVerilog `<=`; VHDL variable assignment $\equiv$ SystemVerilog `=`. `<=` evaluates in Active and updates in NBA: VHDL's evaluate/update pair, recreated by hand.
- Non-blocking for everything a clock edge writes, blocking for everything combinational, never both in one block, never two blocks writing the same variable. VHDL gave you this for free; SystemVerilog does not.
- Declare intent: `always_ff` for sequential, `always_comb` for combinational, `always_latch` when you mean it. One clock and at most one asynchronous reset per `always_ff`.
- In a testbench, drive and sample synchronous signals through a clocking block. `input #1step` samples in Preponed, before any design code has run in that time slot.
- Verification in SystemVerilog is a software problem: transactions, constrained random stimulus, functional coverage as the exit criterion, classes for reuse. Write the checker before the generator: random stimulus without a scoreboard proves nothing.
- OSVVM and UVVM give VHDL most of the same capability, and OSVVM's coverage-driven randomisation closes the loop faster than a raw constraint solver on many problems. The difference is ecosystem, industrial default and learning cost, not possibility.
- Know what you are giving up, because it is not nothing: compile-time width and signedness checking, genuinely distinct user-defined types, physical types with units, user-defined resolution, operator overloading, `fixed_pkg` and `float_pkg`, configurations, deferred constants and separate package bodies. Compensate with lint, naming and review, in that order.
- There is no SystemVerilog fixed-point library. Put the format in the type name, never mix formats without a comment saying why, and assert the numeric range. This is the single largest source of unchecked error in the CORDIC of the *Example Cookbook*.
- On a zero tool budget, VHDL gets the whole language and SystemVerilog gets a fast subset: no VHDL at the boundary, two-state only, no covergroups, no constraint solver, no UVM. Plan the flow around that, not around a hope.
- The language choice is made by the existing code base, the customer and the tool licences, in that order, and by language merit fourth. Mixed-language hierarchies are normal and need a plain port boundary.

The Rosetta Stone: Types and Expressions

The first half of the reference core. For every VHDL type, literal, operator and expression rule you have used during your degree, this chapter gives the SystemVerilog form beside it, says what is identical, what is only superficially similar, and what has no counterpart at all. The centre of it is the width and signedness rules, which are the ones that silently change the arithmetic your design performs. The behavioural half, processes and statements and everything else that happens over time, is Chapter 3.

A translation dictionary is useful only if it also tells you where the two languages disagree. Most of VHDL has a direct SystemVerilog equivalent, and for those constructs a side-by-side pane is enough. A small but dangerous minority looks equivalent and is not: the width rules of arithmetic, the meaning of an unsized literal, the resolution of a multiply driven signal. Those get a trap box, and you should read every one of them.

The dictionary is split across two chapters because it divides cleanly in two, and because reading four hundred mappings in one sitting is not how anybody uses a reference. This chapter is the *static* half: how a design is broken into files, what a port and a parameter are, what the types are, what a literal means, and what happens to widths and signedness when you write an expression. Nothing here depends on time passing. Chapter 3 is the half that does: processes, sequential statements, `generate`, instantiation, subprograms, packages, attributes, reporting and delays. It also carries the complete mapping table for both halves, so if you are looking for the page to photocopy, it is at the end of the next chapter and not this one.

The layout is constant. Each section opens with one or more *rosetta* blocks, VHDL on the left, SystemVerilog on the right. The prose after the block answers three questions in order: what is the same, what is different, what is missing. Where a construct exists in only one language the missing pane is filled with the closest idiom and the text says plainly that it is not a translation.

Throughout, VHDL means IEEE 1076-2008 and SystemVerilog means IEEE 1800-2017. Where a VHDL-2019 feature would change the answer, the text says so and warns you that tool support is thin.

2.1 Design units, files and the compilation model

VHDL has a strict notion of a design unit and an equally strict analysis order. SystemVerilog inherited Verilog's file-oriented, preprocessor-driven model, which is looser in every respect. This is the first thing to internalise, because it explains a great deal of what follows.

2.1.1 Entity and architecture versus module

VHDL

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity counter is
  generic (W : positive := 8);
  port (
    clk : in  std_logic;
    rst : in  std_logic;
    en  : in  std_logic;
    q   : out std_logic_vector(W-1 downto 0)
  );
end entity counter;

architecture rtl of counter is
  signal cnt : unsigned(W-1 downto 0);
begin
  process (clk) is
  begin
    if rising_edge(clk) then
      if rst = '1' then
        cnt <= (others => '0');
      elsif en = '1' then
        cnt <= cnt + 1;
      end if;
    end if;
  end process;
  q <= std_logic_vector(cnt);
end architecture rtl;
```

SystemVerilog

```
`default_nettype none

module counter #(
  parameter int unsigned W = 8
) (
  input wire      clk,
  input wire      rst,
  input wire      en,
  output logic [W-1:0] q
);

  logic [W-1:0] cnt;

  always_ff @(posedge clk) begin
    if (rst)      cnt <= '0;
    else if (en)  cnt <= cnt + 1;
  end

  assign q = cnt;
endmodule

`default_nettype wire
```

The two files describe the same hardware, and almost every line has a counterpart. The differences that matter are structural.

VHDL splits the black box in two. The **entity** is the interface, the **architecture** is one possible implementation of it, and the pair is bound either implicitly (last analysed architecture wins) or explicitly by a **configuration**. SystemVerilog has a single **module** that carries both. There is no separate interface unit to compile, no analysis order between them, and no default binding rule to remember.

SystemVerilog does have a **config** construct, kept from Verilog-2001, that plays roughly the role of a VHDL configuration by selecting which library cell satisfies which instance. In practice nobody writes them for RTL, tool support varies, and the same job is done with library search order on the command line. Treat configurations as a VHDL idiom without a practical SystemVerilog equivalent.

2.1.2 Several architectures for one entity

The VHDL habit of writing **rtl**, **behavioural** and **structural** architectures for one entity has no direct translation. SystemVerilog gives you three replacements, in decreasing order of preference.

VHDL

```
architecture rtl of alu is
begin
    -- synthesisable description
end architecture rtl;

architecture behav of alu is
begin
    -- untimed reference model
end architecture behav;

-- selected by a configuration,
-- or by analysis order
```

SystemVerilog

```
module alu #(
    parameter bit BEHAVIOURAL = 0
) ( /* ports */ );
    if (BEHAVIOURAL) begin : g_ref
        // untimed reference model
    end else begin : g_rtl
        // synthesisable description
    end
endmodule

// or: two modules, alu_rtl and
// alu_behav, chosen by the
// instantiating code
```

The first replacement, shown above, is a parameter tested inside a `generate` region. It keeps one module name and one port list, which is exactly what a VHDL entity gave you. The second is a preprocessor ``ifdef`, which works but hides the choice from the elaborator. The third, and the most common in industry, is simply two modules with different names and a wrapper that instantiates one of them. SystemVerilog has no notion of “the same interface, different body”, so the interface is repeated.

Why the split existed

The entity/architecture split is a 1980s design decision aimed at supporting separate compilation and technology retargeting: analyse the entity once, supply a gate-level architecture later. Verilog solved the same problem with its library and config mechanism, which nobody used, and in practice with file lists. The industry converged on file lists. This is a case where the weaker language mechanism won because the tooling around it was simpler.

2.1.3 Libraries, use clauses and imports

VHDL

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

library work;
use work.cordic_pkg.all;
use work.cordic_pkg.data_t;
```

SystemVerilog

```
// no library clause needed:
// std types are built in

import cordic_pkg::*;
import cordic_pkg::data_t;

`include "cordic_defs.svh"
```

There is no `library` clause in SystemVerilog. A VHDL library is a named container of analysed design units, created by the tool (`vcom -work mylib`), and referenced by name from the source. SystemVerilog libraries exist in the LRM but the everyday model is: everything you compile lands in one namespace, and modules are found by name.

`use work.pkg.all` maps to `import pkg::*`. The two are close enough that the translation is mechanical, but the resolution rules differ, and Section 3.7 treats them properly. The essential difference: `use ... all` makes the names visible immediately, while `import pkg::*` makes them visible *only on reference and only if nothing else already declares that name*. A wildcard import that would collide is not an error until you actually use the ambiguous name.

The two lines that have no VHDL counterpart at all are the absence of `library` and the presence of ``include`. The second is a textual file inclusion performed by a preprocessor before the

language proper sees the text. VHDL has nothing like it, and never needed it, because a package is a first-class compiled unit.

2.1.4 The compilation unit and \$unit

In VHDL every design unit belongs to a library and the analysis order is enforced: a package body cannot be analysed before its declaration, an architecture cannot be analysed before its entity. Change a package and every unit that used it is marked out of date. This is a genuine strength and SystemVerilog has nothing equivalent for the general case.

SystemVerilog defines a *compilation unit*: the set of source files compiled together in one invocation of the tool. Declarations made outside any module, package or interface live in the compilation-unit scope, addressed as \$unit.

```

1 // file: types.sv -- outside any module or package
2 typedef logic [15:0] word_t;      // lands in $unit
3 localparam int unsigned NREG = 32;
4
5 module regfile (input word_t d, output word_t q);
6     word_t mem [NREG];            // visible here only if compiled together
7 endmodule

```

Listing 2.1. A compilation-unit-scope declaration. Legal, and a bad idea.

The trap is that “compiled together” is defined by the tool invocation, not by the language. QuestaSim with two vlog commands gives you two compilation units and word_t is invisible in the second. Verilator, VCS and Xcelium all have their own defaults and their own switches. A design that relies on \$unit compiles for one person and not for the next.

Put nothing in \$unit. Every type, constant and function belongs in a package; every macro belongs in an included header.

2.1.5 The preprocessor

Verilog’s preprocessor is a separate, weaker language that runs over the text before compilation. It is the only place in SystemVerilog where the backtick appears.

VHDL

```

-- VHDL has no preprocessor.
-- The nearest equivalents:

constant DEPTH : positive := 16;

-- conditional analysis, VHDL-2019
-- (thin tool support):
-- if TOOL_TYPE = "SIMULATION" generate

```

SystemVerilog

```

`timescale 1ns/1ps
`define DEPTH 16
`define MAX(a,b) (((a)>(b))?(a):(b))

`ifdef SYNTHESIS
    // synthesis-only text
`elsif FORMAL
    // formal-only text
`else
    // simulation-only text
`endif

`undef DEPTH
`include "defs.svh"

```

Macros are textual, untyped and unscoped. `DEPTH is not a constant: it is the characters 16 pasted in. A macro defined in one file is visible in every file compiled after it in the same compilation unit, which means macro definitions leak across your whole design in file order.

Prefer a package `localparam` for every value, and reserve macros for the two jobs they alone can do: conditional inclusion of text, and generating repetitive declarations that the language cannot parameterise.

``timescale` deserves its own trap and gets one in Section 3.10.

Recipe: Always start an RTL file with ``default_nettype none`

Verilog creates a one-bit wire for any undeclared identifier used in a port connection or on the right of a continuous assignment. A misspelt signal name therefore compiles cleanly and produces a silently disconnected net. Putting ``default_nettype none` at the top of every file turns that into an error. Restore it with ``default_nettype wire` at the bottom of the file so that you do not break code compiled after yours.

△ Trap: Implicit nets: the typo that compiles

VHDL rejects an undeclared signal. Verilog invents one.

```
1 module top (input wire a, b, output logic y);
2   wire and_out;
3   assign and_out = a & b;
4   assign y      = and_ou;    // typo. Compiles. y is z forever.
5 endmodule
```

The name `and_ou` is undeclared, so Verilog creates a new one-bit net that nothing drives. Simulation shows `z`, synthesis shows a constant or an optimised-away output, and there is no message. ``default_nettype none` is the only reliable defence, and it is not the default because thirty years of existing code depends on the old behaviour.

2.2 Ports

VHDL

```
entity fifo is
  port (
    clk  : in  std_logic;
    rst_n : in  std_logic;
    wr_en : in  std_logic;
    din   : in  std_logic_vector(7 downto 0);
    rd_en : in  std_logic;
    dout  : out std_logic_vector(7 downto 0);
    full  : out std_logic;
    empty : buffer std_logic;
    sda   : inout std_logic
  );
end entity fifo;
```

SystemVerilog

```
module fifo (
  input wire      clk,
  input wire      rst_n,
  input wire      wr_en,
  input wire [7:0] din,
  input wire      rd_en,
  output logic [7:0] dout,
  output logic     full,
  output logic     empty,
  inout wire      sda
);
endmodule
```

2.2.1 Modes and directions

Table 2.1 gives the mapping. Of the four modes you will actually meet, three translate directly; the fifth entry in the table, `linkage`, is obsolete in VHDL as well.

The interesting entry is `buffer`. Before VHDL-2008 an out port could not be read inside the architecture, so you either declared an internal signal and assigned it to the port, or used mode `buffer` and accepted its restrictions on how the port could be connected. VHDL-2008 removed

Table 2.1. Port modes.

VHDL mode	SystemVerilog direction	Notes
<code>in</code>	<code>input</code>	Read only in both.
<code>out</code>	<code>output</code>	Readable inside the module in SystemVerilog.
<code>inout</code>	<code>inout</code>	Must be a net type in SystemVerilog.
<code>buffer</code>	<code>output</code>	No equivalent needed; <code>output</code> already reads back.
<code>linkage</code>	—	Obsolete in VHDL too. Ignore.

the restriction: an `out` port is now readable. SystemVerilog never had the restriction. An `output logic` is an ordinary variable that happens to be visible outside, and you read it as freely as any internal signal:

```

1 module gray_counter (input wire clk, output logic [3:0] cnt);
2     always_ff @(posedge clk)
3         cnt <= cnt + 1;           // cnt is read and written; no helper signal
4 endmodule

```

Listing 2.2. Reading an output port is normal in SystemVerilog.

The habit of declaring `signal cnt_int` and writing `cnt <= cnt_int` at the end of the architecture is a pre-2008 VHDL reflex. Do not carry it into SystemVerilog.

2.2.2 Port types: `wire` or `logic`

A VHDL port has a type and nothing else. A SystemVerilog port has a direction, a *data type*, and separately a *net or variable kind*. The default, if you write nothing, is `wire`, which is a four-state net type resolved across all its drivers.

The convention used throughout this book, and the one you should adopt, is:

- `input wire` for inputs. An input is always driven from outside, so a net is correct, and it also documents that the module does not drive it.
- `output logic` for outputs. A variable can be assigned from an `always` block or from a continuous assignment, which covers every RTL case.
- `inout wire` for true bidirectional pads. A variable cannot be used, because more than one driver is the whole point.

Writing `output wire` is legal and forces you to drive the port with a continuous `assign`; writing `input logic` is legal and mostly harmless. Consistency matters more than the choice, but `output logic` is the one that prevents the most mistakes.

2.2.3 Default values on ports

VHDL

```
entity dsp is
  port (
    clk : in std_logic;
    -- default used if the port is
    -- left unconnected at the
    -- instantiation:
    ce   : in std_logic := '1';
    mode : in std_logic_vector(1 downto 0)
          := "00"
  );
end entity dsp;
```

SystemVerilog

```
module dsp (
  input wire      clk,
  input wire      ce,
  input wire [1:0] mode
);
  // SystemVerilog has no default
  // port values. An unconnected
  // input is high impedance, and
  // reads as x through any logic.
endmodule
```

This is a real loss. VHDL lets you add an optional port to an existing entity and leave every existing instantiation untouched. SystemVerilog does not: an unconnected input port is undriven, and *z* propagating into combinational logic gives *x*. The two available workarounds are a parameter instead of a port, when the value is elaboration-time constant, or an explicit tie-off at every instantiation site.

△ Trap: An unconnected input is not zero

Leaving an input off a SystemVerilog instantiation is legal and silent. The port is *z*. It does not default to 0, it does not default to the last value, and most simulators will not warn you. Verilator does warn (PINMISSING) and this is one of the reasons to run it over your code even if you simulate elsewhere. Connect every port, every time.

2.2.4 ANSI and non-ANSI port lists

You will meet legacy code that declares ports twice. Recognise it, and never write it.

VHDL

```
-- VHDL has only one form.
entity adder is
  port (
    a, b : in  std_logic_vector(7 downto 0);
    s    : out std_logic_vector(8 downto 0)
  );
end entity adder;
```

SystemVerilog

```
// non-ANSI (Verilog-1995). Legacy.
module adder (a, b, s);
  input  [7:0] a;
  input  [7:0] b;
  output [8:0] s;
  reg    [8:0] s;
  // ...
endmodule
```

The non-ANSI form names the ports in the header and gives their direction, size and kind in separate declarations in the body. It is verbose, it lets the two halves disagree, and it makes parameterised widths awkward. The ANSI form, introduced in Verilog-2001 and used everywhere in this book, puts everything in the header. Use it exclusively.

2.3 Generics and parameters

VHDL

```
entity fifo is
  generic (
    WIDTH : positive := 8;
    DEPTH : positive := 16;
    NAME  : string   := "fifo"
  );
  port (...);
end entity fifo;

architecture rtl of fifo is
  constant ADDR_W : natural :=
    integer(ceil(log2(real(DEPTH))));
  type mem_t is array (0 to DEPTH-1) of
    std_logic_vector(WIDTH-1 downto 0);
  signal mem : mem_t;
begin
end architecture rtl;
```

SystemVerilog

```
module fifo #(
  parameter int unsigned WIDTH = 8,
  parameter int unsigned DEPTH = 16,
  parameter string  NAME  = "fifo"
) (...);

  localparam int unsigned ADDR_W =
    $clog2(DEPTH);

  logic [WIDTH-1:0] mem [DEPTH];

endmodule
```

2.3.1 The mapping

A VHDL **generic** is an elaboration-time constant supplied by the instantiating code. So is a SystemVerilog **parameter**. A VHDL **constant** declared in the architecture is computed from the generics and cannot be overridden. So is a SystemVerilog **localparam**. The correspondence is exact, and it is worth stating the rule as a discipline:

***parameter** for anything the instantiator may set, **localparam** for anything derived from it. Never leave a derived value as a **parameter**: it can then be overridden inconsistently with the value it was derived from.*

2.3.2 Types on parameters

A VHDL generic is always typed. A Verilog parameter historically was not: its type came from the type of its default value, which was usually an untyped integer. SystemVerilog allows and encourages an explicit type, and you should always write one.

```
1 parameter          WIDTH_A = 8;           // type taken from 8: 32-bit signed int
2 parameter int unsigned WIDTH_B = 8;       // explicit, checked at override
3 parameter logic [7:0] INIT   = 8'hA5;     // explicit vector
4 parameter real     FREQ_HZ = 100.0e6;
5 parameter string   NAME    = "u_fifo";
6 parameter type     payload_t = logic [31:0]; // a type parameter
```

Listing 2.3. Typed and untyped parameters.

The last line has no VHDL-2008 equivalent in common use. SystemVerilog lets a parameter be a *type*, so a module can be generic over the type of its payload:

VHDL

```
-- VHDL-2008 generic types.
-- Legal, but tool support is thin
-- and few designers use them.
entity reg_stage is
  generic (type payload_t);
  port (d : in payload_t;
        q : out payload_t);
end entity reg_stage;
```

SystemVerilog

```
module reg_stage #(
  parameter type payload_t = logic
) (
  input wire      clk,
  input payload_t d,
  output payload_t q
);
  always_ff @(posedge clk) q <= d;
endmodule
```

VHDL-2008 does have generic types, and generic packages and generic subprograms with them. Support in the mainstream simulators is partial and in the synthesis tools it is worse, so most VHDL designers have never used them. In SystemVerilog a type parameter is ordinary, well supported, and the standard way to write a reusable pipeline register, FIFO or arbiter.

2.3.3 Overriding at instantiation

VHDL

```
u_fifo : entity work.fifo
  generic map (
    WIDTH => 16,
    DEPTH => 32
  )
  port map (
    clk => clk,
    ...
  );

-- positional generic map, legal
-- but discouraged:
u_bad : entity work.fifo
  generic map (16, 32)
  port map (...);
```

SystemVerilog

```
fifo #(
  .WIDTH (16),
  .DEPTH (32)
) u_fifo (
  .clk (clk),
  ...
);

// positional, legal but
// discouraged:
fifo #(16, 32) u_bad (...);

// defparam: obsolete, never use
// defparam u_fifo.WIDTH = 16;
```

Named association works in both languages and is the only form you should write. Note the position of the instance name: VHDL puts the label first, SystemVerilog puts it after the parameter list and before the port list. This trips up every VHDL designer for about a week.

`defparam` is a Verilog-1995 mechanism for reaching into a hierarchy and changing a parameter of an already-instantiated module from outside. It is deprecated in IEEE 1800, it defeats separate elaboration, and several tools reject it. There is no VHDL equivalent because VHDL never made the mistake. Never write it.

Parameters that must not be overridden

A VHDL generic can be constrained by its subtype: `generic (N : positive)` rejects zero and negative values at elaboration. SystemVerilog has no subtype constraints, so `parameter int unsigned N = 8` will happily accept `-1` reinterpreted as a very large unsigned number. Guard it yourself with an elaboration-time assertion:

```
1 | if (N < 1 || N > 64) $error("N out of range: %0d", N);
```

placed in a `generate` region, or with the `$fatal` form inside an `initial` block. The first is checked at elaboration and is the better choice.

2.4 Data types

This is the section where the two languages diverge most, and where a careless translation costs you a week of debugging. VHDL is strongly typed: the type system is the main tool the language gives you for catching mistakes. SystemVerilog is weakly typed by inheritance from Verilog, which had exactly one type, the bit vector, and grew everything else on top of it. Almost every expression that VHDL would reject, SystemVerilog accepts by converting something silently.

2.4.1 The four-state and nine-state logic types

VHDL	SystemVerilog
<pre>-- from ieee.std_logic_1164 signal a : std_ulogic; -- unresolved signal b : std_logic; -- resolved -- the nine values: -- 'U' uninitialised -- 'X' forcing unknown -- '0' forcing zero -- '1' forcing one -- 'Z' high impedance -- 'W' weak unknown -- 'L' weak zero -- 'H' weak one -- '-' don't care</pre>	<pre>logic a; // 4-state variable wire b; // 4-state net, resolved bit c; // 2-state, 0 or 1 only reg d; // same type as logic // the four values: // 0 // 1 // x unknown // z high impedance // no 'U', no 'W'/'L'/'H', // no '-'</pre>

Take the value systems first. `std_logic` has nine values because it was designed to model both drive strength and the difference between “never assigned” and “assigned a conflicting value”. SystemVerilog has four, because Verilog modelled strength separately, through net strengths, which nobody uses in RTL. The mapping is:

Table 2.2. Value systems.

<code>std_logic</code>	SystemVerilog	Comment
'U'	—	Reset value of a SystemVerilog four-state variable is <code>x</code> , not a separate value.
'X'	<code>x</code>	Direct equivalent.
'0', '1'	<code>0</code> , <code>1</code>	Direct equivalent.
'Z'	<code>z</code>	Direct equivalent.
'W', 'L', 'H'	—	Drive strengths; SystemVerilog keeps these in the (unused) strength system.
'-'	—	No value. Don't-care is expressed by <code>casez/case inside</code> , not by a value.

The loss of 'U' matters more than it looks. In VHDL an unassigned signal starts at 'U' and any logic that touches it produces 'U', so a missing reset shows up immediately as a distinct colour in the waveform. In SystemVerilog an unassigned four-state variable starts at `x`, which is the same value a genuine conflict produces. You lose the ability to distinguish “never driven” from “driven to an unknown”. Two-state types (`bit`, `int`) are worse still: they start at `0`, so a missing reset simply looks like a working reset.

△ Trap: Two-state types hide missing resets

```

1 bit [7:0] count_a;    // starts at 0
2 logic [7:0] count_b;  // starts at x
3 always_ff @(posedge clk) begin
4     count_a <= count_a + 1; // no reset. Simulates perfectly from 0.
5     count_b <= count_b + 1; // no reset. x forever. Bug visible.
6 end

```

Silicon does not start at zero. Use four-state types (`logic`) for everything in the RTL, and reserve `bit`, `int` and friends for testbench bookkeeping where a two-state value is what you actually mean. The speed advantage of two-state simulation is real but it is not yours to take at the RTL level.

2.4.2 Resolution and multiple drivers

`std_logic` is a *resolved subtype* of `std_ulogic`. The resolution is done by an ordinary VHDL function, `resolved`, written in `std_logic_1164` and visible in the source. You can write your own resolution function for your own type. `std_ulogic` is unresolved, and two drivers on a `std_ulogic` signal are an elaboration error.

SystemVerilog moves the choice into the type kind rather than the type.

VHDL

```

-- unresolved: two drivers = error
signal s : std_ulogic;

-- resolved: two drivers merge
-- through function `resolved'
signal r : std_logic;

-- your own resolution:
function my_res (v : my_vec)
    return my_type;
subtype my_res_t is
    my_res my_type;

```

SystemVerilog

```

// variable: one driver only
logic s;

// net: drivers merge through the
// built-in wire resolution table
wire r;

// user-defined resolution:
// nettype with a resolution
// function -- exists, but is
// not supported for synthesis

```

A `logic` is a variable. It may be written by exactly one continuous assignment, or by procedural assignments from one `always` block, and any second source is an elaboration error in a conforming tool. A `wire` is a net. Two drivers on a `wire` are legal and are resolved by the built-in wire resolution: 0 against 1 gives x, anything against z gives the anything. That table is fixed by the LRM and you cannot change it.

SystemVerilog-2012 added user-defined nettypes (`nettype real_net_t with sum;`), which is the closest thing to a VHDL resolution function. It is intended for analogue and mixed-signal modelling, it has no synthesis meaning, and you will not need it.

logic unless you need multiple drivers. wire only for a real tri-state bus or for module inputs.

2.4.3 Vectors, ranges and direction

VHDL

```

signal a : std_logic_vector(7 downto 0);
signal b : std_logic_vector(0 to 7);
signal u : unsigned(15 downto 0);
signal s : signed(15 downto 0);

-- attributes
a'range      -- 7 downto 0
a'length     -- 8
a'high       -- 7
a'low        -- 0
a'left       -- 7
a'right      -- 0
a'ascending  -- false
a'reverse_range -- 0 to 7

```

SystemVerilog

```

logic      [7:0] a;
logic      [0:7] b;
logic      [15:0] u; // unsigned
logic signed[15:0] s;

// system functions
$bits(a)    // 8
$size(a)    // 8
$high(a)    // 7
$low(a)     // 0
$left(a)    // 7
$right(a)   // 0
$increment(a) // 1 (i.e. [7:0])
// no $range, no reverse_range

```

Both languages let you write the range in either direction. Both make the descending form the sane choice for a number, because bit n then has weight 2^n . The difference is one of convention strength: in VHDL you meet 0 to $N-1$ vectors regularly, often in bus standards translated from a document; in SystemVerilog an ascending packed range is so rare that many engineers have never seen one, and some tools handle it poorly.

Write $[N-1:0]$ for every packed vector. If you inherit $[0:N-1]$ code, convert it before you extend it.

The attribute-to-system-function mapping is nearly one to one, with two exceptions. `'range` has no SystemVerilog equivalent, because SystemVerilog has no range objects; a VHDL `for i in a'range` loop becomes `for (int i = 0; i < $size(a); i++)` or, for unpacked arrays, `foreach (a[i])`. `'reverse_range` has no equivalent either. In the other direction `$bits` is more general than `'length`: it returns the total number of bits of *any* type, including a packed struct or an enumeration, which VHDL can only do by hand.

2.4.4 The scalar types

Table 2.3 is the table to memorise. Note in particular that SystemVerilog has both `int` and `integer`, that they differ, and that the one whose name matches VHDL's is the one you will almost never use.

Four entries deserve comment.

boolean. VHDL has a distinct boolean type, and a condition must be of that type: `if a = '1' then` is required because `a` itself is not a boolean. SystemVerilog has no boolean type. A condition is any expression; it is true if it is non-zero and, in a four-state type, known. `if (a)` is therefore idiomatic and correct, and `if (a == 1'b1)` is merely redundant. Where VHDL forces you to be explicit, SystemVerilog does not, and this is the single most common cosmetic change in a translated file.

int versus integer. `int` is 32-bit signed two-state. `integer` is 32-bit signed four-state and exists only for Verilog compatibility. VHDL's `integer` maps to SystemVerilog's `int`, not to SystemVerilog's `integer`. Use `int` for loop counters and for parameters. If you need a loop variable that can hold `x`, you have a design problem, not a type problem.

natural and positive. These are VHDL subtypes with range constraints, checked at run time. SystemVerilog has `int unsigned`, which constrains the interpretation but not the range: nothing

Table 2.3. Scalar types. “4-state” means the type can hold x and z.

VHDL	SystemVerilog	Width and sign	States
bit	bit	1, unsigned	2
std_ulogic	logic	1, unsigned	4
std_logic	wire	1, unsigned	4, resolved
boolean	—	use bit or logic; any non-zero is true	—
—	byte	8, signed	2
—	shortint	16, signed	2
integer	int	32, signed	2
—	integer	32, signed	4
—	longint	64, signed	2
natural	—	int unsigned is close but unconstrained	2
positive	—	no equivalent	—
real	real	64-bit IEEE 754	—
—	shortreal	32-bit IEEE 754	—
time	time	64, unsigned; see Section 3.10	4
character	byte	8, signed	2
string	string	dynamic in SystemVerilog, fixed length in VHDL	—

stops you assigning -1 and reading back 4294967295. See the trap below.

string. A VHDL string is a constrained array of **character** and its length is part of its subtype, which is why concatenating strings of different lengths is such a nuisance. A SystemVerilog **string** is a dynamically sized built-in type with methods (`.len()`, `.substr()`, `.toupper()`) and ordinary `==` comparison. This is one of the few places where SystemVerilog is unambiguously more pleasant.

△ **Trap: There are no range constraints and no run-time checks**

VHDL

```
subtype small_t is
  natural range 0 to 15;
signal s : small_t;

-- assigning 16 is a run-time
-- error: "value 16 out of
-- range 0 to 15"
s <= 16;
```

SystemVerilog

```
// no subtype constraints exist.
typedef int unsigned small_t;
small_t s;

// this is simply legal:
s = 16;
s = -1;           // reads as
                  // 4294967295
```

Every VHDL run-time range check you rely on disappears. A state variable declared as **natural range 0 to 3** becomes **logic [1:0]**, which wraps instead of failing, or **int**, which silently holds anything. The replacement is explicit: an **assert** in the RTL, an **enum** for state variables, or an **assertion** in the testbench. Do not assume the tool will tell you.

2.4.5 numeric_std and arithmetic types

VHDL

```

use ieee.numeric_std.all;

signal u : unsigned(7 downto 0);
signal s : signed(7 downto 0);
signal v : std_logic_vector(7 downto 0);
signal i : integer;

u <= unsigned(v);
v <= std_logic_vector(u);
i <= to_integer(u);
u <= to_unsigned(i, 8);
s <= to_signed(i, 8);
u <= resize(u, 8);
u <= shift_left(u, 2);
s <= shift_right(s, 2); -- arithmetic
u <= rotate_left(u, 3);

```

SystemVerilog

```

logic [7:0] u;
logic signed [7:0] s;
logic [7:0] v;
int i;

assign u = v; // no cast needed
assign v = u; // no cast needed
assign i = u; // zero-extends
assign u = i; // truncates
assign s = i; // truncates
// resize: implicit everywhere
assign u = u << 2;
assign s = s >>> 2; // arithmetic
assign u = {u[4:0], u[7:5]}; // rotate

```

Look at the right-hand pane and count the conversion functions: there are none. This is the central practical difference. In VHDL, `unsigned`, `signed` and `std_logic_vector` are three distinct array types with the same element type, and the type marks (`unsigned(v)`) are what let you move between them. Every conversion is written down, and every one is checked. In SystemVerilog, signedness is a property of the vector, not a separate type, so `logic [7:0]` and `logic signed [7:0]` interoperate freely and the compiler applies whatever conversion makes the expression legal.

The individual mappings:

- `unsigned(v)` and `std_logic_vector(u)`: no operation at all in SystemVerilog. The bits are the bits.
- `to_integer(u)`: implicit. Assigning a vector to an `int` zero-extends if the vector is unsigned and sign-extends if it is signed. `$signed(v)` and `$unsigned(v)` force the interpretation where you need it.
- `to_unsigned(i, N)` and `to_signed(i, N)`: implicit truncation to the width of the target. There is no width argument because the target supplies it. If the value does not fit, the extra bits are dropped without a warning.
- `resize(u, N)`: implicit. This is convenient and it is the source of half the bugs in Section 2.6.
- `shift_left/shift_right`: `<<`, `>>` and `>>>`. Note carefully that `>>>` is arithmetic only if its left operand is signed. On an unsigned operand it is identical to `>>`.
- `rotate_left/rotate_right`: no equivalent. Write the concatenation, or a small function.

Recipe: A rotate function

Since SystemVerilog has no rotate operator, put one in your package once:

```

1 function automatic logic [31:0] rot132 (logic [31:0] x, int unsigned n);
2   return (x << n) | (x >> (32 - n)); // n in 1..31
3 endfunction

```

Note `automatic`: see Section 3.6 for why this matters. For a parameterised width, make the function part of a parameterised package or give it a `parameter` type.

std_logic_arith and std_logic_unsigned

You may have inherited code that writes `use ieee.std_logic_unsigned.all` and then adds `std_logic_vector` values directly. Those packages are Synopsys creations that were placed in the `ieee` library without ever being IEEE standards. They overload the arithmetic operators on `std_logic_vector` itself, which means the same vector cannot be treated as signed in one expression and unsigned in another, and mixing them with `numeric_std` produces ambiguous overloads. Do not use them in new VHDL. There is an irony here worth noting: what those packages did to VHDL, by making a raw bit vector arithmetic and implicitly unsigned, is exactly what SystemVerilog does by design. When you move to SystemVerilog you are permanently in `std_logic_unsigned` mode, which is precisely why Section 2.6 needs so much space.

2.5 Literals and aggregates

VHDL

```
'0'          -- bit literal
'1'
'Z'
'X'
"1010"       -- 4-bit string literal
x"A5"        -- 8-bit hex
o"17"        -- 6-bit octal
b"1010_1010" -- with separators
16#FF#       -- integer 255
2#1010#      -- integer 10
1_000_000    -- integer
1.5e-3       -- real
10 ns        -- physical
'a'          -- character
"hello"      -- string
```

SystemVerilog

```
1'b0         // sized bit literal
1'b1
1'bz
1'bx
4'b1010      // 4-bit binary
8'hA5        // 8-bit hex
6'o17        // 6-bit octal
8'b1010_1010 // with separators
255          // 32-bit int
10           // 32-bit int
1_000_000    // separators work
1.5e-3       // real
10ns         // time literal
"a"          // 8-bit value
"hello"      // string
```

2.5.1 Sized and unsized

A VHDL literal is typed and, if it is an array literal, its length comes from the context. `"1010"` is four bits because it has four characters; `x"A5"` is eight bits because a hex digit is four bits. There is no way to write a bit-vector literal whose length adapts.

A SystemVerilog literal has three parts: an optional width, a base, and the digits. `8'hA5` is eight bits wide. `'hA5` without the width is unsized: it takes its width from the context, and where there is no context it is at least 32 bits. A bare number such as 255 is an unsized decimal, which means a 32-bit signed `int`.

Then there are the special unsized fill literals, added in SystemVerilog-2005, which are the most useful literals in the language:

```
1 logic [31:0] a;
2 a = '0;      // 32 zeros   -- replaces (others => '0')
3 a = '1;      // 32 ones    -- replaces (others => '1')
4 a = 'x;      // 32 x
5 a = 'z;      // 32 z
6 a = 1'b1;    // 32'h0000_0001 -- NOT all ones
```

Listing 2.4. The fill literals.

△ Trap: '1 and 1'b1 are not the same thing

'1 fills every bit of the target. 1'b1 is a one-bit value that is zero-extended to the target. In an eight-bit context the first is 8'hFF and the second is 8'h01. The same distinction bites in comparisons:

```
1 logic [7:0] mask;
2 if (mask == '1) ... // true when mask is 8'hFF -- what you meant
3 if (mask == 1'b1) ... // true when mask is 8'h01 -- almost never
4 if (mask == 1) ... // same as above: 1 is a 32-bit int
```

The VHDL translation of (others => '1') is '1, always. Writing 1'b1 where you meant '1 produces code that is legal, silent, and wrong.

△ Trap: An unsized literal in a wide expression

'1 has no width of its own: it takes the width of its context. That is usually what you want, but the context is not always what you think.

```
1 logic [63:0] wide;
2 wide = {32'h0, '1}; // inside a concatenation every operand is
3 // self-determined. '1 has no size -> error or
4 // an implementation-defined width. Write 32'hFFFF_FFFF.
```

Unsized literals are illegal as concatenation operands for exactly this reason. Some tools accept them and pick a width. Always give an explicit width inside {}.

2.5.2 Aggregates and assignment patterns

VHDL

```
constant ZERO : std_logic_vector(7 downto 0)
:= (others => '0');
constant ONE : std_logic_vector(7 downto 0)
:= (others => '1');
constant M : std_logic_vector(7 downto 0)
:= (3 downto 0 => '1', others => '0');
constant P : std_logic_vector(3 downto 0)
:= ('1', '0', '1', '0');
constant N : std_logic_vector(3 downto 0)
:= (3 => '1', 1 => '1', others => '0');

type pair_t is record
  x : integer; y : integer;
end record;
constant C : pair_t := (x => 1, y => 2);
```

SystemVerilog

```
localparam logic [7:0] ZERO = '0;
localparam logic [7:0] ONE = '1;
localparam logic [7:0] M = 8'h0F;
// or: {4'b0, {4{1'b1}}}}
localparam logic [3:0] P = 4'b1010;
localparam logic [3:0] N = 4'b1010;

typedef struct packed {
  int x; int y;
} pair_t;
localparam pair_t C = '{x:1, y:2};
```

For packed vectors the VHDL aggregate almost always collapses to an ordinary literal in SystemVerilog, because a packed vector *is* a number. (others => '0') becomes '0. A named aggregate that sets specific bits becomes a hexadecimal constant, a concatenation, or a part-select assignment:

```
1 logic [31:0] ctrl;
2 always_comb begin
3   ctrl = '0; // (others => '0')
4   ctrl[3:0] = 4'hF; // (3 downto 0 => '1', ...)
5   ctrl[15:8] = len_q; // a named field
```

```
6 end
```

Listing 2.5. Setting a field of a vector.

The `'{...}` form, the *assignment pattern*, is the true equivalent of a VHDL aggregate, and it applies to unpacked arrays, structures and unions. Note the leading apostrophe: `{1,2,3}` is a concatenation, `'{1,2,3}` is an assignment pattern, and they mean entirely different things.

```
1 typedef struct packed { logic [7:0] a; logic [7:0] b; } two_t;
2 two_t s = '{a: 8'h12, b: 8'h34}; // named, like VHDL's (a => ..., b => ...)
3 two_t t = '{8'h12, 8'h34}; // positional
4
5 logic [7:0] mem [0:3] = '{8'h00, 8'h11, 8'h22, 8'h33};
6 logic [7:0] all9 [0:3] = '{default: 8'h99}; // like (others => x"99")
7 logic [7:0] two [0:3] = '{2{8'hAA, 8'hBB}}; // replicated pattern
```

Listing 2.6. Assignment patterns for composite types.

`'{default: v}` is the closest thing to `(others => v)` for an unpacked array or a structure, and it works at any nesting depth.

2.5.3 Concatenation and replication

VHDL

```
y <= a & b & c;

-- replication needs a loop or an
-- aggregate:
y <= (others => a(7));
-- or
y <= a(7) & a(7) & a(7) & a(7);

-- sign extension by hand:
s16 <= (15 downto 8 => s8(7)) & s8;
-- or, with numeric_std:
s16 <= std_logic_vector(
    resize(signed(s8), 16));
```

SystemVerilog

```
assign y = {a, b, c};

// replication operator:
assign y = {4{a[7]}};

// nested:
assign y = {{4{a[7]}}}, a[3:0];

// sign extension:
assign s16 = {{8{s8[7]}}}, s8;
// or, since s8 is signed:
assign s16 = $signed(s8);
```

`&` becomes `{}`. Note that `&` in SystemVerilog is bitwise AND, so mistranslating a concatenation gives you a legal expression with a completely different meaning; the widths usually differ, but as Section 2.6 explains, differing widths are not an error.

The replication operator `{N{expr}}` has no VHDL counterpart short of an aggregate or a loop, and it is used constantly, above all for sign extension. The double brace is not a typo: the outer braces are the concatenation, the inner ones delimit the replicated expression. `N` must be a constant expression.

2.6 Operators, expressions and the width rules

If you read only one section of this chapter, read this one. The operator tables are boring and the width rules are not, and the width rules are where VHDL designers lose days.

2.6.1 The operators

Three rows deserve attention.

Table 2.4. Operators.

VHDL	SystemVerilog	Note
and or xor xnor nand nor not —	& ^ ~^ ~& ~ ~ && !	Bitwise. Bitwise. Logical (result is one bit). VHDL uses the same words for both.
= /=	== !=	x on either side gives x.
—	=== !==	Four-state identity: compares x and z literally.
—	==? !=?	Wildcard: x/z in the right operand match anything.
?= ?/= (2008)	==? !=?	Nearest equivalent.
< <= > >=	< <= > >=	Signedness comes from the operands in SystemVerilog.
+ - * / mod rem **	+ - * / % **	% follows the sign of the dividend, like rem.
abs	—	No operator. Write (x[N-1] ? -x : x).
sll srl sla sra rol ror	<< >> << >>	No rotate. >> is arithmetic only on a signed operand.
&	{}	Concatenation.
—	{N{x}}	Replication.
—	&x x ^x	Unary reduction over all bits. No VHDL-2008 equivalent except and_reduce etc.
when else	?:	Conditional expression.

Logical versus bitwise. VHDL uses `and` for both, and the type system tells them apart: `and` on two booleans is logical, on two vectors is bitwise. SystemVerilog has separate operators. `a & b` on two eight-bit vectors gives an eight-bit vector; `a && b` gives one bit, true if both operands are non-zero. Writing `if (a & b)` where you meant `&&` is legal and usually still works, which makes it a habit that survives until the day it does not.

The four-state comparisons. `==` propagates unknowns: if either operand contains an `x`, the result is `x`, and `if` treats `x` as false. `===` compares the four-state values bit by bit and always returns 0 or 1, so `a === 8'hxx` is a way of asking “is a exactly all-x”. `===` is not synthesisable and belongs in testbenches only. VHDL has no equivalent to `==` because `std_logic` equality already compares the nine values literally: VHDL’s `=` behaves like SystemVerilog’s `===`, not like `==`. This is a real difference in behaviour, not just in spelling.

Reduction operators. `|v` is one bit, true if any bit of `v` is set. `&v` is true if all are. `^v` is the parity. VHDL-2008 added `or_reduce`, `and_reduce` and `xor_reduce` for the same purpose but the SystemVerilog form is one character.

2.6.2 Width rules: the part that matters

VHDL’s rule is short. The operands of `+` on `unsigned` must be of the same length or one of them must be an integer; the result has the length of the longer operand; assigning it to a target of a different length is an error. Every width is written down and every mismatch is a compile error.

SystemVerilog has no such rule. Instead, every operand of an expression is classified as *self-determined* or *context-determined*. A self-determined operand is evaluated at its own natural width. A context-determined operand is first widened to the width of the whole expression, and the width of the whole expression is the maximum of all its context-determined operands *and, for an assignment, of the left-hand side*. Operands are then zero-extended or sign-extended to

that width, the operation happens, and the result is truncated to fit the target.

That is the whole rule, and it has three consequences you must internalise.

VHDL

```
signal a, b : unsigned(7 downto 0);
signal c    : unsigned(15 downto 0);

-- illegal: lengths differ
-- c <= a + b;

-- you must say what you mean:
c <= resize(a, 16) + resize(b, 16);
c <= "00000000" & (a + b); -- 8-bit
                                -- add
```

SystemVerilog

```
logic [7:0] a, b;
logic [15:0] c;

// legal. The 16-bit target makes
// the addition 16 bits wide, so
// a and b are zero-extended
// first and no carry is lost:
assign c = a + b;
```

Consequence one: the target width propagates into the expression. This is the opposite of what most VHDL designers expect, and it is usually helpful. `c = a + b` with a sixteen-bit `c` really does compute a sixteen-bit sum. The carry out of bit 7 is kept.

Consequence two: the propagation stops at self-determined operands. The operands that are self-determined are the ones inside a concatenation or a replication, the right-hand operand of a shift, the condition of `?:`, the index of a part-select, an argument of a function or a task, and a port connection. In every one of those places the expression is evaluated at its own width and the surrounding context is ignored.

△ Trap: Where context stops: the carry you did not get

```
1 logic [7:0] a, b;
2 logic [15:0] c;
3 logic      cout;
4
5 assign c    = a + b;           // 16-bit add. Carry kept. Fine.
6 assign c    = {8'h00, a + b}; // 8-bit add inside {}. Carry LOST.
7 assign cout = (a + b) >> 8;    // the whole expression is 8 bits wide, so
8                                // a+b wraps and >>8 is always 0. LOST.
9 assign {cout, c[7:0]} = a + b; // 9-bit LHS -> 9-bit add. Carry kept.
```

The rule of thumb that protects you in every case, whatever the context, is to widen an operand explicitly. `{1'b0, a} + b` is nine bits wide by itself, regardless of where you put it, and it is exactly the SystemVerilog spelling of `resize(a,9) + resize(b,9)`. Get into the habit and the width rules stop being able to hurt you.

Recipe: Widen the operand, never trust the target

```
1 // Full adder result with explicit carry, safe anywhere:
2 assign sum9 = {1'b0, a} + {1'b0, b} + cin;
3
4 // Signed multiply-accumulate, safe anywhere:
5 assign acc = $signed({8{a[7]}}, a) * $signed({8{b[7]}}, b);
```

Writing the extension by hand costs you a handful of characters and makes the expression's width independent of where it appears. This is the single most useful defensive habit in SystemVerilog RTL.

Consequence three: truncation is silent. Assigning a wide expression to a narrow target drops the high bits with no message from most simulators. Verilator warns (`WIDTHTRUNC`,

WIDTHEXPAND) and this is one of the strongest arguments for lint-checking every file with it. Treat a Verilator width warning as an error.

2.6.3 Signedness

The signedness rule is one line, and it is the source of the second family of disasters:

If any operand of an expression is unsigned, the whole expression is unsigned.

Signedness is decided over the whole expression, before any values are computed, and it is decided pessimistically. Two more facts complete the picture. First, a concatenation is *always* unsigned, whatever it contains. Second, a part-select or bit-select is always unsigned, even of a signed vector: `s[7:0]` is unsigned even if `s` is `logic signed [7:0]`, though the whole-vector reference `s` is signed.

△ Trap: One unsigned operand poisons the whole expression

```
1 logic signed [7:0] a;      // want -3
2 logic          [7:0] b;    // want 5
3 logic signed [8:0] r;
4
5 assign a = -8'sd3;
6 assign b = 8'd5;
7 assign r = a + b;          // b is unsigned -> the WHOLE add is unsigned.
8                             // a is zero-extended, not sign-extended:
9                             // 9'd253 + 9'd5 = 9'd258, not 2.
10
11 assign r = a + $signed(b); // correct: 9'sd(-3) + 9'sd5 = 2
12
13 logic signed [7:0] x, y;
14 assign r = {x, 1'b0} + y; // concatenation is unsigned -> whole add unsigned
```

The compiler says nothing. The waveform shows a plausible-looking number. This is the bug that `numeric_std` exists to prevent, and SystemVerilog gives you no protection whatsoever: declare every signed quantity `logic signed`, and put `$signed()` around anything you mix with it.

△ Trap: Unsigned comparison with a negative

```
1 logic [7:0] cnt;
2 if (cnt - 1 < 0) ... // NEVER true: unsigned result cannot be < 0
3 if (cnt < -1) ...  // -1 is a 32-bit signed int, but cnt is unsigned,
4                     // so the comparison is done unsigned:
5                     // cnt < 32'hFFFF_FFFF -> always true
```

VHDL is no better on the first of these. `numeric_std` defines both `"-"` (UNSIGNED, NATURAL) and `"<"` (UNSIGNED, NATURAL), so `cnt - 1 < 0` analyses cleanly and is likewise always false; only a lint tool will tell you. VHDL does save you from the second: the only matching operator takes a NATURAL on the right, and `-1` is outside that subtype, so `cnt < -1` is a bounds violation. An analyser will normally warn about it, and it is a hard error when the expression is evaluated. SystemVerilog accepts both, and the second is worse than useless: it is a tautology that the tool will happily optimise away, taking your intended logic with it.

2.6.4 The width of a comparison and of a ternary

The result of a comparison is always one bit. Its *operands*, however, are context-determined with each other: they are widened to the maximum of the two, and only then compared. So $a > b$ with an eight-bit a and a sixteen-bit b does a sixteen-bit comparison, which is what you want. Beware the intermediate: $(a + b) > c$ sizes $a + b$ against c , so if c is eight bits the addition wraps at eight bits before the comparison happens.

The conditional operator is different again. In $\text{cond} ? x : y$, the condition is self-determined, and x and y are context-determined with the whole expression *and with each other*. If x is signed and y is unsigned, the result is unsigned, and both are treated as unsigned.

```

1 logic [7:0] a, b;
2 logic [15:0] c;
3 logic [15:0] r;
4
5 if ((a + b) > c) ... // 16-bit add (c is 16 bits) then 16-bit compare
6 if ((a + b) > 8'd200) ... // 8-bit add: wraps. 8'd200 is 8 bits.
7 if ((a + b) > 200) ... // 200 is a 32-bit int: 32-bit add. No wrap.
8
9 assign r = sel ? a : c; // a is widened to 16 bits (context = r and c)
10 assign r = {8'h0, sel ? a : b}; // inside {}: self-determined, 8-bit result

```

Listing 2.7. Widths in comparisons and conditionals.

The three `if` lines differ only in how the constant is written, and they compute three different things. That is the width rule in one picture.

2.7 Arrays, records and enumerations

2.7.1 Packed and unpacked

SystemVerilog splits the idea of an array in two, and this split has no VHDL counterpart at all. Dimensions written *before* the object name are *packed*; dimensions written *after* it are *unpacked*.

VHDL	SystemVerilog
<pre> -- an array of 4 bytes type mem_t is array (0 to 3) of std_logic_vector(7 downto 0); signal m : mem_t; -- element access m(2) -- one byte m(2)(7) -- one bit -- VHDL makes no distinction -- between "packed" and -- "unpacked": the array is the -- array. </pre>	<pre> // unpacked: 4 separate bytes logic [7:0] u [0:3]; // packed: one 32-bit vector // viewed as 4 bytes logic [3:0][7:0] p; // element access is identical u[2] p[2] // one byte u[2][7] p[2][7] // one bit // but only p can do this: p[19:4] // arbitrary bit slice p + 1 // arithmetic </pre>

Figure 2.1 is the picture to keep in mind. A packed array is a single vector with a shape; it is stored contiguously, it can be sliced at any bit boundary, it can be used in arithmetic, and it can be cast to and from any other packed type of the same total width. An unpacked array is a collection of separate objects; it is what you want for a memory, and it is what synthesis tools recognise as a RAM.

The VHDL translation rule is simple in practice. A VHDL array of vectors that you use as a

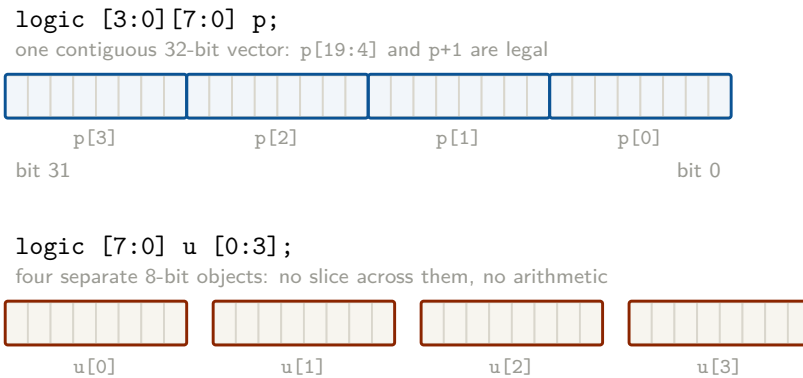


Figure 2.1. Packed and unpacked dimensions. A packed array is a bit vector with a grouping imposed on it; an unpacked array is a collection of separate objects. VHDL has only the second kind, and gets the first through explicit conversion functions.

Table 2.5. What you may do with each kind.

Operation	Packed	Unpacked
Whole-object assignment	yes	yes (types must match exactly)
Arithmetic and bitwise operators	yes	no
Arbitrary bit slice	yes	no
Element and part select	yes	yes (one element at a time)
Module port	yes	yes
Cast to another type	yes	only through a bit-stream cast
Element type may be unpacked	no	yes
Inferred as a RAM	no (it is one big register)	yes
<code>foreach</code> iteration	yes	yes

memory becomes an unpacked array of packed vectors. A VHDL array of vectors that you slice, concatenate or convert to a flat vector becomes a packed array.

```

1 // A 256 x 32 memory. Unpacked outer dimension: synthesis infers a RAM.
2 logic [31:0] ram [0:255];
3 always_ff @(posedge clk) if (we) ram[addr] <= wdata;
4
5 // An 8-lane bus you also want as one 256-bit word. Packed both ways.
6 logic [7:0][31:0] lanes;
7 assign flat_256 = lanes;           // legal: same total width
8 assign lanes[3] = something;      // per-lane access still works

```

Listing 2.8. Memories and buses.

△ Trap: Two dimensions, two meanings, one syntax

`logic [3:0] x [0:7]` is eight four-bit words. `logic [7:0][3:0] y` is one thirty-two-bit vector grouped into eight nibbles. `logic [3:0][7:0] z` is one thirty-two-bit vector grouped into four bytes. They are different types and the difference is which side of the name the dimension is on.

Worse, the index order reverses. For `logic [1:0][7:0] q`, `q[1]` is the *upper* byte, bits 15 down to 8; the leftmost packed dimension is the outermost. For unpacked arrays the leftmost dimension is also the outermost. When you have both, you read left to right through the packed dimensions and then left to right through the unpacked ones, but the packed ones are written before the name and the unpacked ones after it, so on the page the order is reversed. Write the declaration, then write `$bits()` of it in a quick testbench and check.

2.7.2 Multidimensional arrays

VHDL

```

-- a true 2-D array of scalars
type grid_t is array
  (0 to 3, 0 to 7) of std_logic;
signal g : grid_t;
-- indexed with a comma:
g(2, 5)

-- an array of arrays
type row_t is array (0 to 7)
  of std_logic;
type grid2_t is array (0 to 3)
  of row_t;
signal h : grid2_t;
h(2)(5)

```

SystemVerilog

```

// SystemVerilog has no comma-
// indexed arrays. Every multi-
// dimensional array is an array
// of arrays.
logic g [0:3][0:7];
g[2][5]

// same thing, packed:
logic [3:0][7:0] h;
h[2][5]

// slices of unpacked arrays:
logic row [0:7];
assign row = g[2]; // legal, but see below

```

VHDL's comma-indexed multidimensional array has no SystemVerilog equivalent, and it loses nothing: the array-of-arrays form does everything the comma form does and also lets you name and pass a row. SystemVerilog allows unpacked array slicing (`g[2]` above yields a whole row) and whole-array assignment when the types match exactly, which VHDL also allows. Tool support for the slice in a *continuous* assignment is uneven: Verilator 5.020 accepts `assign row = g[2];`, while Icarus Verilog 12 replies sorry: Array slices are not yet supported for continuous assignment. Inside an `always_comb` block both are happy.

2.7.3 Records and structures

VHDL

```

type req_t is record
    valid : std_logic;
    addr  : std_logic_vector(31 downto 0);
    data  : std_logic_vector(31 downto 0);
end record req_t;

signal r : req_t;

r.valid <= '1';
r <= (valid => '1',
     addr  => a,
     data  => d);

-- a record cannot be treated as
-- a bit vector without a
-- conversion function you write
-- yourself.

```

SystemVerilog

```

typedef struct packed {
    logic    valid;
    logic [31:0] addr;
    logic [31:0] data;
} req_t;           // 65 bits

req_t r;

r.valid = 1'b1;
r = '{valid: 1'b1,
     addr:  a,
     data:  d};

// a packed struct IS a bit vector
assign flat65 = r;
assign r      = flat65;
assign r[63:32] = a;    // legal too

```

The correspondence is close, with one important addition. A VHDL **record** is always what SystemVerilog calls unpacked: it is a collection of named fields with no defined bit layout, and turning it into a vector requires a conversion function you write. A SystemVerilog **struct packed** has a defined layout: the first field occupies the most significant bits, the last field the least significant, and the whole structure is a bit vector of the sum of the field widths. That makes packed structures the natural way to describe a bus transaction, a register field layout, or a pipeline stage payload, and it is one of the genuinely good things SystemVerilog gives you.

Restrictions: a packed structure may contain only packed types, so no unpacked arrays, no **real**, no **string**. An unpacked structure (write struct without **packed**) may contain anything, can cross a port, but cannot be used in arithmetic or sliced.

Unions follow the same split. A **union packed** requires all members to be the same width and gives you a reinterpreting view of the same bits, which is the classic way to describe an instruction word. A plain union is unpacked and needs \$cast or tagged discipline. VHDL has no union at all.

```

1  typedef struct packed { logic [6:0] funct7; logic [4:0] rs2, rs1;
2                          logic [2:0] funct3; logic [4:0] rd;
3                          logic [6:0] opcode; } r_type_t;
4  typedef union packed {
5      logic [31:0] raw;
6      r_type_t    r;
7  } instr_t;
8
9  instr_t i;
10 assign i.raw    = fetch_data;
11 assign is_add   = (i.r.opcode == 7'h33) && (i.r.funct3 == 3'b000);

```

Listing 2.9. A packed union: two views of the same 32 bits.

2.7.4 Enumerations

VHDL

```

type state_t is
  (IDLE, RUN, DONE);
signal s : state_t;

s <= IDLE;
if s = RUN then ... end if;

state_t'pos(RUN)    -- 1
state_t'val(1)      -- RUN
state_t'succ(IDLE)  -- RUN
state_t'pred(DONE)  -- RUN
state_t'image(s)    -- "run"
state_t'left        -- IDLE
state_t'right       -- DONE

```

SystemVerilog

```

typedef enum logic [1:0] {
  IDLE, RUN, DONE
} state_t;
state_t s;

s = IDLE;
if (s == RUN) ...

int'(RUN)      // 1
state_t'(1)    // RUN
s.next()       // successor of s
s.prev()       // predecessor
s.name()       // "RUN"
s.first()      // IDLE
s.last()       // DONE
s.num()        // 3

```

The concepts match, the spelling does not. Two differences matter.

First, the base type. A VHDL enumeration is its own type with an implicit integer position, and synthesis picks the encoding. A SystemVerilog enumeration has a base type, which defaults to `int` if you do not give one. That default is wrong for RTL: a state variable of type `int` is thirty-two bits, is two-state, and starts at zero rather than `x`, so a missing reset is invisible. Always write the base type:

```

typedef enum logic [N-1:0] {...} state_t; Never let an RTL enumeration default to int.

```

Second, the strictness. VHDL will not let you assign an integer to an enumeration without `'val`. SystemVerilog will not either, in principle: assigning an integer expression to an enumeration variable requires an explicit cast `state_t'(expr)`. This is one of the very few places where SystemVerilog is as strict as VHDL, and it is worth knowing because it surprises people who expect SystemVerilog to convert everything. Assigning *from* an enumeration to an integer or a vector is implicit and needs no cast.

The strictness stops at the cast, though, and this is where VHDL is still ahead. `state_t'(2'b11)` compiles and produces a variable holding a value that has no name. VHDL's type simply has no such value, and `'val` with an out-of-range position is a run-time error. If the source of the integer is untrusted, use `$cast`, which checks and reports failure rather than storing a nameless value; Section 2.8 gives the form.

Explicit encodings work in both:

```

1 typedef enum logic [3:0] {
2   IDLE = 4'b0001,
3   ADDR = 4'b0010,
4   DATA = 4'b0100,
5   RESP = 4'b1000
6 } state_e;
7 state_e state_q, state_d;
8 // $onehot(state_q) is then a useful assertion

```

Listing 2.10. A one-hot encoded state type.

The methods in the right-hand pane are the whole set, and they are worth learning together because the VHDL attributes you would reach for are spread across three different mechanisms

in SystemVerilog.

`.name()` returns the enumeration label as a `string`, and it is the direct replacement for the VHDL attribute `'image`. The standard way to print a state in a message is

```
1 $display("state = %s at %0t", state_q.name(), $time);
```

It returns an empty string if the value does not correspond to a label, which is itself a useful check, and it alone justifies giving a state machine an enumerated type: every log message and every assertion failure prints the state by name with no lookup table.

`.next()` and `.prev()` are `'succ` and `'pred` with one important difference: they *wrap round*. `DONE.next()` is `IDLE`, where `state_t'succ(DONE)` is an error in VHDL. Both take an optional count, so `s.next(2)` skips one member. The wrap-around makes “advance until you reach `last()`” an infinite loop if the exit test is written carelessly; iterate over the members by count instead:

```
1 state_t s = state_t'(0);
2 for (int unsigned i = 0; i < s.num(); i++) begin
3     $display("%0d: %s", i, s.name());
4     s = s.next();
5 end
```

`.num()` is the number of members and has no VHDL attribute counterpart; `.first()` and `.last()` are `'left` and `'right`. Note that all six are methods called on a *value*, not attributes of a type, so you need an object of the type to call them, and `state_t::num()` is not legal.

2.8 Type conversion

VHDL

```
-- closely related array types:
u <= unsigned(v);
v <= std_logic_vector(u);

-- numeric conversions:
i <= to_integer(u);
u <= to_unsigned(i, 8);
s <= to_signed(i, 8);

-- real and integer:
r <= real(i);
i <= integer(r);      -- rounds

-- enumeration:
p <= state_t'pos(st);
st <= state_t'val(p);

-- anything else needs a function
-- you write yourself.
```

SystemVerilog

```
// related packed types: nothing
assign u = v;
assign v = u;

// numeric: implicit
assign i = u;
assign u = i;
assign s = i;

// real and integer: implicit,
// rounds to nearest
assign r = i;
assign i = r;

// enumeration:
assign p = int'(st);
assign st = state_t'(p);

// static cast, any packed type:
assign q = my_t'(expr);
```

SystemVerilog has four conversion mechanisms, in increasing order of ceremony.

Implicit conversion, which happens whenever an expression of one type is assigned to an object of another and the two are compatible. Between packed types, “compatible” means “both are collections of bits”, so almost everything converts. Width mismatches are fixed by truncation or extension, signedness by the rule in Section 2.6. This is where the majority of SystemVerilog bugs come from.

The **static cast** `type'(expr)`, which is the direct analogue of a VHDL type mark. It is checked at compile time. It works on any packed type, on enumerations, and on `int/real` pairs. Note the apostrophe: the syntax is the type name, an apostrophe, then the parenthesised expression, and it looks disconcertingly like a VHDL attribute.

The **size cast** `16'(expr)` and the sign casts `$signed(expr)`, `$unsigned(expr)`, which change one property and leave the rest alone. `16'(a+b)` is the cleanest way to force an expression to a particular width without writing a concatenation, and it is much clearer than relying on the target.

The **dynamic cast** `$cast(dest, src)`, which checks at run time and either assigns and returns 1, or leaves the destination alone and returns 0. Use it when converting an integer to an enumeration where the value may not be a valid label, and, in the *Verification Cookbook*, for downcasting class handles. It is a function when you want the result and a task when you do not; used as a task, a failed cast is an error.

```

1 typedef enum logic [1:0] {A, B, C} e_t;
2 e_t      e;
3 logic [1:0] v;
4 int      i;
5 logic [15:0] w;
6
7 assign v = e;           // implicit: enum to vector, always allowed
8 // e = v;              // ILLEGAL: needs a cast
9 assign e = e_t'(v);     // static cast: no run-time check. v = 3 gives
10                        // an out-of-range enum, silently.
11 initial if (!$cast(e, v)) // dynamic cast: returns 0 if v is not A, B or C
12     $error("bad state code %0d", v);
13 assign w = 16'(v);      // size cast: zero-extend to 16
14 assign i = $signed(v);  // sign cast: treat v's bits as signed

```

Listing 2.11. The four mechanisms side by side.

△ Trap: SystemVerilog converts where VHDL refused to compile

Every one of these produces a compile error in VHDL and a silent conversion in SystemVerilog:

```

1 logic [7:0] a; logic [15:0] b; logic [3:0] c;
2 assign a = b;           // truncated to 8 bits
3 assign b = a;           // zero-extended to 16 bits
4 assign c = a + b;        // 16-bit add, then truncated to 4 bits
5 assign a = 1'b1;         // 8'h01
6 assign a = -1;           // 8'hFF
7 real r; assign r = a;    // vector to real
8 int i; assign i = r;     // real to int, rounded

```

None of them is a mistake by itself. All of them are mistakes when they were not intended, and nothing in the language tells you which is which. The compensating discipline is: run Verilator with `-Wall` over every file, and treat `WIDTHTRUNC` and `WIDTHEXPAND` as errors unless you have deliberately written a cast to say the conversion is wanted.

Bit-stream casts

SystemVerilog also supports *bit-stream* casts, which flatten an arbitrary aggregate, including unpacked arrays and unpacked structures, into a bit vector and back:

```

1 typedef struct { logic [7:0] a; logic [7:0] b [0:3]; } msg_t; // unpacked

```

```

2 msg_t      m;
3 logic [39:0] flat;
4 assign flat = {>>{m}};           // pack, left to right
5 assign m    = {>>{flat}};       // unpack

```

This has no VHDL equivalent short of writing the serialisation by hand. It is not synthesisable and it is meant for testbenches: it is the standard way to turn a transaction object into a byte stream for a scoreboard or a file. The {«{...}} form streams right to left, which is how you reverse a bit or byte order in one expression.

Type conversion is the natural place to stop, because it is where the static half of the language runs out. Everything this chapter has described is decided before the simulation starts: a width, a signedness, an encoding, a layout in bits. None of it says when anything happens.

That is the subject of Chapter 3, and the two halves meet almost immediately. A continuous assignment applies the width rules of Section 2.6 on every input change; an `always_ff` decides whether a truncation happens once per clock or never; the packed structure of Section 2.7 is what crosses a port in an instantiation. Keep the rules below to hand while you read it, because the next chapter assumes them rather than restating them.

2.9 Checklist

Operational takeaways for the static half of the language. The behavioural ones are at the end of Chapter 3, and the complete two-part mapping table is with them.

- **Every file starts** with ``timescale 1ns/1ps` and ``default_nettype none`, and ends with ``default_nettype wire`.
- **Ports:** input wire, output logic, inout wire. ANSI style only. Connect every port explicitly; SystemVerilog has no port defaults and an unconnected input is `z`.
- **Generics become parameters**, architecture constants become `localparam`. Never write `defparam`.
- **logic for everything** you drive from one place; `wire` only where you genuinely need resolution.
- **Declare signedness once**, in the declaration, and never mix a signed and an unsigned operand in one expression without a cast.
- **Widen before you add.** `{1'b0, a} + b` for the carry; assigning to a wider target does not create one.
- **>> on signed values**, always. Check every shift.
- `'0` and `'1` for context-width fills; a bare 0 is 32 bits and will surprise you inside a concatenation.
- **A packed struct is a bit vector** with names. Use it wherever you would have written a VHDL record on a port.
- **Give every enumeration a base type** and, in RTL, explicit encodings.
- **Lint every file** with `verilator --lint-only -Wall`. It is the type checker SystemVerilog does not have.

The Rosetta Stone: Behaviour

The second half of the reference core. Everything that *describes behaviour*: concurrent assignments, processes, sequential statements, generate, instantiation, subprograms, packages, attributes, reporting and time. It ends with the complete VHDL-to-SystemVerilog mapping table for both halves of the Rosetta Stone, which is the page to photocopy.

Chapter 2 dealt with the static half of the language. Four of its results are assumed on every page of this one, so they are worth restating before anything moves.

- `logic` is a single-driver four-state variable, the counterpart of `std_ulogic`; `wire` is the resolved net you need only for a genuine multi-driver bus.
- Widths never have to match. An assignment, a port connection or an operand of the wrong width is truncated or extended in silence, and the width of an arithmetic expression is decided partly by the thing you assign it to (Section 2.6).
- Signedness is a property of an expression, not of a variable alone. One unsigned operand makes the whole expression unsigned.
- `'0` is the fill literal that replaces `(others => '0')`, and it is not the same as `1'b0`.

Everything in this chapter is about the other axis: what happens over time, and in what order. The layout is the same as before, a *rosetta* block with VHDL on the left and SystemVerilog on the right, then prose that says what is the same, what is different, and what is missing.

The traps in this half are the more expensive ones, because a type error stops the analyser and a behavioural error does not stop anything at all: a latch you did not mean to infer, a race between two `always` blocks, a function whose local variable is shared between two callers. Chapter 1 explained why the language works this way; here it is as a set of translations.

3.1 Concurrent statements

Everything between `begin` and `end architecture` in VHDL is concurrent. Everything between the port list and `endmodule` in SystemVerilog is too. The set of concurrent statements is almost the same.

3.1.1 Simple, conditional and selected assignment

VHDL

```
-- simple
y <= a and b;

-- conditional (when/else)
y <= a when sel = '1' else b;

z <= "00" when op = "00" else
     "01" when op = "01" else
     "10" when op = "10" else
     "11";

-- selected (with/select)
with op select z <=
  "00" when "00",
  "01" when "01",
  "10" when "10",
  "11" when others;
```

SystemVerilog

```
// simple
assign y = a & b;

// conditional
assign y = sel ? a : b;

assign z = (op == 2'b00) ? 2'b00 :
           (op == 2'b01) ? 2'b01 :
           (op == 2'b10) ? 2'b10 :
           2'b11;

// selected: no concurrent form.
// Use always_comb with case:
always_comb begin
  unique case (op)
    2'b00: z = 2'b00;
    2'b01: z = 2'b01;
    2'b10: z = 2'b10;
    default: z = 2'b11;
  endcase
end
```

VHDL's simple signal assignment becomes SystemVerilog's **assign**, the *continuous assignment*. A continuous assignment drives its target whenever any input changes, exactly like a concurrent signal assignment. The target must be a net or a variable with no other driver, which matches VHDL's single-driver rule for unresolved signals.

The conditional form maps to the ternary operator. Chained ternaries become unreadable quickly; past two levels, write an `always_comb` with an `if/else` if chain. The selected form has no concurrent equivalent at all, and its natural translation is `always_comb` with a `case`.

Note a good VHDL habit that survives: `with/select` forces you to cover every choice, and the translated `case` should keep that discipline. Use `unique case` plus a `default`, or a `default` alone. Never write a `case` inside `always_comb` without a `default` unless every possible value of the selector is listed; the missing branch infers a latch.

VHDL-2008 allows `when/else` and `with/select` inside a process as sequential statements. SystemVerilog has always allowed the ternary operator anywhere an expression is allowed, so this needs no translation.

3.1.2 Delays

VHDL

```
-- inertial (the default): pulses
-- shorter than the delay are
-- swallowed
y <= a after 3 ns;

-- transport: every change is
-- propagated
y <= transport a after 3 ns;

-- explicit pulse rejection limit
y <= reject 1 ns inertial a
    after 3 ns;
```

SystemVerilog

```
// continuous assignment delay:
// inertial-like
assign #3 y = a;

// non-blocking assign with an
// intra-assignment delay:
// transport-like
always @(a) y <= #3 a;

// blocking, delay before the
// assignment: NOT a delay model,
// the process is simply blocked
always @(a) #3 y = a;

// gate/net delay:
wire #3 y;

// SystemVerilog has no keywords
// for transport vs inertial.
```

VHDL distinguishes inertial and transport delay explicitly, and the default for a signal assignment is inertial: a pulse narrower than the delay does not propagate. SystemVerilog has no keywords for the distinction. A delay on a continuous assignment behaves inertially: a new value arriving before the old one has propagated cancels it. The closest thing to transport delay is an *intra-assignment* delay on a non-blocking assignment, `y <= #3 a;`: the right-hand side is sampled immediately and the update is scheduled three time units later, so several pulses can be in flight at once and none is swallowed.

Do not confuse that with `#3 y = a;`, where the delay comes before the assignment. That form models no delay at all. The process stops for three time units, and every change of `a` during those three units is missed, because the process is not back at its event control to see them. It is neither inertial nor transport: it is a blocked process.

None of this matters for synthesisable code. Synthesis ignores every delay, and delays in RTL are a mistake, because they make simulation and synthesis disagree. The only place a delay belongs is in a testbench, where the distinction almost never arises. Learn the syntax so you can read gate-level netlists produced by a synthesis tool, and do not write it.

3.1.3 Blocks and named scopes

VHDL

```
mult_stage : block
    signal partial : unsigned(15 downto 0);
begin
    partial <= a * b;
    p <= std_logic_vector(partial);
end block mult_stage;

-- a guarded block (obsolete):
-- b : block (guard_expr) begin
```

SystemVerilog

```
// named begin/end block:
begin : mult_stage
    // declarations allowed here
end : mult_stage

// but at module level you want
// a generate block:
if (1) begin : mult_stage
    logic [15:0] partial;
    assign partial = a * b;
    assign p      = partial;
end
```

A VHDL `block` groups concurrent statements and gives them a local declarative region and a

name in the hierarchy. SystemVerilog’s nearest equivalent at module level is a `generate` block, which is what Section 3.4 is about. Inside a procedural block, a named `begin : label ... end : label` gives you a local declarative region in the same way. Guarded blocks are obsolete in VHDL too and have no SystemVerilog counterpart.

3.2 Processes and procedural blocks

3.2.1 The combinational process

VHDL

```
-- classic: explicit list
comb : process (a, b, sel) is
begin
    if sel = '1' then
        y <= a;
    else
        y <= b;
    end if;
end process comb;

-- VHDL-2008:
comb2 : process (all) is
begin
    if sel = '1' then y <= a;
    else              y <= b;
    end if;
end process comb2;
```

SystemVerilog

```
// Verilog-2001 explicit list
always @(a or b or sel) begin
    if (sel) y = a;
    else    y = b;
end

// Verilog-2001 implicit list
always @* begin ... end

// SystemVerilog, the only form
// you should write:
always_comb begin
    if (sel) y = a;
    else    y = b;
end
```

`always_comb` is the direct replacement for `process (all)`, and it is better than `process (all)` in the ways that matter. Beyond computing the sensitivity list for you, it carries four extra guarantees:

1. It executes once at time zero, after all `initial` and `always` blocks have started, whether or not anything in its sensitivity set has changed. A plain `always @*` does not, so at time zero its outputs are `x` until an input moves. A VHDL process also runs once at time zero, so this restores the VHDL behaviour that `always @*` lost.
2. Its sensitivity set includes signals read inside functions it calls. `always @*` does not follow into functions, which is a classic source of missing-sensitivity bugs. VHDL-2008’s `process (all)` has the same extended rule as `always_comb`.
3. It forbids blocking timing controls. You cannot write `#5`, `@(posedge clk)` or `wait` inside it. That turns “I accidentally put a delay in my combinational logic” into a compile error.
4. Any variable written inside an `always_comb` may not be written anywhere else in the design. This is checked at elaboration and it is the SystemVerilog equivalent of VHDL’s single-driver rule for signals. It is the strongest reason to use `always_comb` rather than `always @*`.

△ Trap: `always_comb` is not exactly `process (all)`

Two differences catch people.

First, `always_comb` triggers on a change in the value of anything it reads, including through a function call, but *not* on a change to something read only through a hierarchical reference or through a `ref`-passed argument. `process (all)` has the same limitation, so in practice this is a wash. Do not put hierarchical references in combinational logic.

Second, and this is the real one: in VHDL a process assigns to *signals* using `<=`, so all the assignments in the process take effect together at the end of the delta cycle, and reading a signal you assigned earlier in the same process gives you the old value. In SystemVerilog, `always_comb` should use *blocking* assignments `=`, which take effect immediately, so reading back a variable you assigned earlier in the same block gives you the new value. The two models genuinely differ:

VHDL

```
process (all) is
begin
    t <= a;      -- signal
    y <= t;      -- OLD t!
end process;
-- y gets the previous value of t:
-- this infers a register, or
-- oscillates.
```

SystemVerilog

```
always_comb begin
    t = a;      // variable
    y = t;      // NEW t
end
// y = a. What you meant.
```

The VHDL idiom for the right-hand behaviour is a *variable* inside the process, declared in its declarative part and assigned with `:=`. A SystemVerilog `always_comb` variable behaves like a VHDL process variable, not like a VHDL signal. Keep that mapping in your head: *signal* / `<=` / *non-blocking* on one side, *variable* / `:=` / *blocking* on the other.

3.2.2 The clocked process

VHDL

```
-- synchronous reset
sync : process (clk) is
begin
    if rising_edge(clk) then
        if rst = '1' then
            q <= (others => '0');
        else
            q <= d;
        end if;
    end if;
end process sync;

-- asynchronous reset
async : process (clk, rst_n) is
begin
    if rst_n = '0' then
        q <= (others => '0');
    elsif rising_edge(clk) then
        q <= d;
    end if;
end process async;
```

SystemVerilog

```
// synchronous reset
always_ff @(posedge clk) begin
    if (rst) q <= '0;
    else    q <= d;
end

// asynchronous reset
always_ff @(posedge clk or
            negedge rst_n) begin
    if (!rst_n) q <= '0;
    else       q <= d;
end
```

The structural correspondence is exact. `rising_edge(clk)` becomes `@(posedge clk)`; `falling_edge` becomes `negedge`. The reset sensitivity is in the same place in both languages: an asynchronous reset appears in the sensitivity list, a synchronous one does not.

Two points of care.

Polarity in the sensitivity list must match the test. If the reset is active low, you write `negedge rst_n` in the list and `if (!rst_n)` in the body. Writing `posedge rst_n` with `if (!rst_n)` is legal and produces a flip-flop that resets on the wrong edge, or, more often, a synthesis error

you will not understand.

always_ff restricts you usefully. Like `always_comb`, it enforces the single-driver rule at elaboration, and it requires exactly one event control at the top of the block. It also, by convention supported by every tool, means “this is a flip-flop”: lint tools will flag a blocking assignment inside it.

△ Trap: Blocking assignment in a clocked block

```

1 always_ff @(posedge clk) begin
2   a = d;           // blocking: a takes the new value IMMEDIATELY
3   b = a;           // so b gets the NEW a: this is one flop, not two
4 end
5 always_ff @(posedge clk) begin
6   a <= d;          // non-blocking: both updates happen at the end
7   b <= a;          // b gets the OLD a: a two-stage shift register
8 end

```

The VHDL equivalent of the second block is the familiar signal behaviour, and it is what you almost always want. The first block does not just produce different hardware: because the two `always` blocks in a design execute in an unspecified order within the same time step, blocking assignments to signals read by another clocked block produce a genuine race, and two simulators can give different answers for the same code.

The rule is absolute and has no exceptions worth learning:

Non-blocking <= in every always_ff. Blocking = in every always_comb. Never mix the two in one block, and never assign the same variable from two blocks.

3.2.3 Latches

`always_latch` exists and means what it says: it has `always_comb`'s sensitivity rules but does not require every path to assign every output. Use it when you genuinely want a latch, which in synchronous design is approximately never, and rely on the tool to complain when a plain `always_comb` accidentally infers one. VHDL has no equivalent keyword; an incomplete `if` in a combinational process infers a latch there in exactly the same way and with exactly as little warning.

3.2.4 Waiting: wait, @ and

VHDL

```

wait for 10 ns;
wait until rst_n = '1';
wait on a, b;
wait;           -- forever

-- a process with no sensitivity
-- list must end in a wait
stim : process is
begin
  rst_n <= '0';
  wait for 100 ns;
  rst_n <= '1';
  wait;
end process stim;

```

SystemVerilog

```

// initial: runs once at time 0
// and then stops. No sensitivity
// list, no trailing wait needed.
initial begin
  rst_n = 1'b0;
  wait (cfg_done); // no edge
  @(a or b);       // any change
  rst_n = 1'b1; #100ns;
  $finish;
end

```

The mapping of the waiting forms is in Table 3.1.

Table 3.1. Waiting.

VHDL	SystemVerilog	Meaning
<code>wait for 10 ns</code>	<code>#10ns</code>	Suspend for a fixed time.
<code>wait on a, b</code>	<code>@(a or b)</code>	Suspend until any listed signal changes.
<code>wait until c = '1'</code>	<code>wait (c)</code>	Return immediately if already true; otherwise wait for it.
<code>wait until c = '1'</code>	<code>@(posedge c)</code>	Not the same: this always waits for an edge.
<code>wait</code>	<code>wait (0)</code>	Suspend forever. Rare in SystemVerilog.
<code>wait until rising_edge(clk)</code>	<code>@(posedge clk)</code>	The standard testbench step.
—	<code>wait fork</code>	Wait for all processes spawned by <code>fork</code> in this scope.
—	<code>disable fork</code>	Kill them instead.
—	<code>final</code>	A block that runs once at the end of simulation.

The subtle row is `wait until`. VHDL’s `wait until c = '1'` waits for an *event* on `c` that makes the condition true, so if `c` is already '1' the process still blocks. SystemVerilog’s `wait (c)` checks first and returns immediately if the condition already holds. To reproduce the VHDL behaviour exactly you need `@(posedge c)`. Getting this wrong in a reset sequence gives you a testbench that hangs, or one that does not wait at all.

`initial` has no VHDL keyword but corresponds precisely to a process with no sensitivity list that ends in a `wait`. `final` has no VHDL equivalent at all: it is a block that runs once, at the end of simulation, after the last event, and it is the right place for a coverage summary or a final scoreboard check.

The last two rows before `final` are listed for completeness only. `fork` spawns concurrent processes at run time, which a VHDL process cannot do, and `wait fork` and `disable fork` are how you join or kill them. There is no translation to give, because there is no VHDL construct on the left; the *Verification Cookbook* introduces the whole mechanism.

△ **Trap: A module-level variable is shared by every process**

A VHDL process variable is declared in the process’s declarative part and is visible only there. Two processes cannot share one by accident.

VHDL

```
p1 : process is
  variable count : integer := 0;
begin
  count := count + 1;
  wait for 10 ns;
end process p1;

-- count belongs to p1.
-- p2 cannot see it. Sharing
-- requires a shared variable
-- of a protected type.
```

SystemVerilog

```
int count;    // module scope,
              // STATIC lifetime

initial forever begin
  count = count + 1; #10ns;
end
initial forever begin
  count = count + 1; #10ns;
end
// Both processes write the same
// object. Race. No warning.
```

Every variable declared in a module has static lifetime and module scope, so “local to this process” is not the default: it is something you must arrange, by declaring the variable inside a named `begin/end` block, or inside an `automatic` task or function. `always_comb` and `always_ff` protect you for synthesisable code by making a second writer an elaboration

error, but plain `always` and `initial` blocks do not.

3.3 Sequential statements

3.3.1 Conditionals

VHDL

```
if a = '1' then
  y <= b;
elsif c = '1' then
  y <= d;
else
  y <= e;
end if;
```

SystemVerilog

```
if (a)
  y = b;
else if (c)
  y = d;
else
  y = e;
```

Three cosmetic differences and no semantic one: parentheses are required around the condition, `elsif` is spelt `else if`, and there is no `end if` because a multi-statement branch is delimited by `begin/end`. The last is the one that produces bugs, because forgetting `begin/end` silently attaches only the first statement to the branch. Always write `begin/end`, even for a single statement, and your VHDL instinct will be satisfied.

3.3.2 Case

VHDL

```
case op is
  when "00" =>
    y <= a;
  when "01" | "10" =>
    y <= b;
  when others =>
    y <= c;
end case;

-- VHDL-2008 matching case:
case? addr is
  when "1---" => sel <= '1';
  when others => sel <= '0';
end case?;
```

SystemVerilog

```
case (op)
  2'b00:
    y = a;
  2'b01, 2'b10:
    y = b;
  default:
    y = c;
endcase

// wildcard case:
casez (addr)
  4'b1??? : sel = 1'b1;
  default: sel = 1'b0;
endcase
```

The plain `case` matches. The important difference is completeness: VHDL requires a `case` to cover every value of the selector's subtype, so `when others` is compulsory unless every value is listed. SystemVerilog does not require anything, and a `case` without a `default` that fails to match simply leaves the targets unchanged, which in an `always_comb` infers a latch. Write a `default` in every combinational `case`. There is no substitute for the discipline VHDL enforced for you.

`casez` treats `z` (written `?`) in either the case expression or a case item as a wildcard. `casex` treats both `x` and `z` as wildcards. VHDL-2008's `case?` matches `'-'` in a choice and is the counterpart of `casez`.

△ Trap: Never write casex

`casex` treats `x` as a don't-care *in the case expression as well as in the items*. If the selector is unknown, because a reset was missed or an uninitialised register propagated, `casex` matches the first item anyway and the simulation proceeds happily along a branch that the silicon will not take.

```
1 logic [3:0] sel;      // x after reset
2 casex (sel)
3   4'b0001: y = a;     // MATCHES when sel is 4'bxxxx. Bug hidden.
4   4'b0010: y = b;
5   default: y = c;     // never reached
6 endcase
```

`casez` is safer because only `z` is a wildcard and `z` rarely appears by accident inside a chip. Better still, use `case (sel)` inside with `4'b0001:` style patterns, which is the SystemVerilog-2005 form and matches only where you ask. Best of all, for a decoder, write `case (1'b1)` with explicit conditions:

```
1 always_comb begin
2   unique case (1'b1)
3     sel[0]: y = a;
4     sel[1]: y = b;
5     default: y = c;
6   endcase
7 end
```

3.3.3 Case modifiers: unique, unique0, priority**VHDL**

```
-- VHDL has no modifiers. The
-- completeness requirement plus
-- the semantics of 'case' already
-- imply parallel, full behaviour.

-- pragmas are needed only to
-- persuade synthesis:
case op is          -- pragma full_case
  when "00" => y <= a;
  when "01" => y <= b;
  when others => y <= c;
end case;
```

SystemVerilog

```
// exactly one item matches:
unique case (op) ... endcase

// at most one item matches:
unique0 case (op) ... endcase

// items are tried in order,
// first match wins:
priority case (op) ... endcase

// plain: first match wins, no
// check at all
case (op) ... endcase
```

These modifiers do two things at once, and the second is why they are worth using. To synthesis they say “full case” (`unique`, `priority`) and “parallel case” (`unique`), replacing the comment pragmas that Verilog users have written for thirty years. To simulation they add a run-time check: `unique` reports a violation if no item matches or if more than one matches, `unique0` if more than one matches, `priority` if none matches. That run-time check is genuinely valuable and it is something VHDL never offered.

Recipe: Use unique case plus default

The two are not redundant, but be clear about what each one buys. `default` tells the compiler that every path assigns the outputs, so no latch is inferred, and it gives you

VHDL's `when others` completeness guarantee. `unique` adds the overlap check: simulation complains if two items match the same value.

What you do *not* get is the no-match check. `unique` reports a violation when no item matches, and a `default` makes the case always match, so that half of the check is switched off by construction. If it is the unreachable-state case you want to catch, drop the `default` and assign the outputs before the case, or add an explicit `default`: `$error(...)`.

```

1 always_comb begin
2   unique case (state_q)
3     IDLE: state_d = req ? RUN : IDLE;
4     RUN:  state_d = done ? DONE : RUN;
5     DONE: state_d = IDLE;
6     default: state_d = IDLE;    // also covers x propagation
7   endcase
8 end

```

3.3.4 Loops

VHDL

```

for i in 0 to 7 loop
  y(i) <= a(7-i);
end loop;

for i in a'range loop
  ...
end loop;

while cnt < 10 loop
  cnt := cnt + 1;
end loop;

loop                -- infinite
  ...
  exit when done;
end loop;

next when skip;     -- continue
exit;               -- break
null;               -- do nothing

```

SystemVerilog

```

for (int i = 0; i < 8; i++)
  y[i] = a[7-i];

foreach (a[i])
  ...

while (cnt < 10)
  cnt = cnt + 1;

forever begin       // infinite
  ...
  if (done) break;
end

repeat (8) @(posedge clk);
do ... while (cond);

continue;
break;
;                  // do nothing

```

The mapping is direct. Four remarks.

`for` in SystemVerilog is the C form, with an initialisation, a condition and an increment. Declare the loop variable in the header (`for (int i = 0; ...)`) and it is automatic and local to the loop; declare it outside and it is a static module-level variable shared with everything else, which in a testbench with several concurrent loops is a race.

`foreach` has no VHDL counterpart and is the natural translation of `for i in a'range` loop. It works on packed and unpacked arrays, on any number of dimensions (`foreach (m[i][j])`), and it gets the bounds and the direction right without you writing them.

`repeat (N)` has no VHDL equivalent and is extremely useful in testbenches: `repeat (10) @(posedge clk);` is the idiomatic “wait ten clocks”.

`do ... while` exists in SystemVerilog and not in VHDL. `null` becomes an empty statement, a bare semicolon, which is easy to write by accident: an `if (c);` with a stray semicolon compiles and does nothing.

For synthesis, both languages require loop bounds to be static, and the loop is unrolled. The SystemVerilog rule is the same as the VHDL one.

3.4 Generate

VHDL

```
gen_stages : for i in 0 to N-1 generate
  u_st : entity work.stage
    generic map (IDX => i)
    port map (
      clk => clk,
      d   => pipe(i),
      q   => pipe(i+1));
end generate gen_stages;

gen_first : if i = 0 generate
  ...
end generate gen_first;

-- VHDL-2008 adds else/elsif:
gen_c : case MODE generate
  when 0 => ...
end generate gen_c;
```

SystemVerilog

```
for (genvar i = 0; i < N; i++)
  begin : gen_stages
    stage #(.IDX(i)) u_st (
      .clk (clk),
      .d   (pipe[i]),
      .q   (pipe[i+1]));
  end

if (MODE == 0) begin : gen_a
  ...
end else begin : gen_b
  ...
end

case (MODE)
  0: begin : gen_c0 ... end
  default: begin : gen_cd ... end
endcase
```

The three forms correspond one to one, including `case ... generate`, which VHDL-2008 added and SystemVerilog has always had. The keywords `generate` and `endgenerate` are optional in SystemVerilog-2009 and later; most modern code omits them, because the `for`, `if` or `case` at module level is unambiguously a generate construct.

The loop variable is a **genvar**: an elaboration-time integer, not a run-time object. Declare it in the loop header (`for (genvar i = 0; ...)`) so its scope is the loop. VHDL declares the equivalent implicitly in the `for ... generate` header, which is the same idea.

3.4.1 Names matter

△ Trap: An unnamed generate block still has a name

In VHDL the generate label is compulsory. In SystemVerilog it is optional, and if you leave it out the tool invents one: `genblk1`, `genblk2`, numbered in order of appearance in the file. Every hierarchical path, every waveform name, every synthesis constraint and every assertion binding then depends on that number, and inserting an unrelated `if` earlier in the file renumbers everything below it.

```
1 for (genvar i = 0; i < N; i++) begin           // becomes genblk1
2   stage u_st (...);
3 end
4 // path: top.genblk1[3].u_st -- and it changes if you edit the file above
5
6 for (genvar i = 0; i < N; i++) begin : g_stage
7   stage u_st (...);
8 end
9 // path: top.g_stage[3].u_st -- stable
```

Name every generate block, with a consistent prefix such as `g_`. It costs nothing and it is the difference between a constraint file that survives a refactor and one that does not.

3.4.2 A pipeline, both ways

The canonical use of `generate` is a regular structure whose stages differ only by index. Here is a systolic multiply-accumulate chain, in both languages.

```

1 architecture rtl of mac_chain is
2   type acc_arr_t is array (0 to N) of signed(31 downto 0);
3   type dat_arr_t is array (0 to N) of signed(15 downto 0);
4   signal acc : acc_arr_t;
5   signal dat : dat_arr_t;
6 begin
7   acc(0) <= (others => '0');
8   dat(0) <= din;
9
10  gen_taps : for i in 0 to N-1 generate
11    tap : process (clk) is
12      begin
13        if rising_edge(clk) then
14          dat(i+1) <= dat(i);
15          acc(i+1) <= acc(i) + resize(dat(i) * coeff(i), 32);
16        end if;
17      end process tap;
18    end generate gen_taps;
19
20    dout <= std_logic_vector(acc(N));
21 end architecture rtl;

```

Listing 3.1. A generated systolic chain in VHDL.

```

1 module mac_chain #(parameter int unsigned N = 8) (
2   input wire clk,
3   input wire signed [15:0] din,
4   input wire signed [15:0] coeff [0:N-1],
5   output logic signed [31:0] dout
6 );
7   logic signed [31:0] acc [0:N];
8   logic signed [15:0] dat [0:N];
9
10  assign acc[0] = '0;
11  assign dat[0] = din;
12
13  for (genvar i = 0; i < N; i++) begin : g_taps
14    always_ff @(posedge clk) begin
15      dat[i+1] <= dat[i];
16      acc[i+1] <= acc[i] + 32'($signed(dat[i]) * $signed(coeff[i]));
17    end
18  end
19
20  assign dout = acc[N];
21 endmodule

```

Listing 3.2. The same chain in SystemVerilog.

The two are the same design. Note three translation details. The VHDL array types become plain unpacked arrays with no type declaration. The `resize(...)` disappears because the thirty-two-bit target does the widening, but the `32'(...)` size cast is written anyway, so that the multiply is done at thirty-two bits regardless of context. And the `coeff` port is an unpacked array of signed vectors, which SystemVerilog allows across a port boundary and which older Verilog did not.

3.5 Instantiation

3.5.1 Component declaration or direct instantiation

VHDL

```
architecture rtl of top is
  -- the old way: declare a
  -- component, then instantiate it
  component fifo is
    generic (WIDTH : positive);
    port (clk : in std_logic;
          d  : in std_logic_vector;
          q  : out std_logic_vector);
  end component fifo;
begin
  u_fifo : fifo
    generic map (WIDTH => 16)
    port map (clk => clk,
              d  => din,
              q  => dout);
end architecture rtl;
```

SystemVerilog

```
module top (...);

  // there is no component
  // declaration in SystemVerilog.
  // The module itself is the
  // declaration.

  fifo #(
    .WIDTH (16)
  ) u_fifo (
    .clk (clk),
    .d   (din),
    .q   (dout)
  );

endmodule
```

The VHDL component declaration is a local re-declaration of an entity's interface, and it exists because VHDL originally insisted that everything be declared before use in the current design unit. Getting the component declaration out of step with the entity is a classic VHDL bug: the two are checked only at elaboration, and a mismatched port list produces an error a long way from its cause.

SystemVerilog has nothing of the kind. You instantiate a module by name, and the tool finds it in whatever was compiled. If it is not there, most tools issue a warning and create a black box, which is another reason to run Verilator, whose default is an error.

VHDL-2008's direct entity instantiation removes the component declaration on the VHDL side too, and it is what you should already be writing:

```
1 u_fifo : entity work.fifo(rtl)      -- (rtl) selects the architecture
2   generic map (WIDTH => 16)
3   port map (clk => clk, d => din, q => dout);
```

Listing 3.3. Direct entity instantiation: the modern VHDL form.

The architecture name in parentheses has no SystemVerilog equivalent, because a module has only one body.

3.5.2 Named, positional, and shorthand connections

VHDL

```
-- named (always do this)
port map (clk => clk,
          d  => din,
          q  => dout,
          err => open);

-- positional (do not)
port map (clk, din, dout, open);

-- VHDL has no shorthand for
-- "connect to a signal of the
-- same name".
```

SystemVerilog

```
// named (always do this)
.clk (clk),
.d  (din),
.q  (dout),
.err ()           // unconnected

// positional (do not)
fifo u (clk, din, dout, );

// implicit .name
.clk, .d, .q      // same-name

// implicit .*
fifo u_fifo (.*);
```

Named association is the same idea in both languages, and it is the only form you should use. Note the spelling of an unconnected port: VHDL writes `open`, SystemVerilog writes empty parentheses `.err()`. Omitting the port entirely is also legal in SystemVerilog and means the same thing, which is why Section 2.2 insisted you connect everything explicitly.

SystemVerilog adds two shorthands that VHDL does not have.

`.name` connects the port `name` to a signal called `name` in the instantiating scope. `.*` does that for every port not explicitly connected. Both are elaboration-time checks: if no signal of that name exists, or the widths disagree, it is an error, not a silent implicit net.

`.*` is genuinely controversial. In its favour: a hundred-port CPU core instantiated with `.*` is four lines instead of a hundred, name mismatches become errors, and adding a port to the module does not require editing every instantiation. Against it: you cannot see from the instantiation what is connected to what, code review becomes harder, and it encourages a naming convention where every signal in the parent has the same name as the port, which then breaks the moment you need two instances.

Use full named association in RTL you expect a human to read. Use `.*` only in testbench wrappers, where the connection is one-to-one by construction.

3.5.3 Configurations and `bind`

A VHDL configuration binds component instances to entity/architecture pairs, possibly deep in the hierarchy, without editing the design. SystemVerilog has no equivalent for that job. What it does have, and VHDL does not, is `bind`: a way of attaching a module instance to an existing module or instance from outside it.

```
1 // fifo_checker is an ordinary module full of assertions.
2 // This attaches one to every instance of `fifo', with access to its internals:
3 bind fifo fifo_checker u_chk (.clk(clk), .wr(wr_en), .rd(rd_en), .cnt(count_q));
4
5 // or to one specific instance:
6 bind top.u_dma.u_fifo fifo_checker u_chk (.*);
```

Listing 3.4. `bind`: adding checkers without touching the RTL.

`bind` is how assertion and coverage code is kept out of synthesisable files. It is one of the most useful features SystemVerilog has that VHDL lacks entirely. Chapter 4 treats it properly and

the *Verification Cookbook* puts it to work.

3.6 Subprograms

VHDL

```
function parity (v : std_logic_vector)
  return std_logic is
    variable r : std_logic := '0';
begin
  for i in v'range loop
    r := r xor v(i);
  end loop;
  return r;
end function parity;

procedure drive (
  signal clk : in std_logic;
  signal d   : out std_logic_vector;
  constant v : in std_logic_vector) is
begin
  wait until rising_edge(clk);
  d <= v;
end procedure drive;
```

SystemVerilog

```
function automatic logic parity (
  input logic [31:0] v);
  logic r = 1'b0;
  foreach (v[i]) r ^= v[i];
  return r;
endfunction

task automatic drive (
  ref logic clk,
  output logic [7:0] d,
  input logic [7:0] v);
  @(posedge clk);
  d = v;
endtask
```

3.6.1 Function or task?

A VHDL **function** returns a value and cannot wait. A VHDL **procedure** does not return a value and may wait. That is exactly the SystemVerilog distinction between **function** and **task**, with one relaxation and one addition.

The relaxation: a SystemVerilog **function** may have output and inout arguments as well as a return value, so it can pass several results back without a **task**. A function that returns nothing is declared `function void f(...)` and is called as a statement.

The addition: a SystemVerilog **task** may consume time (`#`, `@`, `wait`) exactly as a VHDL procedure may, but a **function** may not, and a synthesisable function may not either. The rule is enforced by the compiler.

3.6.2 Purity

VHDL distinguishes **pure** and **impure** functions: a pure function may read only its arguments, an impure one may read signals and variables in the enclosing scope. The default is pure, and the compiler enforces it.

SystemVerilog has no such distinction. Every function may read anything in scope, and nothing tells you whether it does. Where VHDL would have refused to compile a function that reads a signal you forgot to pass, SystemVerilog compiles it and the function silently depends on module state. In an `always_comb` this is usually harmless, because `always_comb` extends its sensitivity into functions, but it makes the function unusable anywhere else.

Write every SystemVerilog function as if it were VHDL-pure: take everything through arguments, return everything through the return value. Then it is reusable and testable.

Table 3.2. Subprograms.

VHDL	SystemVerilog	Note
<code>function</code>	<code>function</code>	Cannot consume time in either.
<code>procedure</code>	<code>task</code>	May consume time in both.
<code>procedure</code> (no waits)	<code>function void</code>	Callable as a statement.
<code>impure function</code>	(default)	Every SystemVerilog function may read module state.
<code>pure function</code>	—	No way to declare purity.
parameter mode <code>in</code>	<code>input</code>	Copied in.
parameter mode <code>out</code>	<code>output</code>	Copied out on return.
parameter mode <code>inout</code>	<code>inout</code>	Copied both ways.
<code>signal</code> parameter	<code>ref</code>	Passed by reference; the only way to see live changes.
default argument values	default argument values	Same.
overloading by signature	—	SystemVerilog has no subprogram overloading.
operator overloading	—	No equivalent at all.
recursion (always allowed)	<code>automatic</code> required	See the trap below.

3.6.3 Lifetime: the one-word difference

△ Trap: Write `automatic` on every subprogram

In VHDL, a subprogram's local variables are created on entry and destroyed on return, so two concurrent calls have two independent sets and recursion works. In SystemVerilog that behaviour is opt-in. A function or task declared in a module, interface or package without the word `automatic` has *static* lifetime: its locals are allocated once, for the whole simulation, and every call shares them.

Recursion is the immediate victim, and it is the one that shows up while you are still writing RTL:

```

1 // WRONG: static by default, so the recursive call overwrites `n'
2 function int factorial (int n);
3     if (n <= 1) return 1;
4     return n * factorial(n - 1);
5 endfunction
6
7 // RIGHT
8 function automatic int factorial (int n);
9     if (n <= 1) return 1;
10    return n * factorial(n - 1);
11 endfunction

```

The other victim is concurrency, which matters once you start writing testbenches: two processes calling the same static task share its arguments and locals, and the failure looks like random data corruption. That is the trap in detail, together with the enclosing-unit form (`module automatic tb;`) and the reason class methods are exempt, in the *Verification Cookbook*.

For a subprogram, the rule needs no detail at all. Write `automatic` on every function and task. There is no downside: `automatic` is what VHDL always did.

3.6.4 Argument passing and `ref`

A VHDL `signal` parameter lets a procedure see a signal live and drive it, which is how testbench procedures work. The SystemVerilog equivalent is `ref`: a reference argument, not a copy. `const ref` passes by reference without allowing modification, which is the efficient way to pass a large array into a function.

```

1 function automatic void stats (
2     const ref logic [31:0] data [],    // by reference, read only, no copy
3     output int unsigned  n_ones,      // copied out on return
4     inout  int unsigned  running      // copied in and out
5 );
6     n_ones = 0;
7     foreach (data[i]) n_ones += $countones(data[i]);
8     running += n_ones;
9 endfunction

```

Listing 3.5. Argument modes.

Note that output and inout arguments of a SystemVerilog subprogram are copied on entry and on return, not updated live. A `task` that waits and then writes an output argument does not update the caller's variable until it returns. For live updates you need `ref`, and `ref` arguments are not allowed in a function that is called from a continuous assignment.

3.6.5 Overloading

VHDL overloads subprograms by signature and lets you overload operators: this is how `numeric_std` gives "+" a meaning for `unsigned`. SystemVerilog has neither. Two functions in the same scope may not share a name, and there is no way to give + a new meaning.

The consequence for a VHDL designer is concrete: the library-based approach to fixed-point arithmetic (`ieee.fixed_pkg`, with `sfixed` and `ufixed` types and overloaded operators) has no SystemVerilog counterpart. You manage binary points by hand, with named functions and careful comments. This is one of the places where VHDL is simply the better language, and it is worth saying so.

3.7 Packages and visibility

VHDL

```
package demo_pkg is
  constant ITER : positive := 14;
  subtype data_t is
    signed(15 downto 0);
  -- deferred constant: value in
  -- the body
  constant TABLE_SIZE : positive;
  function atan_rom (i : natural)
    return data_t;
end package cordic_pkg;

package body cordic_pkg is
  constant TABLE_SIZE : positive := 16;
  function atan_rom (i : natural)
    return data_t is
  begin
    return to_signed(0, 16);
  end function atan_rom;
end package body demo_pkg;
```

SystemVerilog

```
package demo_pkg;
  localparam int unsigned ITER = 14;
  typedef logic signed [15:0] data_t;

  // no deferred constants: the
  // value must be here
  localparam int unsigned
    TABLE_SIZE = 16;

  function automatic data_t
    atan_rom (int unsigned i);
    return '0;
  endfunction
endpackage
```

3.7.1 One unit, not two

A VHDL package is two design units: the declaration, which is the interface, and the body, which holds the subprogram implementations and the values of deferred constants. Changing the body does not force recompilation of anything that uses the package, which is a real benefit on a large design.

A SystemVerilog package is one unit. There is no body and no separation between interface and implementation. Everything, including function bodies, is in the package, and everything that imports the package is recompiled when it changes.

Deferred constants therefore have no equivalent. A VHDL package can declare `constant TABLE_SIZE : positive;` in the declaration and give it a value in the body, hiding the value from users while still letting them use the name. SystemVerilog cannot do that at all.

3.7.2 Import and visibility

VHDL

```
use work.cordic_pkg.all;
use work.cordic_pkg.data_t;

-- fully qualified, no use clause
signal x : work.cordic_pkg.data_t;

-- a use clause in an entity
-- applies to the entity and to
-- every architecture of it
```

SystemVerilog

```
import cordic_pkg::*;
import cordic_pkg::data_t;

// fully qualified, no import
cordic_pkg::data_t x;

// an import before the module
// header is visible in the port
// list too:
module m
  import cordic_pkg::*;
  (input data_t d);
```

The scope resolution operator is `::`, not `.`, and a fully qualified name needs no import at all. That is the safest form for occasional use, and it documents where the name comes from.

The visibility rules differ in a way that matters. A wildcard import (`import pkg::*`) does not declare anything. It makes names *candidates*: when the compiler meets an identifier that is not declared locally and is not visible from an enclosing scope, it looks through the wildcard imports. If exactly one supplies the name, that name is used and becomes permanently imported into the scope. If two supply it, that reference is an error, but only that reference: the rest of the file compiles.

An explicit import (`import pkg::data_t`) does declare the name. Two explicit imports of the same name into one scope are an error at the declaration, whether or not you use it. A local declaration always wins over a wildcard import, and always conflicts with an explicit one.

△ **Trap: A local name silently shadows a wildcard import**

```
1 package pkg_a;  localparam int WIDTH = 32;  endpackage
2
3 module m;
4   import pkg_a::*;
5   localparam int WIDTH = 8;      // shadows pkg_a::WIDTH. No warning.
6   logic [WIDTH-1:0] bus;        // 8 bits, not 32.
7 endmodule
```

VHDL's use `... all` behaves the same way in that a locally declared homograph hides the used one, but VHDL makes the two candidates ambiguous where SystemVerilog silently prefers the local. In a large design with several imported packages, this is how two modules end up disagreeing about a bus width. Prefix package constants (`CFG_WIDTH`), or use the qualified form `pkg_a::WIDTH` wherever ambiguity is possible.

`export pkg_a::*`; inside a package re-exports names that the package itself imported, so that importing the outer package makes them visible too. VHDL has no equivalent: a `use` clause in a package declaration does not propagate to users of that package. Use `export` sparingly; it makes the origin of a name very hard to find.

3.7.3 Package or header file?

Recipe: When to use a package and when to use .svh

Use a **package** for everything the language can express: types, typedefs, constants (`localparam`), functions, tasks, enumerations and `covergroup` definitions. A package is compiled, type-checked, and namespaced.

Use an **included header** (`.svh`) only for text the language cannot express: macros, and the conditional-compilation guards around them. Guard it against double inclusion:

```
1 `ifndef CORDIC_DEFS_SVH
2 `define CORDIC_DEFS_SVH
3 `define CORDIC_ASSERT(c) assert(c) else $error("cordic: %s", `c`)
4 `endif
```

A package cannot hold macros, because macros are gone before the compiler sees the package; a header cannot hold a type safely, because including it twice declares the type twice. The two mechanisms do not overlap and neither replaces the other.

Packages themselves may be parameterised only indirectly: SystemVerilog has no generic packages. VHDL-2008 does, though tool support is uneven. The usual SystemVerilog workaround is a package of `parameter` type definitions plus a class or a module that carries the parameters,

and it is markedly less elegant than the VHDL form.

3.8 Attributes and system functions

VHDL attributes and SystemVerilog system functions overlap without matching. The mapping is in Table 3.3. Two general remarks first.

VHDL attributes are part of the language and are evaluated by the same rules as everything else; `s'event` is an expression you can use anywhere. SystemVerilog's equivalents are split between system functions usable anywhere (`$bits`, `$clog2`) and sampled-value functions (`$rose`, `$past`, `$stable`) that are meaningful only in a clocked assertion context or with an explicit clocking event. You cannot write `if ($rose(a))` in an ordinary `always_comb` and get VHDL's `a'event` behaviour.

Conversely, SystemVerilog has a set of bit-manipulation functions with no VHDL counterpart at all, and they are used constantly: `$clog2`, `$countones`, `$onehot`, `$isunknown`.

Table 3.3. Attributes and system functions.

VHDL	SystemVerilog	Note
<code>s'event</code>	<code>@(s)</code> or <code>\$changed(s)</code>	No expression form outside assertions.
<code>rising_edge(s)</code>	<code>\$rose(s)</code>	In assertions. In RTL, <code>posedge</code> .
<code>falling_edge(s)</code>	<code>\$fell(s)</code>	
<code>s'stable</code>	<code>\$stable(s)</code>	Sampled-value function; needs a clock.
<code>s'last_value</code>	<code>\$past(s)</code>	Not identical: <code>\$past</code> is the value one clock ago, <code>'last_value</code> the value before the last change.
<code>s'delayed(t)</code>	—	No equivalent.
<code>s'last_event</code>	—	No equivalent.
<code>t'image(x)</code>	<code>\$sformatf("%0d",x), %p</code>	For enumerations, <code>x.name()</code> .
<code>t'value(str)</code>	—	No equivalent; <code>\$sscanf</code> is the nearest.
<code>t'pos(x)</code>	<code>int'(x)</code>	
<code>t'val(i)</code>	<code>t'(i)</code>	Or <code>\$cast</code> for a checked conversion.
<code>t'succ(x)</code> , <code>t'pred(x)</code>	<code>x.next()</code> , <code>x.prev()</code>	Enumerations only.
<code>t'leftof</code> , <code>t'rightof</code>	—	
<code>a'range</code>	—	Use <code>foreach</code> or <code>\$size</code> .
<code>a'reverse_range</code>	—	No equivalent.
<code>a'length</code>	<code>\$size(a)</code>	<code>\$bits(a)</code> for the total bit count.
<code>a'high</code> , <code>a'low</code>	<code>\$high(a)</code> , <code>\$low(a)</code>	
<code>a'left</code> , <code>a'right</code>	<code>\$left(a)</code> , <code>\$right(a)</code>	
<code>a'ascending</code>	—	Test <code>\$left(a) < \$right(a)</code> .
—	<code>\$bits(t)</code>	Bit count of any type, including a struct.
—	<code>\$clog2(n)</code>	Ceiling of $\log_2 n$. Constant expression.
—	<code>\$countones(v)</code>	Population count.

continues

Table 3.3, continued

VHDL	SystemVerilog	Note
—	<code>\$onehot(v)</code> , <code>\$onehot0(v)</code>	Exactly one, or at most one, bit set.
—	<code>\$isunknown(v)</code>	True if any bit is x or z.
—	<code>\$typename(x)</code>	The type of <code>x</code> as a string. Debug only.
—	<code>\$dimensions</code> , <code>\$unpacked_dimensions</code>	Array introspection.
maximum, minimum (2008)	—	No system function; write your own.

Recipe: `$clog2` replaces the `log2` dance

The VHDL idiom for an address width is `integer(ceil(log2(real(DEPTH))))` with use `ieee.math_real.all`, and it has a well-known edge case at exact powers of two. SystemVerilog gives you `$clog2(DEPTH)` as a constant function usable directly in a `localparam`, and `$clog2(1)` is 0, which is what you want. Use it in every parameterised memory:

```
1 localparam int unsigned AW = (DEPTH > 1) ? $clog2(DEPTH) : 1;
2 logic [AW-1:0] rd_ptr, wr_ptr;
```

3.9 Reporting, assertions and files

3.9.1 Assertions and severity

VHDL

```
assert cnt < DEPTH
  report "fifo overflow"
  severity error;

assert false
  report "unreachable"
  severity failure;

report "starting test"
  severity note;

-- severity levels:
-- note, warning, error, failure
```

SystemVerilog

```
assert (cnt < DEPTH)
  else $error("fifo overflow");

assert (0)
  else $fatal(1, "unreachable");

$info("starting test");

// severity tasks:
// $info, $warning, $error,
// $fatal
```

The structural correspondence is exact and the mapping of severities is in Table 3.4. In both languages the assertion fires when the condition is *false*. The SystemVerilog form puts the message in an `else` clause, which reads oddly at first and is logical once you see it: the `else` branch is what happens when the assertion fails, and it may contain any statement, a message being only the usual one.

Two SystemVerilog refinements have no VHDL counterpart. A *deferred* assertion, written `assert #0 (...)`, is evaluated at the end of the time step rather than immediately, which suppresses spurious failures caused by transient values in combinational logic. A *final* assertion, `assert final (...)`, is checked at the end of simulation. Both are worth knowing. Everything above is an *immediate* assertion, a statement that runs when control reaches it; SystemVerilog also has *concurrent* assertions, which describe behaviour over time and are a sublanguage VHDL

Table 3.4. Severity.

VHDL	SystemVerilog	Behaviour
note	\$info	Message only.
warning	\$warning	Message only.
error	\$error	Message; simulation continues unless the tool is told otherwise.
failure	\$fatal	Message and immediate stop. First argument is the exit code passed to \$finish.

has nothing comparable to. the *Verification Cookbook* is where that sublanguage is taught, and where the assertions written with it are wired into a testbench.

3.9.2 Printing

VHDL	SystemVerilog
<pre> use std.textio.all; report "count = " & integer'image(cnt); -- VHDL-2008 to_string: report "vec = " & to_string(v); -- printing a real report "f = " & real'image(f); -- hex needs a helper report "h = " & to_hstring(v); </pre>	<pre> \$display("count = %0d", cnt); \$display("vec = %b", v); \$display("f = %f", f); \$display("h = %h", v); \$write("no newline"); msg = \$sformatf("%0d", cnt); \$display("%s at %0t", state.name(), \$time); </pre>

VHDL builds a string by concatenation and image functions; SystemVerilog uses C-like format specifiers. Table 3.5 lists the ones you need.

Table 3.5. Format specifiers.

SystemVerilog	VHDL equivalent	Meaning
%d	integer'image	Decimal, padded to the width of the type.
%0d	integer'image	Decimal, no padding. Use this one.
%h, %0h	to_hstring (2008)	Hexadecimal.
%b, %0b	to_string	Binary.
%o	to_ostring (2008)	Octal.
%s	(the string itself)	String, or an enumeration's .name().
%f, %e, %g	real'image	Real.
%t	time'image	Time, formatted by \$timeformat.
%p	—	“Pretty print”: any aggregate, including structs and arrays. No VHDL equivalent.
%m	—	The hierarchical name of the current scope. Very useful in messages.

%0d rather than %d is worth a habit: %d pads to the width of the type, so a 32-bit counter prints as ten characters with nine leading spaces. %p is the closest thing SystemVerilog has to a universal 'image, and it will print a whole packed structure or unpacked array in one go, which is invaluable in a scoreboard message.

3.9.3 Files

VHDL

```
use std.textio.all;

file f      : text;
variable l  : line;
variable v  : integer;

file_open(f, "in.txt", read_mode);
while not endfile(f) loop
  readline(f, l);
  read(l, v);
end loop;
file_close(f);

file_open(f, "out.txt", write_mode);
write(l, string'("hello"));
writeline(f, l);
file_close(f);
```

SystemVerilog

```
int fd;
int v;
string s;

fd = $fopen("in.txt", "r");
while (!$feof(fd)) begin
  void'($fscanf(fd, "%d\n", v));
end
fclose(fd);

fd = $fopen("out.txt", "w");
$fdisplay(fd, "hello");
fclose(fd);

// memory initialisation:
$readmemh("init.hex", mem);
$readmemb("init.bin", mem);
```

`std.textio`'s two-step model, read a line object and then parse fields out of it, has no SystemVerilog counterpart. SystemVerilog reads directly with `$fscanf` (formatted), `$fgets` (a whole line into a `string`), or `$fread` (binary). The file handle is an `int`, not a distinct file type, and passing a stale or zero handle is not caught.

`$readmemh` and `$readmemb` have no VHDL equivalent short of a `TEXTIO` loop you write yourself, and they are worth knowing well: one call loads an entire unpacked array from a text file of hexadecimal or binary values, they accept `@address` lines to jump around, and they are supported by every synthesis tool as a way to initialise a ROM.

```
1 module atan_rom #(parameter int unsigned N = 16) (
2   input wire  [3:0]  addr,
3   output logic [15:0] data
4 );
5 logic [15:0] rom [0:N-1];
6 initial $readmemh("atan_table.hex", rom);
7 assign data = rom[addr];
8 endmodule
```

Listing 3.6. A ROM initialised from a file.

3.10 Time and delays

VHDL

```
-- time is a physical type with
-- units built into the language
constant PERIOD : time := 10 ns;

wait for PERIOD;
wait for 100 ps;
wait for 1 us;

-- resolution limit is set by the
-- simulator command line, e.g.
-- vsim -t ps work.tb

-- now / current time:
report time'image(now);
```

SystemVerilog

```
`timescale 1ns/1ps
// or, inside the module:
timeunit      1ns;
timeprecision 1ps;

localparam time PERIOD = 10ns;

initial begin
    clk = 0; #PERIOD;
    clk = 1; #100ps;
    clk = 0; #1us;
    clk = 1; #10;
    // "#10" is 10 TIME UNITS, and
    // which unit that is depends
    // on the timescale in force.
    $display("%0t", $time);
end
```

VHDL's `time` is a physical type: a value carries its unit, the compiler checks it, and `10 ns` means ten nanoseconds no matter where it appears. The simulator's resolution limit affects rounding, not meaning.

SystemVerilog inherited Verilog's model, in which a delay is a bare number scaled by a directive. `#10` means "ten time units", and which unit that is depends on the ``timescale` in force for the module. SystemVerilog added time literals with units (`#10ns`) and the `timeunit/timeprecision` declarations, and between them they make the problem avoidable. It is still there for anyone who does not use them.

△ Trap: The timescale rule is genuinely awful

The timescale that applies to a module is the one established by the last ``timescale` directive that the compiler read *before* that module, in the order the files were given on the command line. Not the file's own directive, if it has none; not the project's; the last one seen. So:

- A module in a file with no ``timescale` inherits whatever the previous file set. Change the file order and its delays change.
- If no file sets one, the timescale is tool-defined, and QuestaSim, VCS and Xcelium do not agree.
- The simulation's time precision is the *smallest* precision of any module in the design, so one testbench file with `1ps` precision slows down the whole simulation.
- `$time` returns the time scaled and rounded to the calling module's time unit, so the same instant prints differently in two modules. `$realtime` avoids the rounding but not the scaling.

```
1 // file a.sv
2 `timescale 1ns/1ps
3 module a; initial #10 $display("a at %0t", $time); endmodule
4 // file b.sv -- no directive of its own
5 module b; initial #10 $display("b at %0t", $time); endmodule
```

Compile `a.sv b.sv` and `b` waits 10 ns. Compile `b.sv a.sv` and `b`'s timescale is tool-defined. Nothing warns you.

The fix is mechanical and you should apply it to every file you write:

```

1 `timescale 1ns/1ps           // first line of every .sv file
2 module m;
3     timeunit      1ns;       // belt and braces: module-local, order-independent
4     timeprecision 1ps;
5     ...
6 endmodule

```

and write delays with explicit units, #10ns, never bare #10.

Delays do not belong in RTL at all

Both languages let you write a delay in synthesisable code and both ignore it at synthesis. The result is a simulation that does not match the netlist. The only delays in a well-written project are in the clock generator and the stimulus of the testbench:

```

1 localparam time TCLK = 10ns;
2 initial begin
3     clk = 1'b0;
4     forever #(TCLK/2) clk = ~clk;
5 end

```

Everything else is driven from clock edges.

3.11 The complete mapping table

Table 3.6 is the summary. It repeats, in one place, everything that Chapter 2 and this chapter have established, static half and behavioural half together. Photocopy it.

Table 3.6. VHDL to SystemVerilog: the complete mapping.

VHDL	SystemVerilog	Notes
<i>Design units</i>		
entity + architecture	module ... endmodule	One unit, not two.
several architectures	parameter, `ifdef, or several modules	No “same interface, other body”.
configuration	config (rare)	Effectively no equivalent.
package + package body	package ... endpackage	One unit. No deferred constants.
library ieee;	—	No library clause.
use pkg.all	import pkg::*	Different resolution rules.
use pkg.name	import pkg::name	
pkg.name	pkg::name	Qualified reference, no import needed.
—	`include "x.svh"	Textual inclusion. Macros only.
—	`define, `ifdef	Preprocessor. No VHDL equivalent.
—	`default_nettype none	Put it in every file.
analysis order	compilation unit, \$unit	Tool-defined. Avoid \$unit.
<i>Ports and parameters</i>		

continues on the next page

Table 3.6, continued

VHDL	SystemVerilog	Notes
in / out / inout buffer	input / output / inout output	SystemVerilog outputs are always readable.
port default := '0'	—	No default port values.
generic	parameter	
architecture constant	localparam	
generic (type T) (2008)	parameter type T	Well supported in SystemVerilog.
generic map (X => 1)	#(.X(1))	Before the instance name.
—	defparam	Deprecated. Never use.
<i>Types</i>		
std_ulogic	logic	4-state variable.
std_logic	wire	4-state resolved net.
std_logic_vector(N-1 downto 0)	logic [N-1:0]	
unsigned	logic [N-1:0]	Unsigned is the default.
signed	logic signed [N-1:0]	
bit	bit	2-state in both.
boolean	—	Any non-zero value is true.
integer	int	Not SystemVerilog's <code>integer</code> , which is 4-state.
natural, positive	—	No range constraints, no run-time checks.
real	real	
character	byte	
string	string	Dynamic in SystemVerilog.
time	time + `timescale	See the timescale trap.
subtype ... range	—	No equivalent. Assert instead.
type ... is array	unpacked [...] after the name	For memories.
—	packed [...] before the name	One vector with a shape.
record	struct packed / struct	Packed gives a defined bit layout.
—	union, union packed	No VHDL equivalent.
type ... is (A, B)	typedef enum logic [...] {A,B}	Always give the base type.
<i>Literals and conversions</i>		
'0', '1'	1'b0, 1'b1	
"1010"	4'b1010	
x"A5"	8'hA5	
16#FF#	255 or 8'hFF	
(others => '0')	'0	Not 1'b0.
(others => '1')	'1	
aggregate (a => 1, b => 2)	'{a:1, b:2}	Note the apostrophe.
& (concatenation)	{ , }	& is bitwise AND in SystemVerilog.

continues on the next page

Table 3.6, continued

VHDL	SystemVerilog	Notes
—	{N{x}}	Replication. No VHDL form.
unsigned(v), std_logic_vector(u)	—	No conversion needed.
to_integer(u)	implicit, or int'(u)	
to_unsigned(i,N)	implicit truncation	
to_signed(i,N)	\$signed(...)	
resize(v,N)	implicit, or N'(v)	Silent. This is the danger.
integer(r) on a real	implicit, rounds	
—	\$cast(d,s)	Checked conversion at run time.
—	{>{x}}	Bit-stream cast.
<i>Statements</i>		
y <= expr; (concurrent)	assign y = expr;	
y <= a when c else b;	assign y = c ? a : b;	
with s select y <= ...	always_comb + case	No concurrent form.
process (a, b)	always @(a, b)	
process (all)	always_comb	Also runs at time zero.
process (clk) + rising_edge	always_ff @(posedge clk)	
process (clk, rst_n)	always_ff @(posedge clk or negedge rst_n)	Asynchronous reset.
signal <= in a process	non-blocking <=	Use in always_ff.
variable := in a process	blocking =	Use in always_comb.
—	initial	Process with no sensitivity list.
—	final	No VHDL equivalent.
if / elsif / else / end if	if / else if / else	begin/end required for blocks.
case + when others	case + default	SystemVerilog does not require completeness.
case? (2008)	casez	Never casex.
—	unique, unique0, priority	Full/parallel case, plus a run-time check.
for i in r loop	for (int i=..) / foreach	
while, loop	while, forever, repeat, do..while	
next, exit	continue, break	
null	;	
wait for 10 ns	#10ns	
wait until c = '1'	wait (c)	Different: no edge required.
wait on a, b	@(a or b)	
after, transport	#	No inertial/transport distinction.
<i>Structure</i>		
for ... generate	for (genvar i=..)	Name every block.
if ... generate	begin : g_x	
case ... generate (2008)	if (..) begin : g_x	
block	case (..) ... endcase	
	named begin/end or generate block	

continues on the next page

Table 3.6, continued

VHDL	SystemVerilog	Notes
component + port map	direct instantiation	No component declaration.
entity work.f(rtl)	f u_f (...)	No architecture selection.
=> open	.p()	Leave nothing implicit.
—	.name, .*	Implicit connection. Use .* sparingly.
—	bind	Attach checkers from outside. No VHDL form.
<i>Subprograms and reporting</i>		
function	function automatic	Always write automatic.
procedure	task automatic	Or function void.
signal parameter	ref	
impure	(default)	No purity control in SystemVerilog.
overloading, operator overloading	—	No equivalent.
assert ... report ...	assert (c) else	Fires when false in both.
severity error	\$error(..)	
note, warning, error, failure	\$info, \$warning, \$error, \$fatal	
report x & integer'image(i)	\$display("x %0d", i)	
std.textio	\$fopen, \$fscanf, \$fdisplay	No line object.
—	\$readmemh, \$readmemb	Memory initialisation.

3.12 Checklist

Operational takeaways for the behavioural half. When you see the left, write the right, and never do the things marked “never”. The static half is at the end of Chapter 2 and is not repeated here, with one exception: keep running Verilator with `-Wall` over every file and treating `WIDTHTRUNC`, `WIDTHEXPAND` and `PINMISSING` as errors, because none of what follows protects you from a width.

- **Every module repeats its timescale** as `timeunit/timeprecision`, because ``timescale` is inherited across files in command-line order and nothing warns you when it is wrong.
- **Processes:** `process (all)` becomes `always_comb`; `process (clk)` with `rising_edge` becomes `always_ff @(posedge clk)`. Use `blocking =` in `always_comb` and `non-blocking <=` in `always_ff`, never the other way round, never both in one block.
- **One writer per variable.** `always_comb` and `always_ff` enforce this; plain `always` and `initial` do not, and module-level variables are shared by every process.
- **case:** always write a `default` in combinational logic, because SystemVerilog does not require completeness and the missing branch is a latch. Prefer `unique case` to get a run-time check as well. Never write `casex`.
- **Name every generate block** with a `g_` prefix, or the tool numbers them and your hierarchical paths change when you edit the file.

- **Instantiate with full named association** in RTL. There is no component declaration to keep in step, but there is also no width check: `.*` belongs in testbench wrappers only.
- **wait until c = '1' is *not* wait (c).** VHDL waits for an event that makes the condition true; SystemVerilog returns at once if it already holds. Where you meant the VHDL behaviour, write `@(posedge c)`.
- **Assertions:** `assert ... report ... severity` becomes `assert (c) else $error(...)`, so the message moves into the `else`. Both fire when the condition is false.
- **Every function and task is *automatic*.** A static subprogram breaks recursion and races when two processes call it.
- **Put types, constants and functions in a package**, macros in an `.svh` header, and nothing at all in `$unit`.
- **Reach for `$clog2`, `$countones`, `$onehot`, `$isunknown` and `$bits`** where VHDL made you write a function. Do not expect `'range`, `'reverse_range`, `'delayed` or `'value`: they have no equivalent.
- **Delays belong in testbenches only**, written with explicit units (`#10ns`), and everything else is driven from a clock edge.

Together with Chapter 2, this chapter is enough to read and write SystemVerilog RTL: every construct above has a VHDL original you already know. What comes next has no original. Chapter 4 takes the design-side constructs SystemVerilog has and VHDL does not, starting with interfaces, packed structures and parameterised types, and the *Verification Cookbook* takes the verification-side ones, from classes and constrained randomisation to the assertion sublanguage. Those two chapters are what makes the move worth making.

Beyond VHDL: The Design-Side Additions

Chapter 2 and Chapter 3 were a translation exercise: every VHDL construct had a SystemVerilog counterpart, and the job was to find it. This chapter covers the constructs with no VHDL source to translate from: interfaces, clocking blocks, the checked `always` variants, packed structures and unions, typed enumerations, parameterised types, the preprocessor, `generate` and `bind`. Every one of them elaborates into hardware or into the hierarchy around it. At the end you will be able to read a modern SystemVerilog design, and to write one that is shorter and safer than the VHDL you would have written instead.

A large part of SystemVerilog is Verilog with the sharp edges filed off, and the two preceding chapters covered that part. What remains is genuinely new, and it splits cleanly in two.

The first half is design-side, and it is this chapter. The order is deliberate. Interfaces come first, because they change the shape of a hierarchy and everything after them can be declared inside one. Then come the constructs that change the shape of a single module: intent-carrying `always` forms, packed structures, typed enumerations, type parameters, and the preprocessor you should mostly avoid. `generate` and `bind` close the chapter, because `bind` is the one construct here that changes what you can attach to a module you did not write, and it is the bridge to everything that follows.

The second half is verification-side: dynamic data types, classes, constrained randomisation, functional coverage, assertions and dynamic threads. VHDL has nothing comparable, and it is the reason the industry verifies in SystemVerilog even where it designs in VHDL. That half is the *Verification Cookbook*, immediately after this one, because none of it makes sense until you have seen what it is for.

One question belongs to neither half: what does VHDL still do better? That is a question about the two languages rather than about SystemVerilog alone, and Chapter 1 answers it, next to the account of where the two languages came from.

4.1 Interfaces

This is the single biggest structural addition to the language, and the one that changes how you write hierarchy.

Consider a small memory-mapped bus: request, write enable, address, write data, byte enables, grant, read valid, read data, error. Nine signals, plus a clock and a reset. In VHDL you name every one of them at least three times: once in the entity of the master, once in the entity of the slave, and once in each port map. Add a byte-enable signal six months later and you edit four files. Get the direction wrong in one of them and the compiler tells you nothing until elaboration, if at all.

SystemVerilog lets you declare the bundle once, as a first-class design object:

```
1 interface simple_bus #(parameter int AW = 12,
```

```

2         parameter int DW = 32)
3         (input logic clk, input logic rst_n);
4
5     logic      req;      // master -> slave
6     logic      we;
7     logic [AW-1:0] addr;
8     logic [DW-1:0] wdata;
9     logic [DW/8-1:0] be;
10    logic      gnt;      // slave -> master
11    logic      rvalid;
12    logic [DW-1:0] rdata;
13    logic      err;
14
15    // One write transaction, written once, usable by every master.
16    task automatic write(input logic [AW-1:0] a,
17                        input logic [DW-1:0] d);
18        @(posedge clk);
19        req <= 1'b1; we <= 1'b1; addr <= a; wdata <= d; be <= '1;
20        do @(posedge clk); while (gnt != 1'b1);
21        req <= 1'b0; we <= 1'b0;
22    endtask
23
24    modport master (input  clk, rst_n, gnt, rvalid, rdata, err,
25                  output req, we, addr, wdata, be,
26                  import write);
27
28    modport slave  (input  clk, rst_n, req, we, addr, wdata, be,
29                  output gnt, rvalid, rdata, err);
30
31    modport monitor (input  clk, rst_n, req, we, addr, wdata, be,
32                   gnt, rvalid, rdata, err);
33 endinterface

```

Listing 4.1. A small memory-mapped bus as an interface

Three things are happening in Listing 4.1, and they are worth separating.

The bundle. The nine `logic` declarations are the signals themselves. An interface instance is a real object in the hierarchy, like a module instance: it has a name, it can be parameterised, and it can be found in the wave viewer at `top.bus.addr`.

The direction views. A `modport` is a named view of the bundle from one participant's point of view. It carries no signals of its own; it says which of the interface's signals that participant may drive and which it may only read. The same wires are `output` in the `master` view and `input` in the `slave` view. Declaring the directions once, in one place, is exactly the property VHDL record ports lack.

The behaviour. The `write` task lives inside the interface and drives the interface's own signals. Any master that imports it through its `modport` can call `bus.write(12'h040, 32'hDEAD_BEEF)` and get a protocol-correct transaction. The protocol is written once and cannot drift between testbench and design.

Instantiation collapses accordingly:

```

1 module ram (simple_bus.slave b);
2     // inside, the signals are b.req, b.addr, b.wdata, ...
3 endmodule
4
5 module top;
6     logic clk, rst_n;
7
8     simple_bus #(.AW(12), .DW(32)) bus (.clk(clk), .rst_n(rst_n));

```

```

9
10  cpu    u_cpu (.b(bus.master));
11  ram    u_ram (.b(bus.slave));
12  bus_mon u_mon (.b(bus.monitor));
13  endmodule

```

Listing 4.2. Instantiating the interface: one connection instead of fifteen



Figure 4.1. The same connection with discrete ports and with an interface. The modport names on the right are the only place a direction is written.

Figure 4.1 makes the structural point. What it does not show is the maintenance point: adding a signal to the bus means editing one file, and every module that does not use the new signal needs no change at all.

4.1.1 Parameterised interfaces

An interface takes parameters exactly as a module does, and the parameters propagate into every module that receives it. Inside `ram`, the width of `b.addr` is whatever `AW` was at the instance. You can read the parameter back through the port:

```

1  module ram (simple_bus.slave b);
2      localparam int DEPTH = 1 << $bits(b.addr);
3      logic [$bits(b.wdata)-1:0] mem [DEPTH];
4  endmodule

```

This is the closest SystemVerilog gets to a VHDL generic package: the interface carries both the types and the objects, and the module adapts to them without declaring a single generic of its own.

4.1.2 Tasks in interfaces, and virtual interfaces

A modport can `import` a task or a function, and it can also `export` one, meaning that the interface declares a prototype and the connected module supplies the body. Importing is the common case and the one to learn first.

Interfaces become far more powerful once classes enter the picture (the *Verification Cookbook*). A class-based testbench holds a *virtual interface*: a handle to an interface instance, stored in a variable, assigned at run time. That is what lets a testbench object which lives in no hierarchy at all reach down and wiggle a wire. the *Verification Cookbook* builds a complete testbench on the mechanism. The declaration looks like this, and it is the only bridge between the class world and the module world:

```

1  class bus_driver;
2      virtual simple_bus.master vif;    // a handle, not an instance
3      function new(virtual simple_bus.master v); vif = v; endfunction
4      task run(); vif.write(12'h10, 32'h1); endtask
5  endclass

```

Interfaces are not entities

An interface is not a lightweight module. It may contain `initial` and `always` blocks, assertions, covergroups, tasks, functions, parameters, types, and even other interface instances. Some verification IP is delivered almost entirely as an interface. Keep design interfaces boring: signals, modports, and at most a clocking block.

4.1.3 What synthesises, and what does not

Interfaces are synthesisable, and every major synthesis tool in current use accepts them. They are still avoided in some flows: certain sign-off, formal-equivalence and legacy scripting flows expect flat port lists, and some IP-delivery rules forbid them outright because the customer's tool chain may be older than yours. The pragmatic answer is to keep to a safe subset:

- interface instances that are static and elaborated once, never created in a `generate` loop whose bounds are not constant;
- `modport` on every port, so that directions are explicit;
- no tasks with timing controls (@, #) in anything that is synthesised, and no clocking blocks in the design half;
- no dynamic types, no classes, no covergroups in an interface that reaches synthesis; put those in a separate verification interface bound to the design (Section 4.8);
- parameters, packed structs and enums are fine.

Use interfaces freely at the testbench boundary and between blocks you own. Think twice before putting one on the top-level pins of a chip or on a deliverable IP block.

4.1.4 Compared with VHDL records and mode views

VHDL-2008 lets you put a record in a port, which solves half the problem:

VHDL

```
type bus_req_t is record
  req  : std_logic;
  addr : std_logic_vector(11 downto 0);
  wdata : std_logic_vector(31 downto 0);
end record;

entity ram is
  port (clk  : in  std_logic;
        req_i : in  bus_req_t;
        rsp_o : out bus_rsp_t);
end entity;
```

SystemVerilog

```
module ram (simple_bus.slave b);
  // b.req, b.addr, b.wdata,
  // b.gnt, b.rdata ...
endmodule
```

The VHDL record port bundles signals, but every element of a record port has the mode of the port itself. A bidirectional bus therefore needs two record types, one per direction, and you are back to naming things twice. The record also cannot carry the write procedure.

VHDL-2019 addresses this directly with *interfaces* and *mode views*, which are close in spirit to a modport: a view declaration gives each record element its own direction, and a port declared with that view behaves like a modport. It is the right feature. The problem is support: at the time of writing, mode views are implemented in only a handful of tools, and no mainstream synthesis flow accepts them. Treat VHDL-2019 as a language you can read but not yet ship.

4.2 Clocking blocks

Every VHDL testbench you have written contains a line like `wait for 2 ns;` placed just after a clock edge, so that the stimulus does not change at the same instant the design samples it. That convention is the entire race-avoidance strategy of a classical VHDL testbench, and it is carried in the author's head, not in the language.

SystemVerilog makes it a declaration. A `clocking` block names a clock, lists the signals that are read and written with respect to that clock, and attaches a *skew* to each direction. It is declared inside an interface, which is why it appears in a design chapter at all.

```

1 interface simple_bus (input logic clk, input logic rst_n);
2     logic req, gnt;
3     logic [11:0] addr;
4
5     clocking cb @(posedge clk);
6         default input #1step output #1ns;
7         output req, addr;
8         input gnt;
9     endclocking
10
11     modport tb (clocking cb, input clk, rst_n);
12 endinterface

```

Listing 4.3. A clocking block fixes sampling and driving skew

One line declares both skews. `input #1step` samples every signal listed as input one *time step* before the clock edge, so a reader sees the value the design saw and never the value the design has just produced. `output #1ns` delays every assignment to a clocking-block output until 1 ns after the edge, in the middle of the cycle, where no flip-flop can be confused by it. Using the block is unremarkable: `@(bus.cb)` waits for the clocking event, `bus.cb.addr <= 12'h040` drives, and `bus.cb.gnt` reads.

Note the direction convention, because it trips up everyone once. A clocking block is written from the point of view of the *testbench*, so what the testbench drives is `output`, even though those same wires are inputs to the design.

That is as much as a design reader needs. A clocking block is not synthesisable, and the rest of the subject belongs to the testbench: the *Verification Cookbook* explains what `#1step` means in terms of the scheduler regions, why the drive skew is what stops the stimulus racing the design, and what happens when a clocking-block output and a plain procedural assignment target the same signal. The point to take away here is structural. The timing relationship between testbench and design is now written down once, inside the interface that carries the protocol, instead of being repeated as a hand-tuned delay in every stimulus block.

4.3 Declaring intent: `always_comb`, `always_ff`, `always_latch`

VHDL has one process. SystemVerilog has one general `always` plus three specialised forms whose only purpose is to let you state what you meant so that the tool can check it. This is a language feature with no VHDL counterpart: in VHDL an accidental latch is caught by a synthesis warning, if you read your logs.

```

1 always_comb begin                // combinational; tool checks it
2     y = a & b;
3 end
4
5 always_ff @(posedge clk or negedge rst_n) begin

```

```

6   if (!rst_n) q <= '0;
7   else      q <= d;
8   end
9
10  always_latch begin
11    if (en) q <= d;           // latch, and you said so on purpose
12  end

```

What is actually checked differs between the three, and the differences matter.

`always_comb` has an inferred sensitivity list that includes everything read in the block *and everything read inside any function it calls*. That last part is the fix for the classic `always @(a or b)` bug where a signal was left out of the list. It also runs once at time zero regardless of any event, so the outputs are consistent from the start; a VHDL process with a sensitivity list does the same, so this will feel natural. Two further restrictions: no timing controls are allowed inside it, and the variables it writes may not be written by any other process. That second rule is stronger than anything in VHDL, and it turns a whole class of multiple-driver bugs into a compile error.

`always_ff` requires exactly one event control and the same single-writer rule. What the LRM does *not* require is that you use non-blocking assignments inside it; that is a lint rule, not a language rule. Tools warn, and you should treat the warning as an error.

`always_latch` checks nothing you could not check yourself. Its value is documentary: it tells the next reader that the latch is intentional, and it lets a lint rule flag every latch that is *not* inside one.

In new SystemVerilog RTL, plain `always` appears only in testbenches. Every synthesisable process is `always_comb`, `always_ff` or, rarely, `always_latch`.

4.3.1 unique, unique0 and priority

The same idea applies to decisions. VHDL case statements must be exhaustive, which the compiler enforces; SystemVerilog case statements need not be, so the language offers modifiers that state your assumption instead.

```

1  unique case (sel)           // exactly one branch matches, no overlap
2    2'b00: y = a;
3    2'b01: y = b;
4    2'b10: y = c;
5    2'b11: y = d;
6  endcase
7
8  priority if (irq_hi) grant = HI; // at least one matches;
9  else if (irq_mid) grant = MID;   // first match wins
10 else if (irq_lo) grant = LO;

```

`unique` asserts two things: at least one branch matches, and no two branches match simultaneously. `unique0` drops the first requirement, so "none of them" is legal but overlap is not. `priority` asserts only that at least one branch matches, and keeps the written order.

At simulation time these become runtime checks: the simulator issues a violation report when the assumption fails, with the values that caused it. At synthesis time they become directives, the modern replacement for the `// synopsys parallel_case` and `full_case` pragmas. That is where the danger lies.

△ **Trap: unique is a promise, not a check, in synthesis**

Synthesis believes you. If you write `unique case` over a one-hot-encoded signal and the signal is momentarily not one-hot in silicon, the synthesised logic may produce something that the RTL simulation never produced, because synthesis used your promise to optimise away the multiplexer priority. The simulator flags the violation; the gates do not. Only write `unique` when a separate mechanism makes the claim true: an assertion, a formal proof, or a structurally one-hot encoding.

While on the subject of case: SystemVerilog inherits `casex` and `casez` from Verilog, and adds `case ... inside`, which uses wildcard matching in one direction only.

△ **Trap: casex treats an X in the expression as a wildcard**

`casez` treats Z (and ?) in the case *items* as don't-care, which is what you want. `casex` additionally treats X and Z in the *selecting expression* as don't-care. So an uninitialised or corrupt signal carrying X matches the first branch and simulation carries on as though nothing were wrong, hiding exactly the bug that X was there to reveal. VHDL's `std_logic` case statement would have forced you to write a `when others` and think about it. Never write `casex`. Prefer `case (x) inside` with `4'b1???` style items, which is both readable and X-safe.

4.4 Packed structures and unions

This is the construct VHDL designers miss most once they have used it, and it is the one to adopt first.

A **packed** struct is a named bit vector. The fields occupy contiguous bits, the first-declared field is the most significant, and the whole object behaves as a single vector wherever a vector is expected. It can cross a port, sit in an array, be assigned from a literal, be cast to and from a plain vector, and be indexed by name.

```

1  typedef struct packed {
2      logic [6:0] funct7;
3      logic [4:0] rs2;
4      logic [4:0] rs1;
5      logic [2:0] funct3;
6      logic [4:0] rd;
7      logic [6:0] opcode;
8  } instr_r_t;           // exactly 32 bits, MSB first
9
10 module decoder (
11     input logic [31:0] iword,
12     output logic [4:0] rd_o,
13     output logic      is_add_o
14 );
15     instr_r_t ins;
16     assign ins      = instr_r_t'(iword); // a cast, not a conversion
17     assign rd_o      = ins.rd;
18     assign is_add_o  = (ins.opcode == 7'b0110011) &&
19                       (ins.funct3 == 3'b000) && (ins.funct7 == '0);
20 endmodule

```

Listing 4.4. An instruction word decoded by field name

Compare this with the same decoder written against bit indices. The SystemVerilog version

cannot silently shift by one when someone widens `funct7`, and `$bits(instr_r_t)` tells you the width without counting. The cast `instr_r_t'(iword)` is free: it reinterprets bits, it does not convert values, and it is a compile error if the widths differ.

Packed structs nest, cross ports, and form arrays:

```

1 typedef struct packed {
2     logic        valid;
3     logic [3:0]  id;
4     instr_r_t    ins;
5 } pipe_stage_t;           // 1 + 4 + 32 = 37 bits
6
7 pipe_stage_t stage [0:3]; // unpacked array of packed structs
8 pipe_stage_t [3:0] pipe;  // packed array: one 148-bit object
9
10 always_ff @(posedge clk) begin
11     stage[0] <= '{valid: 1'b1, id: next_id, ins: instr_r_t'(iword)};
12     for (int i = 1; i < 4; i++) stage[i] <= stage[i-1];
13 end

```

The `'{...}` form is an assignment pattern, and naming the fields makes it order-independent. `'{default: '0}` zeroes an arbitrarily complicated structure in one line, which is worth knowing for reset branches.

VHDL

```

type instr_r_t is record
    funct7 : std_logic_vector(6 downto 0);
    rs2    : std_logic_vector(4 downto 0);
    rs1    : std_logic_vector(4 downto 0);
    funct3 : std_logic_vector(2 downto 0);
    rd     : std_logic_vector(4 downto 0);
    opcode : std_logic_vector(6 downto 0);
end record;

-- and a hand-written function
-- to_record(slv) : 20 lines of
-- slicing, maintained by hand.

```

SystemVerilog

```

typedef struct packed {
    logic [6:0] funct7;
    logic [4:0] rs2;
    logic [4:0] rs1;
    logic [2:0] funct3;
    logic [4:0] rd;
    logic [6:0] opcode;
} instr_r_t;

// and the conversion is
instr_r_t'(iword)

```

The VHDL record is a fine data structure, but it has no defined bit layout, so moving between a record and a `std_logic_vector` means writing and maintaining a pair of conversion functions. That is the whole difference: **packed** gives the record a layout.

△ Trap: Unpacked structs are not vectors

Leave out the word **packed** and you get an unpacked struct. It is still useful in testbenches, where fields can be queues and strings, but it has no bit layout, cannot be cast to a vector, cannot cross a synthesisable port as a vector, and cannot be assigned from a literal vector. Forgetting **packed** on an RTL type produces a wall of type errors that never mention the missing word.

4.4.1 Packed unions

A packed union gives two names to the same bits:

```

1 typedef union packed {
2     instr_r_t    r;           // decoded as an R-type
3     instr_i_t    i;           // decoded as an I-type
4     logic [31:0] raw;         // or as a plain word
5 } instr_t;

```

```

6
7 instr_t w;
8 assign w.raw = iword;
9 assign rs1   = w.r.rs1;           // same bits, different view

```

Every member of a packed union must have the same width; the compiler enforces it. This is the clean way to express a register that has a field-oriented view and a byte-oriented view, or an instruction word with several formats.

SystemVerilog also has *tagged* unions, which add a hidden tag field and let the compiler check that you read the member you last wrote, with `case ... matches` for discrimination. They are elegant and almost unsupported: several major simulators accept them and most synthesis tools do not. Know they exist; do not build a design on them.

4.5 Enumerations that carry a base type

A VHDL enumeration is an abstract type whose encoding is chosen by synthesis unless you attach an attribute. A SystemVerilog enumeration is a set of named constants of a base type you choose, and the encoding is part of the declaration:

```

1 typedef enum logic [2:0] {
2     IDLE = 3'b001, RUN = 3'b010, DONE = 3'b100
3 } state_e;           // explicitly one-hot
4
5 typedef enum logic [1:0] { A, B, C } small_e; // 0, 1, 2 by default

```

The base type also fixes what happens on reset: logic-based enumerations start at X, which is what you want, because an uninitialised state machine should be visibly broken. Using `bit` as the base type would start it at zero and hide the missing reset.

Enumerations have built-in methods, which have no VHDL counterpart at all beyond `'image` and `'succ`:

```

1 state_e s = RUN;
2 $display("%s", s.name()); // "RUN" -- the reason to use them in $display
3 s = s.next();             // DONE (wraps round after the last)
4 s = s.prev();             // RUN
5 $display("%0d", s.num()); // 3 -- number of members
6 state_e f = s.first();    // IDLE
7 state_e l = s.last();     // DONE

```

`.name()` alone justifies the construct: every waveform dump, every log message and every assertion failure can print the state by name with no lookup table. `.next()` and `.prev()` wrap around, so a loop written as "advance until you reach `last()`" runs forever if you write the exit test wrongly; iterate with `for (int i = 0; i < s.num(); i++)` instead.

Enumerations are strongly typed, up to a point

You cannot assign an integer to an enumeration variable without an explicit cast: `s = state_e'(2)` compiles, `s = 2` does not. But the cast is not checked: `state_e'(3'b111)` produces a variable holding a value with no name, and `.name()` returns an empty string for it. This is weaker than VHDL, where the type simply has no such value.

4.6 Parameterised types and elaboration-time computation

VHDL generics carry values. SystemVerilog parameters carry values *and types*:

```

1 module fifo #(
2     parameter type T      = logic [7:0],
3     parameter int  DEPTH = 8
4 ) (
5     input  logic clk, rst_n,
6     input  logic push, pop,
7     input  T    din,
8     output T    dout,
9     output logic full, empty
10 );
11 T mem [DEPTH];
12 localparam int PTRW = $clog2(DEPTH);
13 // ...
14 endmodule
15
16 // instantiated with a struct as the payload:
17 fifo #(.T(pipe_stage_t), .DEPTH(16)) u_f ( ... );

```

Listing 4.5. A FIFO parameterised by the type it carries

The FIFO now works for any packed type without knowing its width, and `$bits(T)` recovers the width where it is needed. In VHDL the same effect requires a generic package (2008) or a generic subprogram (2019), both of which are more verbose and, more importantly, only partially supported: generic packages work in the major commercial simulators, *QuestaSim* among them, but synthesis support has historically been patchy, and generic subprograms are 2019 and effectively unavailable.

Related tools:

- `type(expr)` yields the type of an expression, so `type(din) tmp;` declares a variable that follows the port. It is useful in macros and in generic code.
- `$bits(x)` gives the width in bits of a type or an expression, including a whole struct.
- Functions are evaluated at elaboration when their arguments are constant, so you can compute parameters:

```

1 function automatic int unsigned atan_rom_entry(int unsigned i);
2     // evaluated at elaboration time; no hardware is built
3     return int'($atan(2.0**(-real'(i))) * (2.0**13));
4 endfunction
5
6 localparam int unsigned ATAN [0:13] =
7     '{ atan_rom_entry(0), atan_rom_entry(1), atan_rom_entry(2),
8       atan_rom_entry(3), atan_rom_entry(4), atan_rom_entry(5),
9       atan_rom_entry(6), atan_rom_entry(7), atan_rom_entry(8),
10      atan_rom_entry(9), atan_rom_entry(10), atan_rom_entry(11),
11      atan_rom_entry(12), atan_rom_entry(13) };

```

Listing 4.6. A constant function initialising a localparam array

This is the same technique `cordic_pkg` in the *Example Cookbook* uses to build its arctangent table, and it is one place where SystemVerilog and VHDL are genuinely equivalent: a VHDL constant initialised by a pure function does the same job, and does it with better type checking.

The real package differs from the sketch above in three details, and each of them matters. It scales by 2^{15} , not 2^{13} , because the rotator's internal angle format `iangle_t` carries two guard

bits below the external Q2.13 format. It rounds with `$rtoi(x + 0.5)` rather than truncating with `int'(x)`, which is worth up to one LSB on every entry. And it writes a `for` loop instead of fourteen calls. The first difference alone scales the whole table by four: the sketch above starts 6433, 3798, 2006, ... while `ATAN_ROM` starts 25736, 15193, 8027, ... Read the real one before you copy either.

Recipe: Write the loop, not the list

A `for` loop inside a constant function returning an unpacked array is cleaner than the fourteen calls above, and every current simulator and synthesis tool supports it. Write `localparam int ATAN [0:13] = build_atan();` and put the loop in `build_atan`. That is what `cordic_pkg` does. Icarus Verilog is the exception: it cannot return an unpacked array from a function, which is why the *Simulation Cookbook* shows how to generate the same table from Python instead.

4.7 let, 'define, and the preprocessor

Parameters and constant functions cover almost everything a design needs to compute before elaboration. What they do not cover is text, and for text SystemVerilog has a C-style preprocessor. VHDL has nothing of the kind before 2019, and you have not missed much.

`'define` performs textual substitution with no type, no scope and no respect for the hierarchy. A macro defined in one file is visible in every file compiled afterwards in the same compilation unit, which means two IP blocks that both define `'WIDTH` will fight. Use `parameter` and `localparam` for values, `typedef` for types, and reserve `'define` for things a parameter genuinely cannot express: include guards, tool-conditional code, and macros that must expand to declarations.

`let` covers many of the remaining cases with proper scoping and proper argument semantics:

```
1 let handshake(v, r) = v && r;           // scoped, typed, no textual games
2 let is_pow2(n)      = (n & (n-1)) == 0;
3
4 always_comb accept = handshake(req_valid_i, req_ready_o);
```

A `let` is expanded at each use with the arguments bound at that point, so it can refer to signals in the scope where it is used, unlike a function. It is legal in assertions, which is where it earns its keep.

△ Trap: Macros are text, and text does not respect precedence

`'define SUM(a,b) a + b` followed by `x = 'SUM(1,2) * 3;` yields `1 + 2 * 3`, that is 7. Always parenthesise both the arguments and the whole body: `'define SUM(a,b) ((a) + (b))`. This is the oldest bug in C and it arrived in SystemVerilog unchanged.

4.8 generate and bind

Everything so far has changed what you can write *inside* a module. The last two constructs change what you can build *out of* modules, and the second of them is the most useful thing in this chapter.

`generate` needs little explanation, because you already have it: `for`, `if` and `case` generate blocks map one-to-one onto the VHDL constructs of Section 3.4. Two differences are worth

remembering. The loop variable is declared `genvar` rather than being implicit in the loop, and naming the block, `: g_lane` after the loop header, is strongly advised because the name becomes part of the hierarchical path. An unnamed generate block gets a tool-invented name such as `genblk3`, and every waveform path, every `bind` target and every constraint that mentions it then depends on how many unnamed blocks happen to precede it in the file.

`bind` is the interesting one, and it has no VHDL equivalent whatsoever.

```

1 module handshake_chk (input logic clk, rst_n,
2                       input logic valid, ready);
3     property no_drop;
4       @(posedge clk) disable iff (!rst_n)
5         valid && !ready |>= valid;
6     endproperty
7     a_no_drop: assert property (no_drop)
8       else $error("valid dropped before ready");
9 endmodule
10
11 // Attach it to every instance of the module type:
12 bind cordic_rot handshake_chk u_chk_req (
13   .clk(clk_i), .rst_n(rst_ni),
14   .valid(req_valid_i), .ready(req_ready_o));
15
16 // ...or to one specific instance only:
17 bind u_dut.u_cordic handshake_chk u_chk_rsp (
18   .clk(clk_i), .rst_n(rst_ni),
19   .valid(rsp_valid_o), .ready(rsp_ready_i));

```

Listing 4.7. Binding a checker to a design without touching its source

`bind` instantiates a module (or an interface, or a program) *inside* another module's scope, from outside, without editing that module. Inside the bound instance, the target's internal signals are visible by name, so a checker can watch a state register that never reaches a port.

Why this matters: it is the mechanism that makes assertion-based verification possible on RTL you do not own. You receive a block from another team, or from a vendor, and you attach a hundred assertions and a functional coverage model to it in a separate file that your flow can strip out for synthesis. In VHDL your only options are to edit the source or to be content with what the ports expose.

Recipe: Keep bound checkers in their own file list

Put `bind` statements and checker modules in a `*_bind.sv` file compiled only for simulation and formal. The design file list stays clean, the synthesis tool never sees an assertion, and nobody has to remember a `'ifndef SYNTHESIS` guard.

△ Trap: A bound port connection is elaborated in the target's scope

The expressions in a `bind` port list are resolved inside the target module, not where the `bind` statement is written. That is the whole point, and it is also the trap: a name that happens to exist in the target with a different width is connected and silently truncated or extended, exactly as an ordinary port connection would be, and a name that does not exist in the target is an elaboration error pointing at a file that contains no such signal. Connect every port by name, and check the widths by hand the first time.

Two things in Listing 4.7 are not explained here. The `property` and the `assert property` belong to SystemVerilog Assertions, which is the *Verification Cookbook*; and what a bound checker

is actually worth, in a testbench that is already running constrained-random stimulus, is the subject of the *Verification Cookbook*. This chapter owns the construct: `bind` attaches a module to a module. The next two chapters spend it.

4.9 Checklist

- **Use an interface** for any bundle of more than about four signals that crosses more than one module boundary. Give it a `modport` per role.
- **Use a packed struct** wherever you would have written a VHDL record on a port. It crosses a port, it casts to a vector, and the field names survive into the waveform.
- **Give every enumeration a base type.** The tool will not choose one for you the way a VHDL synthesiser does.
- `always_comb`, `always_ff`, `always_latch`, never a bare `always`, in RTL. They are free assertions.
- **unique and priority are promises**, and the synthesiser believes them. Only write one where you can prove it.
- **Compute constants at elaboration** with a constant function rather than pasting a table into the source, and check that your tools accept it before you rely on it.
- **Declare the testbench timing, do not time it by hand.** Where a VHDL testbench writes `wait for 2 ns;` after every edge, an SystemVerilog interface carries a clocking block with default input `#1step` output `#1ns` and the question does not come up again. the *Verification Cookbook* uses it.
- **Name every generate block.** : `g_lane` after the loop header, always. The name becomes the hierarchical path, and `genblk3` is not a path you want in a script.
- **bind a checker** onto somebody else's RTL rather than editing it. There is no VHDL equivalent and it is one of the most useful things in the language.
- **Prefer a parameter to a ``define`.** The preprocessor has no scope and no types.

Quick Reference

The VHDL/SystemVerilog language mapping, compressed onto a few pages to keep next to the keyboard. Everything here is explained properly somewhere in the preceding chapters; this is the version you use once you already understand it.

A.1 The ten things that will bite you first

1. **Widths do not have to match.** $a + b$ with different widths compiles. The result is truncated or extended silently. Lint for it.
2. **One unsigned operand makes the whole expression unsigned.** Mixing logic signed and logic in one expression produces an unsigned result, and your negative numbers become very large positive ones.
3. **$\gg$ is not $\ggg$.** On a signed value you almost always want $\ggg$. VHDL's `shift_right` on a signed is arithmetic by definition and never wrong.
4. **$\leq$ in an `always_comb` block, or $=$ in an `always_ff` block,** will simulate and may even synthesise. It is still wrong. Non-blocking for sequential, blocking for combinational.
5. **A function is static by default.** Declare function automatic unless you know why you want otherwise. The same applies to tasks and to variables inside forked blocks.
6. **An undeclared identifier becomes a one-bit wire.** Put `'default_nettype none` at the top of every file and `'default_nettype wire` at the bottom.
7. **'timescale is positional and inherited.** A module with no preceding directive picks up whatever the last compiled file set. Put one in every file, or set it on the command line.
8. **A class handle is a reference.** `b = a` makes two names for one object. Use `copy()` or a new plus field copy.
9. **randomize() returns a status you must check.** An unsatisfiable constraint set silently leaves the old values in place if you ignore it.
10. **An unsized literal is 32 bits.** `'1` is all ones at the context width; `1` is a 32-bit signed one. They are not the same thing in a concatenation.

A.2 Construct map

VHDL	SystemVerilog	Note
entity + architecture	module ... endmodule	one unit, not two
library/use	import pkg::*	plus 'include
generic	parameter	localparam if not overridable
std_ulogic	logic	4-state, unresolved

VHDL	SystemVerilog	Note
<code>std_logic</code>	<code>wire</code>	resolved; use <code>logic</code> unless there are two drivers
<code>std_logic_vector(7 downto 0)</code>	<code>logic [7:0]</code>	
<code>unsigned/signed</code>	<code>logic [N-1:0] / logic signed [N-1:0]</code>	no separate type
<code>(others => '0')</code>	<code>'0</code>	context-width fill
<code>a & b</code>	<code>{a, b}</code>	concatenation
<code>—</code>	<code>{N{a}}</code>	replication
<code>process (all)</code>	<code>always_comb</code>	<code>always_comb</code> also runs at time 0
<code>process (clk) + rising_edge</code>	<code>always_ff @(posedge clk)</code>	
variable in a process	<code>blocking =</code>	lifetime differs
signal assignment	<code>non-blocking <=</code>	
<code>wait for 10 ns</code>	<code>#10ns</code>	bare #10 takes its unit from 'timescale
<code>wait until cond</code>	<code>wait(cond) or @(...)</code>	
<code>for ...generate</code>	<code>for (genvar ...)</code>	<code>generate</code> optional
<code>if ...generate</code>	<code>if (...) begin : name</code>	name the block
<code>port map (a => x)</code>	<code>.a(x)</code>	<code>.*</code> exists, use sparingly
<code>function</code>	<code>function automatic</code>	
<code>procedure</code>	<code>task or void function</code>	
<code>to_integer(unsigned(v))</code>	<code>int'(v)</code>	implicit in most contexts
<code>resize(v, N)</code>	assignment or cast	implicit and silent
<code>shift_right(s, n)</code>	<code>s >> n</code>	signed operand required
<code>'length, 'range</code>	<code>\$bits, \$size</code>	no 'range equivalent
<code>'event, 'stable</code>	<code>\$rose, \$fell, \$stable</code>	sampled semantics
<code>assert ...severity error</code>	<code>assert (...) else \$error(...)</code>	
<code>report</code>	<code>\$display</code>	
<code>integer'image(x)</code>	<code>%0d</code> in a format string	
<code>std.textio</code>	<code>\$fopen/\$fscanf/\$fdisplay</code>	
<code>record</code>	<code>struct packed</code>	packed struct is a bit vector
<code>type ...is (A, B)</code>	<code>typedef enum logic [1:0] {A, B}</code>	encoding is explicit
<code>protected type</code>	<code>class</code>	class is not synthesisable
<code>configuration</code>	<code>bind / factory override</code>	no direct equivalent
<code>—</code>	<code>interface / modport</code>	no VHDL-2008 equivalent
<code>—</code>	<code>covergroup, constraint, class</code>	verification only
<code>ieee.fixed_pkg</code>	<code>—</code>	track the binary point yourself

A.3 Format string cheat sheet

Specifier	Meaning	Specifier	Meaning
%d	decimal, padded	%0d	decimal, minimum width
%h	hexadecimal	%0h	hexadecimal, no padding
%b	binary	%o	octal
%s	string	%c	character
%f	real, fixed	%e	real, exponential
%g	real, shorter of the two	%t	time, per <code>\$timeformat</code>
%p	any aggregate, pretty	%m	hierarchical name
%v	net strength	%%	a literal per cent sign

A.4 Severity and what it does

VHDL	SystemVerilog	Effect
severity note	\$info	message only
severity warning	\$warning	message only
severity error	\$error	message, counts as a failure
severity failure	\$fatal(n)	message, ends the simulation with status n

Further Reading

Standards

- IEEE Std 1800-2017, *IEEE Standard for SystemVerilog — Unified Hardware Design, Specification, and Verification Language*. The language reference manual. Chapter 9 (processes), Chapter 10 (assignment statements) and Chapter 4 (scheduling semantics) are the three a VHDL designer should read first.
- IEEE Std 1076-2008, *IEEE Standard VHDL Language Reference Manual*, and IEEE Std 1076-2019 for the newer features discussed in Chapter 4.
- IEEE Std 1800.2-2020, *Universal Verification Methodology Language Reference Manual*.
- IEEE Std 1364-2005, the last standalone Verilog standard, useful only when reading legacy code.

Books

- C. Spear and G. Tumbush, *SystemVerilog for Verification*, 3rd ed., Springer. The standard text on the class-based testbench, and the natural sequel to the *Verification Cookbook*.
- S. Sutherland, S. Davidmann and P. Flake, *SystemVerilog for Design*, 2nd ed., Springer. The design-side counterpart, written by people who were in the room when the language was standardised.
- D. Thomas, *The Verilog Hardware Description Language*, and P. Ashenden, *The Designer's Guide to VHDL*. The two classics, for when you want to know why something is the way it is.
- P. Ashenden and J. Lewis, *VHDL-2008: Just the New Stuff*. Short, and it covers most of what Chapter 1 says VHDL can do.
- M. Keating, *The Simple Art of SoC Design*. Not about either language, and worth reading anyway.

Papers and application notes

- C. Cummings, *Nonblocking Assignments in Verilog Synthesis, Coding Styles That Kill!*, SNUG 2000. The clearest published explanation of the blocking/non-blocking rules and the races they exist to prevent. Everything else by the same author on `always_comb`, `unique/priority` and clock-domain crossing is worth the time.
- J. E. Volder, *The CORDIC Trigonometric Computing Technique*, IRE Transactions on Electronic Computers, 1959. The original paper behind the *Example Cookbook*.
- J. S. Walther, *A Unified Algorithm for Elementary Functions*, Spring Joint Computer Conference, 1971. The generalisation.
- R. Andraka, *A Survey of CORDIC Algorithms for FPGA Based Computers*, FPGA 1998. The practical implementation reference, including the gain, the convergence range and the quadrant folding used in the *Example Cookbook*.

Tools and online resources

- *Verilator* manual, <https://verilator.org/guide/>. Read the chapters on the language subset and on the C++ API before writing a C++ testbench.
- *cocotb* documentation, <https://docs.cocotb.org/>.
- *GTKWave* (<https://gtkwave.sourceforge.net>) and *Surfer* (<https://surfer-project.org>), the two waveform viewers that read VCD and FST.
- *OSVVM* (<https://osvvm.org>) and *UVVM* (<https://uvvm.org>), the two VHDL verification frameworks referred to throughout Chapter 1 and the *Verification Cookbook*.
- The *lowRISC* and *OpenTitan* SystemVerilog style guide, <https://github.com/lowRISC/style-guides>. The best free statement of what disciplined SystemVerilog looks like, and the source of the naming conventions used in this book.
- *Verible*, <https://github.com/chipsalliance/verible>, for formatting and linting.

Index

- \$bits, 96
- ‘define, 97

- aggregate, 44
- always_comb, 14, 60, 91
- always_ff, 14, 61, 91
- always_latch, 14, 91
- array
 - packed, 49
 - unpacked, 49
- assertion, 19
 - immediate, 77
- assignment pattern, 44

- bind, 70, 97
- blocking assignment, 11

- case
 - casex, 64
 - casez, 64
- cast
 - dynamic, 54
 - static, 54
- clocking block, 13, 91
- compilation unit, 8, 29, 32
- component, 69
- concatenation, 45
- conditional compilation, 8
- configuration, 23
- constrained random, 17
- context-determined, 46
- continuous assignment, 58

- default_nettype, 6
- defparam, 37
- delay
 - inertial, 59
 - transport, 59
- delta cycle, 9
- design unit, 7, 29
- downto, 40

- enumeration, 53, 95
- event regions, 10

- file I/O, 79
- fixed_pkg, 24
- float_pkg, 24

- foreach, 66
- four-state, 38
- function, 71
- functional coverage, 17

- generate, 67, 97
- generic, 36
- genvar, 67

- IEEE 1076, 2
- IEEE 1364, 2
- IEEE 1800, 3
- implicit net declaration, 5
- import, 31
- initial, 63
- instantiation, 69
- interface, 87

- let, 97
- library
 - VHDL, 31
- linting, 6
- literal
 - sized, 43
 - unsized, 43
- localparam, 36
- logic, 5
- logic type, 38
- loop
 - for, 66
- lowRISC style guide, 7

- macro
 - Verilog, 32
- modport, 88
- module, 30
- multiple drivers, 39

- non-blocking assignment, 11
- numeric_std, 42

- open-source tools, 25
- OpenTitan, 7
- operator overloading, 23
- OSVVM, 19, 25

- package, 8, 74
- parameter, 36

- type, 96
- physical type, 23
- port
 - direction, 33
- preprocessor, 8, 32
- priority, 92
- priority case, 65
- process
 - combinational, 60
- program block, 14
- readmemh, 79
- record, 52
- reg, 5
- replication, 45
- reset
 - asynchronous, 61
- resolution function, 23, 39
- self-determined, 46
- signed, 42
 - expression rules, 48
- std_logic, 5, 38
- struct
 - packed, 52, 93
- system task, 78
- SystemVerilog
 - history, 2
- task, 71
- testbench
 - VHDL, 16
- time
 - units, 80
- timescale directive, 80
- type system, 22
- type() operator, 96
- union
 - packed, 93
- unique, 92
- unique case, 65
- unsigned, 42
- UVVM, 19, 25
- vector range, 40
- Verilator, 6
- Verilog
 - history, 2
- VHDL
 - history, 2
- VHDL-2019, 90
- virtual interface, 89
- wait, 63
- width mismatch, 4
- width rules, 46
- wire, 5